

Big Data

Big data is a term that refers to the large and complex sets of data that are generated from various sources and require advanced techniques and technologies to store, process, analyze, and visualize.

Some of the characteristics of big data are:

- **Volume:** The amount of data that is produced and collected. For example, social media platforms, online transactions, sensors, and IoT devices generate huge volumes of data every day.
- **Variety:** The diversity of data types and formats that are available. For example, big data can include structured data (such as numbers, dates, and categories), unstructured data (such as text, images, and videos), and semi-structured data (such as XML and JSON files).
- **Velocity:** The speed at which data is generated and processed. For example, big data can be produced in real-time or near-real-time, such as streaming data from sensors, web logs, and social media feeds.
- **Veracity:** The quality and reliability of data that is collected and analyzed. For example, big data can be affected by noise, inconsistency, incompleteness, and ambiguity, which can affect the accuracy and validity of the results.
- **Value:** The potential and usefulness of data that is extracted and applied. For example, big data can provide insights, patterns, trends, and predictions that can help in decision making, problem solving, innovation, and optimization.

Some of the benefits of big data are:

- It can help in understanding customer behavior, preferences, and needs, and provide personalized and customized products and services.
- It can help in improving operational efficiency, productivity, and performance, and reduce costs and risks.
- It can help in enhancing innovation, creativity, and competitiveness, and create new business opportunities and models.
- It can help in solving complex and challenging problems, and finding new solutions and discoveries.
- It can help in improving social and environmental outcomes, and addressing global issues and challenges.

Some of the challenges of big data are:

- It requires specialized skills, tools, and infrastructure to manage, process, analyze, and visualize big data.
- It poses ethical, legal, and social issues, such as privacy, security, ownership, and governance of data.
- It demands a cultural and organizational change, such as collaboration, communication, and alignment of goals and strategies.

Unit 1 - Introduction to Big Data

- Big data is a term that refers to the large, complex, and diverse sets of data that are generated at high speed from various sources, such as social media, sensors, web logs, etc.
- Big data has four main characteristics, also known as the 4Vs: volume, variety, velocity, and veracity.
 - Volume: The amount of data that is generated and stored. Big data can range from terabytes to petabytes and beyond.
 - Variety: The different types of data that are collected and analyzed. Big data can include structured, semi-structured, and unstructured data, such as text, images, audio, video, etc.
 - Velocity: The speed at which data is generated and processed. Big data can be produced and consumed in real-time or near-real-time, requiring fast and efficient processing techniques.
 - Veracity: The quality and reliability of data. Big data can be noisy, incomplete, inconsistent, or inaccurate, requiring methods to ensure data quality and trustworthiness.
- Big data poses many challenges and opportunities for businesses, organizations, and society. Some of the challenges include:
 - Data storage and management: How to store and organize big data in a scalable, secure, and cost-effective way.
 - Data processing and analysis: How to extract meaningful insights and value from big data using advanced analytics, machine learning, and artificial intelligence techniques.
 - Data visualization and communication: How to present and communicate the results of big data analysis in an understandable and actionable way.
 - Data privacy and security: How to protect the confidentiality and integrity of big data and the rights and interests of data owners and users.
- Some of the opportunities and benefits of big data include:
 - Improved decision making and problem solving: Big data can help businesses and organizations make better and faster decisions based on data-driven evidence and insights.
 - Enhanced customer experience and satisfaction: Big data can help businesses and organizations understand and anticipate the needs and preferences of their customers and provide personalized and tailored services and products.
 - Increased efficiency and productivity: Big data can help businesses and organizations optimize their operations and processes and reduce costs and waste.
 - Innovation and competitiveness: Big data can help businesses and organizations create new products and services, discover new markets and opportunities, and gain a competitive edge in their industry.
 - Social and environmental impact: Big data can help address some

of the global and local challenges and issues, such as health care, education, security, sustainability, etc.

Types of digital data in big data

Big data refers to the large and complex datasets that are generated from various sources and at high velocities. Big data can be classified into different types based on the format, structure, and accessibility of the data. The following types of digital data in big data are explained below:

- **Structured data:** Any data that can be processed, is easily accessible, and can be stored in a fixed format is called structured data. Structured data follows a predefined schema and can be queried using standard tools and methods. Examples of structured data include relational databases, spreadsheets, and CSV files. Structured data is usually numerical, categorical, or textual in nature.
- **Unstructured data:** Unstructured data in big data is where the data format constitutes multitudes of unstructured files (images, audio, log, and video). Unstructured data does not follow any schema or rules and is difficult to process and analyze using conventional tools and methods. Examples of unstructured data include social media posts, emails, PDF documents, and web pages. Unstructured data is usually rich in media, natural language, or binary content.
- **Semi-structured data:** Semi-structured data in big data is where the data format has some structure or organization, but not enough to fit into a fixed schema. Semi-structured data often contains metadata or tags that describe the data, but the data itself may vary in format and content. Examples of semi-structured data include XML, JSON, and HTML files. Semi-structured data is usually a mix of structured and unstructured data elements.
- **Open data:** Open data in big data is where the data is publicly available and accessible to anyone without any restrictions or limitations. Open data is often published by governments, organizations, or individuals for the purpose of transparency, accountability, or innovation. Examples of open data include census data, weather data, and open source software. Open data is usually structured or semi-structured in format.
- **Dark/lost data:** Dark/lost data in big data is where the data is hidden, inaccessible, or unused by the data owners or users. Dark/lost data is often stored in siloed systems, such as CCTV cameras, legacy databases, or offline devices, that are not connected or integrated with other data sources. Examples of dark/lost data include security footage, medical records, and sensor data. Dark/lost data can be structured, unstructured, or semi-structured in format.
- **Linked data:** Linked data in big data is where the data is transmitted,

shared, or connected across different web platforms and applications using APIs and other methods. Linked data enables the interoperability and integration of data from various sources and domains. Examples of linked data include social media APIs, web services, and semantic web technologies. Linked data is usually semi-structured or unstructured in format.

History of Big Data Innovation

- The concept of big data is relatively new, but the origins of large data sets go back to the 1960s and 1970s when the first data centers and the relational database were developed.
- In 2005, Roger Mougallas from O'Reilly Media coined the term “big data” for the first time, only a year after they created the term “Web 2.0”. It refers to a large set of data that is almost impossible to manage and process using traditional business intelligence tools .
- In 2006, Google published a paper on MapReduce, a programming model for processing large data sets in parallel using clusters of commodity hardware. This inspired the development of Apache Hadoop, an open-source framework for distributed data processing that became widely adopted by big data applications.
- In 2009, the U.S. government launched the Federal Big Data Initiative, a multi-agency effort to invest in big data research and development. The initiative aimed to address the challenges and opportunities of big data in areas such as health care, education, energy, and national security.
- In 2010, IBM announced Watson, a cognitive computing system that can process natural language and answer complex questions using big data. Watson demonstrated its capabilities by winning the Jeopardy! game show against human champions.
- In 2012, the term “big data analytics” became popular, referring to the use of advanced data analytics techniques such as machine learning, predictive modeling, and data visualization to extract insights from big data.
- In 2014, the European Union launched the Horizon 2020 program, which allocated 4.8 billion euros for big data research and innovation. The program aimed to foster the development of a data-driven economy and society in Europe.
- In 2015, the International Data Corporation (IDC) estimated that the global volume of data would grow from 4.4 zettabytes to 44 zettabytes by 2020, driven by the proliferation of internet-connected devices, social media, cloud computing, and the Internet of Things.
- In 2018, the General Data Protection Regulation (GDPR) came into effect in the European Union, imposing strict rules on how personal data can be collected, processed, and shared by organizations. The GDPR aimed to protect the privacy and rights of data subjects and to promote accountability and transparency among data controllers and processors.
- In 2019, the World Economic Forum launched the Global Data Commons,

a platform for sharing and accessing data for public good. The platform aimed to facilitate data collaboration and innovation for addressing global challenges such as climate change, health, and education.

- In 2020, the COVID-19 pandemic highlighted the importance and challenges of big data in public health and crisis management. Big data was used to track and model the spread of the virus, to inform policy decisions and interventions, and to accelerate the development of vaccines and treatments.

Introduction to Big Data Platform

- A big data platform is a type of IT solution that combines the features and capabilities of several big data applications and utilities within a single solution.
- A big data platform acts as an organized storage medium for large amounts of data that are hard to manage with traditional data systems.
- A big data platform usually utilizes a combination of data management hardware and software tools to store aggregated data sets, usually onto the cloud.
- A big data platform also supports custom development, querying and integration with other systems.
- A big data platform enables organizations to develop, deploy, operate and manage a big data infrastructure or environment.
- A big data platform also provides business intelligence and analytics tools to process and analyze the data and derive insights and value from it .
- A big data platform is different from typical data assets because of its volume, complexity and need for advanced business intelligence tools to process and analyze it.
- The attributes that define big data are volume, velocity, variety, veracity and value .
- Some examples of big data platforms are Google Cloud Platform, Amazon Web Services, Microsoft Azure, Hadoop, Spark, MongoDB, etc .

Drivers for Big Data

Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze. Some of the drivers for big data are:

- **The growth of data sources and types:** Data is being generated from various sources, such as social media, sensors, mobile devices, web logs, transactions, etc. These data sources produce different types of data, such as structured, unstructured, semi-structured, streaming, etc. The diversity and volume of data sources and types pose challenges for traditional data management systems and require new approaches and tools to handle big data.

- **The need for faster and better decision making:** Data is a valuable asset for organizations and individuals, as it can provide insights and knowledge that can improve decision making, performance, and innovation. However, to leverage the value of data, it needs to be analyzed and processed quickly and efficiently. Big data analytics can enable faster and better decision making by using advanced techniques, such as machine learning, artificial intelligence, natural language processing, etc. to extract meaningful information and patterns from big data.
- **The emergence of new technologies and platforms:** The development of new technologies and platforms, such as cloud computing, distributed systems, parallel computing, etc. has enabled the storage, processing, and analysis of big data. These technologies and platforms provide scalability, flexibility, and cost-effectiveness for big data applications. They also enable new capabilities, such as real-time analysis, interactive visualization, etc. that can enhance the value and usability of big data.

Big Data Architecture

Big data architecture is the system that supports big data analytics, which is the process of extracting insights from large and complex data sets. Big data architecture is designed to handle the ingestion, processing, and analysis of data that is too large or complex for traditional database systems .

Some of the key components of a big data architecture are :

- **Data sources:** These are the inputs that provide data in various formats, such as structured, unstructured, or semi-structured. Data sources can include databases, files, streams, sensors, web services, etc.
- **Data storage:** This is the layer that receives, stores, and converts the data into a suitable format for processing and analysis. Data storage can include relational databases, NoSQL databases, data lakes, data warehouses, etc.
- **Batch processing:** This is the layer that performs offline processing of large batches of data, typically using distributed computing frameworks such as Hadoop or Spark. Batch processing can perform complex transformations, aggregations, and computations on the data.
- **Stream processing:** This is the layer that performs real-time processing of data streams, typically using frameworks such as Kafka or Storm. Stream processing can perform filtering, enrichment, and analysis of the data as it arrives.
- **Data analysis:** This is the layer that performs various types of analysis on the data, such as descriptive, diagnostic, predictive, or prescriptive. Data analysis can use tools such as SQL, BI, machine learning, or data mining to generate insights, reports, dashboards, or recommendations.
- **Data consumption:** This is the layer that delivers the results of the analysis to the end users, such as business analysts, data scientists, or customers. Data consumption can use tools such as web applications, mobile appli-

cations, APIs, or visualizations to present the data in an interactive and user-friendly way.

Some of the best practices for designing a big data architecture are:

- Define the business goals and requirements clearly and align them with the data sources, storage, processing, and analysis layers.
- Choose the appropriate technologies and tools for each layer based on the data characteristics, such as volume, velocity, variety, veracity, and value.
- Design the data storage layer to support both structured and unstructured data, and use data compression, partitioning, and indexing techniques to optimize performance and scalability.
- Design the data processing layer to support both batch and stream processing, and use parallelization, fault tolerance, and load balancing techniques to ensure reliability and efficiency.
- Design the data analysis layer to support both exploratory and operational analysis, and use machine learning, data mining, and statistical techniques to extract meaningful insights and patterns from the data.
- Design the data consumption layer to support both internal and external users, and use visualization, interactivity, and security techniques to enhance the user experience and trust.

Big data characteristics

Big data is a term that refers to data sets that are too large, complex, or diverse to be processed by conventional methods. Big data can be generated from various sources, such as social media, sensors, web logs, transactions, etc. Big data can provide valuable insights for businesses, governments, and researchers, but it also poses many challenges in terms of storage, analysis, and security.

To understand the nature and scope of big data, it is useful to consider its characteristics, which are often described by the five Vs: volume, variety, velocity, value, and veracity.

- **Volume:** This refers to the amount or size of data that is generated and collected. Big data can range from terabytes to petabytes or even exabytes of data. The volume of data can affect the performance, scalability, and cost of data storage and processing systems.
- **Variety:** This refers to the types or formats of data that are available. Big data can include structured, semi-structured, or unstructured data, such as text, images, audio, video, etc. The variety of data can affect the complexity and compatibility of data integration and analysis methods.
- **Velocity:** This refers to the speed or rate at which data is generated, collected, and processed. Big data can be produced in real-time or near-real-time, such as streaming data from sensors, social media, or web logs. The velocity of data can affect the timeliness and relevance of data analysis and decision making.

- **Value:** This refers to the usefulness or potential of data for achieving a specific goal or outcome. Big data can provide valuable insights for improving business performance, customer satisfaction, social welfare, etc. However, not all data is equally valuable, and some data may be redundant, irrelevant, or inaccurate. Therefore, it is important to assess the quality and relevance of data before investing in data analysis and processing.
- **Veracity:** This refers to the accuracy or reliability of data. Big data can be affected by various sources of uncertainty, such as noise, errors, biases, or fraud. The veracity of data can affect the validity and credibility of data analysis and interpretation. Therefore, it is important to verify and validate the data and the methods used to process and analyze it.

5 Vs of Big Data

Big data is a term that refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to process, analyze and derive insights from. Big data can be characterized by five Vs: volume, velocity, variety, veracity and value. These are explained below:

- **Volume:** This refers to the amount or size of data that is generated and stored. Big data typically involves terabytes, petabytes or even exabytes of data. The volume of data can pose challenges for storage, processing and transmission.
- **Velocity:** This refers to the speed or rate at which data is generated, collected and analyzed. Big data often involves real-time or near-real-time data streams that need to be processed quickly and efficiently. The velocity of data can pose challenges for data quality, latency and scalability.
- **Variety:** This refers to the diversity or heterogeneity of data sources, formats and types. Big data can include structured, semi-structured and unstructured data, such as text, images, audio, video, sensor data, web logs, social media data, etc. The variety of data can pose challenges for data integration, transformation and analysis.
- **Veracity:** This refers to the reliability or trustworthiness of data. Big data can be affected by noise, inconsistency, incompleteness, ambiguity and deception. The veracity of data can pose challenges for data validation, cleaning and quality assurance.
- **Value:** This refers to the usefulness or significance of data for decision making, problem solving and innovation. Big data can provide valuable insights, patterns and trends that can help organizations improve their performance, efficiency and competitiveness. The value of data can pose challenges for data visualization, interpretation and communication.

Big Data Technology Components

Big data technology refers to the technical innovation and super-large datasets that enable automated parallel computation, data management schemes, and data mining. Some of the main components of big data technology are:

- **Machine Learning:** This is the process of creating algorithms and models that can learn from data and make predictions or decisions without being explicitly programmed. Machine learning can be used for various purposes, such as classification, clustering, regression, recommendation, anomaly detection, etc.
- **Natural Language Processing (NLP):** This is the branch of artificial intelligence that deals with the interaction between computers and human languages. NLP can be used for various purposes, such as text analysis, sentiment analysis, text summarization, machine translation, speech recognition, etc.
- **Business Intelligence:** This is the process of collecting, analyzing, and presenting data to support decision making and improve business performance. Business intelligence can be used for various purposes, such as reporting, dashboarding, data visualization, data mining, etc.
- **Cloud Computing:** This is the delivery of computing services, such as servers, storage, databases, networking, software, analytics, etc., over the internet. Cloud computing can be used for various purposes, such as scalability, elasticity, cost-effectiveness, security, etc.
- **Data Sources:** These are the origins of the data that are used for big data solutions. Data sources can be static files produced by applications (such as web server log files), application data stores (such as relational databases), or real-time data sources (such as IoT devices).
- **Data Pipeline:** This is the process of moving and transforming data from data sources to data destinations. Data pipeline can be used for various purposes, such as data ingestion, data integration, data quality, data processing, data storage, etc.
- **Data Warehouse:** This is a centralized repository of structured and integrated data that can be used for analytical queries and reporting. Data warehouse can be used for various purposes, such as data consolidation, data modeling, data mart, data cube, etc.
- **Data Analytics:** This is the process of applying analytical techniques and tools to data to discover insights and generate value. Data analytics can be used for various purposes, such as descriptive analytics, diagnostic analytics, predictive analytics, prescriptive analytics, etc.
- **Data Lake:** This is a distributed repository of raw and unstructured data that can be used for data exploration and machine learning. Data lake can be used for various purposes, such as data ingestion, data catalog, data governance, data security, etc.
- **Data Refinery:** This is a component that cleanses, merges, and transforms data to make it ready for analysis and consumption. Data refinery can be used for various purposes, such as data profiling, data quality, data enrichment, data standardization, etc.
- **Data Delivery:** This is a component that delivers data to the end users or applications in a fast and reliable manner. Data delivery can be used for various purposes, such as data query, data visualization, data streaming, data API, etc.

Big Data importance

Big data is the term used to describe the large and complex datasets that are generated from various sources, such as social media, sensors, transactions, web logs, etc. Big data has the potential to provide valuable insights and solutions for various domains, such as business, health, education, science, and government. However, big data also poses many challenges, such as storage, processing, analysis, security, and privacy.

Some of the benefits and importance of big data are:

- **Cost Savings:** Big data tools like Apache Hadoop, Spark, etc. bring cost-saving benefits to businesses when they have to store and process large amounts of data. These tools can handle data in different formats and scales, and can run on commodity hardware, reducing the need for expensive infrastructure and software licenses.
- **Time-Saving:** Big data tools can also speed up the data processing and analysis, enabling faster and more accurate decision making. For example, big data can help businesses optimize their supply chain, predict customer demand, and improve product quality.
- **Understand the market conditions:** Big data can help businesses understand the market trends, customer preferences, and competitive strategies, and tailor their products and services accordingly. Big data can also help businesses identify new opportunities and niches, and create innovative solutions .
- **Social Media Listening:** Big data can help businesses monitor and analyze the social media activities of their customers and potential customers, and gain insights into their opinions, sentiments, and feedback. This can help businesses improve their customer service, loyalty, and satisfaction, and also generate leads and sales.
- **Boost Customer Acquisition and Retention:** Big data can help businesses segment their customers based on their behavior, preferences, and needs, and offer them personalized and relevant offers, recommendations, and incentives. This can help businesses attract new customers and retain existing ones, and increase their revenue and profitability .
- **Solve Advertisers Problem and Offer Marketing Insights:** Big data can help advertisers and marketers measure the effectiveness and impact of their campaigns, and optimize their strategies and budgets. Big data can also help advertisers and marketers target their audience more precisely and efficiently, and deliver more engaging and persuasive messages.
- **The driver of Innovations and Product Development:** Big data can help businesses and organizations generate new ideas and solutions, and test and validate them quickly and cheaply. Big data can also help businesses and organizations improve their existing products and services, and enhance their quality and performance .
- **Improve patient outcomes by rapidly converting data into in-**

sights: Big data can help healthcare providers and researchers collect and analyze data from various sources, such as electronic health records, medical devices, genomic sequencing, etc. and provide better diagnosis, treatment, and prevention for various diseases and conditions. Big data can also help healthcare providers and researchers discover new drugs and therapies, and improve the quality and safety of healthcare services.

Big Data applications

Big data applications are software systems that use big data technologies to process, analyze and derive insights from large and complex data sets. Big data applications can help companies to make better business decisions, improve customer experience, optimize operations, reduce costs and risks, and create new products and services. Some of the domains where big data applications are widely used are:

- **Healthcare:** Big data applications can help healthcare providers to improve patient care, diagnosis, treatment, and prevention. For example, big data applications can analyze electronic health records, medical images, genomic data, wearable devices, and social media data to identify patterns, trends, and anomalies in health conditions and outcomes. Big data applications can also help healthcare organizations to manage resources, reduce errors, and comply with regulations.
- **Finance:** Big data applications can help financial institutions to enhance customer service, detect fraud, manage risk, and optimize performance. For example, big data applications can analyze transaction data, customer behavior data, market data, and social media data to provide personalized recommendations, offers, and services to customers. Big data applications can also help financial institutions to monitor and prevent fraudulent activities, assess creditworthiness, and comply with regulations.
- **Retail:** Big data applications can help retailers to understand customer preferences, behavior, and feedback, and to offer personalized and relevant products, services, and experiences. For example, big data applications can analyze customer data, purchase data, inventory data, and social media data to provide tailored recommendations, promotions, and loyalty programs to customers. Big data applications can also help retailers to optimize pricing, merchandising, supply chain, and store operations.
- **Manufacturing:** Big data applications can help manufacturers to improve product quality, efficiency, and innovation. For example, big data applications can analyze sensor data, machine data, production data, and customer data to monitor and optimize the performance, maintenance, and reliability of machines and equipment. Big data applications can also help manufacturers to design and develop new products, processes, and services based on customer feedback and market trends.
- **Education:** Big data applications can help educators and learners to enhance learning outcomes, engagement, and retention. For example, big

data applications can analyze student data, course data, assessment data, and feedback data to provide personalized and adaptive learning paths, content, and support to learners. Big data applications can also help educators to monitor and improve student progress, performance, and satisfaction.

Big Data features – security, compliance, auditing and protection

- **Security:** The ability to protect the data from unauthorized access, modification, or deletion. Security features include encryption, authentication, authorization, firewall, and intrusion detection and prevention systems.
- **Compliance:** The ability to adhere to the rules and regulations that govern the data collection, processing, storage, and usage. Compliance features include data governance, data quality, data lineage, and data retention policies.
- **Auditing:** The ability to monitor and record the activities and events that occur on the data and the systems that handle the data. Auditing features include logging, auditing trails, and reporting.
- **Protection:** The ability to ensure the availability, integrity, and resilience of the data and the systems that handle the data. Protection features include backup, recovery, replication, and disaster recovery.

Security of Big Data

- Big data security is the collective term for all the measures and tools used to guard both the data and analytics processes from attacks, theft, or other malicious activities that could harm or negatively affect them.
- Big data security is concerned with attacks that originate either from the online or offline spheres, and it covers data in transit and at rest, as well as data across different platforms and devices .
- Big data security faces several challenges, such as:
 - The volume, variety, and velocity of big data make it difficult to monitor and protect using traditional methods .
 - The distributed nature of big data frameworks, such as Hadoop, introduces new vulnerabilities and risks, such as unauthorized access, data leakage, and data tampering .
 - The lack of standardization and regulation of big data security practices and policies across different industries and organizations .
- Big data security requires several best practices, such as:
 - Safeguarding distributed programming frameworks by implementing encryption, authentication, authorization, auditing, and logging mechanisms .
 - Securing non-relational data by applying data masking, tokenization, access control, and encryption techniques .
 - Securing data storage and processing by using secure cloud services, encryption keys, firewalls, and intrusion detection and prevention

- systems .
- Securing data transmission by using secure protocols, encryption, and VPNs .
- Securing data analytics by ensuring data quality, integrity, and privacy, and by applying data anonymization, pseudonymization, and differential privacy techniques .
- Securing data governance by establishing clear roles, responsibilities, and policies for data ownership, access, usage, and disposal, and by enforcing compliance with relevant laws and regulations .

Compliance of Big Data

- Compliance of big data refers to the process of ensuring that the collection, processing, storage, and use of big data adhere to the relevant laws, regulations, and ethical standards.
- Compliance of big data is important for several reasons, such as:
 - Protecting the privacy and security of personal data and sensitive information from unauthorized access, misuse, or breach.
 - Preventing or detecting criminal activities, such as fraud, money laundering, or cyberattacks, using big data analytics and predictive models.
 - Enhancing the trust and confidence of customers, partners, regulators, and stakeholders in the organization's data governance and management practices.
 - Avoiding legal risks, penalties, or reputational damage due to non-compliance or violation of data protection laws, such as the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA).
- Compliance of big data involves several challenges and best practices, such as:
 - Data security: Implementing appropriate technical and organizational measures to safeguard the confidentiality, integrity, and availability of big data, such as encryption, authentication, authorization, backup, and recovery.
 - Transparency: Informing the data subjects about the purpose, scope, and duration of the data processing, as well as their rights and choices regarding their data, such as access, rectification, erasure, or portability.
 - Data minimization: Collecting and retaining only the data that is necessary, relevant, and adequate for the specified purpose, and deleting or anonymizing the data when it is no longer needed or required.
 - Data quality: Ensuring that the data is accurate, complete, consistent, and up-to-date, and correcting or deleting any inaccurate or outdated data.
 - Data ethics: Respecting the dignity, autonomy, and interests of the data subjects, and avoiding any harm, discrimination, or bias that

may result from the data processing or analysis.

Auditing of Big Data

- Big data is the term used to describe the collection, processing, and analysis of large and complex data sets that exceed the capabilities of traditional data management tools .
- Big data has many applications in various fields, such as business, health, education, and government, and can provide valuable insights for decision making, performance improvement, and risk management .
- Auditing of big data is the process of examining and evaluating the quality, reliability, and security of big data and the systems that generate, store, and analyze it .
- Auditing of big data can help organizations to ensure compliance with regulations, standards, and policies, to identify and mitigate risks, to improve efficiency and effectiveness, and to enhance stakeholder confidence and trust .
- Auditing of big data poses new challenges and opportunities for auditors, as they need to adapt their skills, methods, and tools to deal with the volume, variety, velocity, and veracity of big data .
- Auditors can use big data to expand the scope of their projects and draw comparisons over larger populations of data, to apply unique analytics techniques and artificial intelligence to uncover patterns, trends, and anomalies, and to deliver reports that are more timely, relevant, and actionable .
- Auditors also need to consider the ethical, legal, and social implications of big data, such as data privacy, data ownership, data quality, data governance, and data security, and to ensure that big data is used in a responsible and transparent manner .
- Auditing of big data requires the participation and collaboration of internal and external auditors, management, IT, and other stakeholders, to establish a common understanding, a clear vision, and a robust framework for big data in the organization .

Protection of Big Data

Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, analyze, and share. Big data has many potential benefits for businesses, governments, and society, such as improving decision making, enhancing customer experience, increasing efficiency, and creating new value. However, big data also poses significant challenges and risks for data privacy and security, as it may contain sensitive and personal information that can be misused, breached, or exploited by unauthorized parties.

To protect the privacy of big data, it is important to adopt a comprehensive and proactive approach that covers the following aspects :

- **Data collection:** Data collection should be based on the principles of data minimization, purpose limitation, and consent. This means that only the necessary and relevant data should be collected for specific and legitimate purposes, and that the data subjects should be informed and given the choice to agree or disagree with the data collection and processing.
- **Data retention and archiving:** Data retention and archiving should be based on the principles of storage limitation, integrity, and confidentiality. This means that the data should be kept only for as long as it is needed for the purposes for which it was collected, and that the data should be protected from unauthorized access, modification, or deletion.
- **Data use:** Data use should be based on the principles of data quality, accuracy, and transparency. This means that the data should be accurate, complete, and up to date, and that the data subjects should be informed and given the right to access, rectify, erase, or restrict the processing of their data. Data use should also respect the data subjects' rights and preferences, such as the right to object, the right to data portability, and the right to withdraw consent. Data use should also include data masking, encryption, anonymization, or pseudonymization techniques to reduce the risk of re-identification or linkage of the data.
- **Data disclosure:** Data disclosure should be based on the principles of accountability, responsibility, and compliance. This means that the data controllers and processors should be able to demonstrate how they comply with the applicable data protection laws and regulations, such as the General Data Protection Regulation (GDPR) in the European Union, and that they should be liable for any data breaches or violations that may occur. Data disclosure should also follow the data subjects' requests and the data protection authorities' orders, and should notify the data subjects and the authorities in case of any data breaches or incidents.

By following these principles and practices, big data can be protected and used in a responsible and ethical manner, while respecting the data subjects' rights and interests.

Big Data Privacy

Big data privacy is the practice of properly managing large and complex data sets to minimize risk and protect sensitive data. Big data privacy is important for both data owners and data users, as it involves ethical, legal, and social implications. Some of the key aspects of big data privacy are:

- **Data collection:** Data owners should collect only the data that is necessary and relevant for their purposes, and obtain consent from the data subjects whenever possible. Data owners should also inform the data subjects about the purpose, scope, and duration of the data collection, and respect their rights to access, correct, or delete their data.
- **Data retention and archiving:** Data owners should store and archive the data securely, and delete or anonymize the data when it is no longer

needed or requested by the data subjects. Data owners should also comply with the applicable data retention laws and regulations in their jurisdictions.

- **Data use:** Data owners and users should use the data only for the intended and authorized purposes, and avoid any misuse, abuse, or discrimination based on the data. Data owners and users should also implement data protection measures, such as encryption, pseudonymization, or masking, to prevent unauthorized access, disclosure, or modification of the data.
- **Data disclosure:** Data owners and users should disclose the data only to the authorized parties, and respect the privacy preferences and expectations of the data subjects. Data owners and users should also notify the data subjects and the relevant authorities in case of any data breach or incident that may compromise the data privacy.

Big data privacy is also a matter of customer trust and social responsibility. The more data that is collected and analyzed, the more insights and value that can be derived from it, but also the more risks and challenges that arise for the data privacy. Therefore, data owners and users should adopt a proactive and transparent approach to big data privacy, and balance the benefits and harms of big data for themselves and the society.

Big Data Ethics

Big data ethics is the study of the moral principles and values that guide the collection, analysis, and use of large amounts of data, especially personal data. Big data ethics is important because it helps to ensure that data is used in a responsible, fair, and beneficial way, and that the rights and interests of data subjects are respected and protected.

Some of the main topics and challenges in big data ethics are:

- **Ownership:** Who owns the data and who has the right to access, modify, or delete it? How can data subjects exercise control over their own data and benefit from its use?
- **Consent:** How can data subjects give informed and voluntary consent to the collection and use of their data? How can consent be obtained, recorded, and revoked? How can consent be meaningful and not coerced or manipulated?
- **Transparency:** How can data collectors and users disclose their data practices and purposes to data subjects and other stakeholders? How can data subjects access and understand how their data is used and by whom? How can data subjects request corrections or deletions of their data?
- **Privacy:** How can data collectors and users protect the confidentiality, integrity, and availability of data? How can data collectors and users minimize the collection and retention of data and avoid unnecessary or excessive data sharing? How can data collectors and users prevent or mitigate data breaches, leaks, or misuse?

- **Security:** How can data collectors and users safeguard data from unauthorized access, modification, or destruction? How can data collectors and users implement appropriate technical and organizational measures to ensure data security? How can data collectors and users respond to and report data incidents?
- **Quality:** How can data collectors and users ensure the accuracy, completeness, and relevance of data? How can data collectors and users avoid or correct data errors, biases, or inconsistencies? How can data collectors and users ensure the validity and reliability of data analysis and interpretation?
- **Accountability:** How can data collectors and users be held responsible for their data practices and outcomes? How can data collectors and users comply with relevant laws, regulations, and ethical standards? How can data collectors and users monitor and audit their data activities and address any complaints or concerns?

Big Data Analytics

- Big data analytics is the process of extracting useful insights from large and complex data sets that are often difficult to handle by traditional data processing methods.
- Big data analytics can help organizations to improve their performance, enhance their decision making, discover new opportunities, and gain competitive advantage.
- Big data analytics involves four main steps: data acquisition, data storage, data processing, and data analysis.
- Data acquisition is the process of collecting and ingesting data from various sources, such as sensors, web logs, social media, transactions, etc.
- Data storage is the process of organizing and storing data in a suitable format and location, such as databases, data warehouses, data lakes, cloud storage, etc.
- Data processing is the process of transforming and preparing data for analysis, such as cleaning, filtering, aggregating, joining, etc.
- Data analysis is the process of applying various techniques and tools to explore, visualize, and extract insights from data, such as statistics, machine learning, data mining, natural language processing, etc.
- Big data analytics can be classified into three types: descriptive analytics, predictive analytics, and prescriptive analytics.
- Descriptive analytics is the process of summarizing and presenting the current state of data, such as dashboards, reports, charts, etc.
- Predictive analytics is the process of using historical data to forecast future outcomes and trends, such as regression, classification, clustering, etc.
- Prescriptive analytics is the process of using data to recommend optimal actions and solutions, such as optimization, simulation, decision support, etc.
- Big data analytics can be applied to various domains and industries, such

as healthcare, retail, finance, education, manufacturing, etc.

Challenges of conventional systems compared to Big Data

- Conventional systems are designed to work with structured data, which have a fixed schema and format. Big data, on the other hand, can be structured, semi-structured, or unstructured, and can vary in size, velocity, and variety. This makes it difficult for conventional systems to store, process, and analyze big data efficiently and effectively .
- Conventional systems rely on relational databases, which have limitations in scalability, performance, and flexibility. Big data requires distributed and parallel processing, which can handle large volumes of data across multiple nodes. Big data also requires different types of data storage, such as NoSQL databases, Hadoop Distributed File System (HDFS), or cloud storage, which can handle different types of data and queries .
- Conventional systems have difficulty in maintaining the accuracy and confidentiality of the data, as the quality of the data is high and the cost of storing such massive quantity of data is expensive. Big data, on the other hand, can leverage techniques such as data cleansing, data compression, data encryption, and data anonymization, which can improve the data quality and security, and reduce the storage cost .
- Conventional systems have limited analytical capabilities, as they can only perform predefined and structured queries on the data. Big data, on the other hand, can use advanced analytical tools and techniques, such as machine learning, data mining, natural language processing, and sentiment analysis, which can extract valuable insights and patterns from the data, and support complex and dynamic queries .
- Conventional systems are less flexible and adaptable, as they require predefined data models and schemas, and are difficult to change or update. Big data, on the other hand, can be more flexible and adaptable, as it can handle different types of data sources and formats, and can be easily integrated and transformed .

Intelligent data analysis in Big Data

- Intelligent data analysis (IDA) is one of the most important approaches in the field of data mining, which attracts great concerns from the researchers.
- IDA aims to extract underlying patterns, trends, and insights from large, complex, and heterogeneous data sets, using advanced analytical techniques, such as data mining, statistical analysis, and predictive modeling .
- Big data is a term that refers to the massive volume, velocity, and variety of data that is generated by various sources, such as social media, sensors, mobile devices, and online transactions.
- Big data poses significant challenges for traditional data analysis methods,

such as scalability, complexity, heterogeneity, and quality.

- Big data analytics (BDA) is the application of IDA on big data sets, using computational intelligence techniques, such as artificial intelligence (AI), machine learning, and deep learning, to transform raw data into valuable insights for decision making .
- AI makes big data analytics simpler by automating and enhancing data preparation, data visualization, predictive modeling, and other complex analytical tasks that would otherwise be labor-intensive and time-consuming.
- AI helps users work with, manipulate, and surface actionable insights faster from large, complex data sets.
- BDA can provide various benefits for different domains and applications, such as business intelligence, customer behavior analysis, fraud detection, health care, smart cities, and recommender systems .
- BDA can also help to improve the performance, efficiency, and innovation of organizations and industries, by enabling data-driven decision making, optimizing business processes, and creating new products and services .
- BDA requires a new theoretical understanding of the capabilities, challenges, and opportunities of IDA in the big data environment, as well as the development of novel methods, tools, and frameworks for effective and efficient data analysis.

Nature of data in Big Data

Big data is a term that refers to the large and complex data sets that are collected and analyzed by organizations for various purposes, such as business intelligence, machine learning, predictive modeling and advanced analytics. Big data has the following characteristics:

- **Volume:** Big data is measured in terabytes, petabytes or even exabytes of data. The volume of data is increasing rapidly due to the proliferation of digital devices, sensors, social media platforms and other sources of data generation.
- **Velocity:** Big data is generated and processed at high speed, often in real time or near real time. The velocity of data is important for applications that require timely and accurate insights, such as fraud detection, online recommendation systems and smart city management.
- **Variety:** Big data encompasses a wide range of data types, such as structured, unstructured and semistructured data. Structured data is organized in a predefined format, such as tables and records. Unstructured data is not organized in a predefined format, such as text, images, videos and audio. Semistructured data is a hybrid of structured and unstructured data, such as web logs, JSON and XML files.
- **Veracity:** Big data is often noisy, incomplete, inconsistent or inaccurate, which affects the quality and reliability of the data. Veracity of data is important for applications that require high precision and accuracy, such

as health care, finance and security.

- **Value:** Big data has the potential to provide valuable insights and opportunities for organizations, such as improving customer satisfaction, increasing operational efficiency, reducing costs and risks, and creating new products and services. Value of data is determined by the usefulness and relevance of the data for the specific purpose and context.

Analytic processes and tools for Big Data

Big data analytics is the process of uncovering trends, patterns, and correlations in large amounts of raw data to help make data-informed decisions. Big data analytics can be used for various purposes, such as customer behavior analysis, fraud detection, market segmentation, sentiment analysis, and more.

The analytic processes and tools for big data can be divided into four main steps:

1. **Collection and storage:** This step involves acquiring and storing the data from various sources, such as web logs, social media, sensors, etc. The data can be structured, semi-structured, or unstructured, and may have different formats, such as text, images, audio, video, etc. Some of the common tools and technologies for collection and storage of big data are:
 - **Hadoop:** An open-source framework that efficiently stores and processes large data sets through a distributed computing environment. Hadoop consists of several components, such as HDFS (Hadoop Distributed File System), MapReduce (a programming model for parallel processing), YARN (a resource management system), and Hive (a data warehouse system) .
 - **NoSQL databases:** Non-relational data management systems that do not require a fixed schema, making them a great choice for handling unstructured or semi-structured data. Some examples of NoSQL databases are MongoDB, Cassandra, Redis, and CouchDB .
 - **Cloud storage:** A service that provides scalable, reliable, and cost-effective storage of data on remote servers. Cloud storage can be accessed through the internet, and can offer various benefits, such as elasticity, security, and backup. Some examples of cloud storage providers are Amazon S3, Google Cloud Storage, and Microsoft Azure Blob Storage .
2. **Processing:** This step involves transforming and integrating the data into a suitable format for analysis. The processing can involve various tasks, such as cleaning, filtering, aggregating, joining, and enriching the data. Some of the common tools and technologies for processing big data are:
 - **Data integration software:** A software that enables the extraction, transformation, and loading (ETL) of data from different sources and formats. Data integration software can help automate and stream-

line the data processing workflow, and ensure data quality and consistency. Some examples of data integration software are Talend, Informatica, and Pentaho .

- **Data preprocessing and scrubbing tools:** A software that helps prepare the data for analysis by removing noise, outliers, missing values, duplicates, and errors. Data preprocessing and scrubbing tools can improve the accuracy and efficiency of the analysis, and reduce the risk of bias and errors. Some examples of data preprocessing and scrubbing tools are Trifacta, Dataiku, and OpenRefine .
3. **Analysis:** This step involves applying various techniques and methods to discover insights and patterns in the data. The analysis can involve various types of analytics, such as descriptive, diagnostic, predictive, and prescriptive. Some of the common tools and technologies for analysis of big data are:
- **Data mining:** A process of extracting useful information and knowledge from large and complex data sets. Data mining can use various techniques, such as classification, clustering, association, regression, and anomaly detection, to identify patterns, trends, and relationships in the data .
 - **Artificial intelligence (AI):** A branch of computer science that aims to create systems and machines that can perform tasks that normally require human intelligence, such as reasoning, learning, and decision making. AI can use various methods, such as machine learning, deep learning, natural language processing, computer vision, and speech recognition, to analyze and understand the data, and generate outputs, such as predictions, recommendations, and actions .
 - **Statistical analysis:** A process of applying statistical methods and techniques, such as hypothesis testing, correlation, regression, and ANOVA, to analyze and interpret the data, and draw conclusions and inferences. Statistical analysis can help measure the significance, reliability, and validity of the results, and test the assumptions and hypotheses .
4. **Visualization and communication:** This step involves presenting and communicating the results and insights of the analysis in a clear and effective way. The visualization and communication can involve various tools and techniques, such as charts, graphs, dashboards, reports, and stories, to convey

Analysis vs Reporting in Big Data

- Analysis and reporting are two essential aspects of Big Data that help to derive insights and value from large and complex datasets.
- Analysis is the process of exploring, transforming, modeling, and interpreting data to answer specific questions, test hypotheses, or discover patterns

and trends.

- Reporting is the process of presenting, visualizing, and communicating data and analysis results to inform decision-making, monitor performance, or share knowledge.
- Analysis and reporting can be performed using various tools, techniques, and frameworks, such as SQL, Python, R, Spark, Hadoop, Tableau, Power BI, etc.
- Analysis and reporting can have different goals, audiences, and formats, depending on the context and the use case.
- Some examples of analysis and reporting in Big Data are:
 - Business intelligence (BI): BI is the process of using data to support strategic and operational decisions, such as identifying market opportunities, optimizing processes, or improving customer satisfaction. BI involves both analysis and reporting, as it requires collecting, processing, and analyzing data, and then presenting and delivering the insights to the relevant stakeholders, such as managers, executives, or customers.
 - Data science: Data science is the interdisciplinary field that applies scientific methods, algorithms, and systems to extract knowledge and insights from data. Data science involves both analysis and reporting, as it requires applying advanced techniques, such as machine learning, artificial intelligence, or natural language processing, to solve complex problems, and then communicating and visualizing the findings and recommendations to the end-users, such as researchers, engineers, or policymakers.
 - Data journalism: Data journalism is the practice of using data to enhance, inform, or create news stories, such as investigative reports, interactive features, or data-driven narratives. Data journalism involves both analysis and reporting, as it requires finding, verifying, and analyzing data, and then storytelling and publishing the stories to the public, such as readers, viewers, or listeners.

Modern data analytic tools for Big Data

Big data analytics is the process of extracting insights from large and complex datasets using various methods, techniques, and tools. Big data analytics can help businesses make better decisions, improve operations, enhance customer experience, and gain a competitive edge.

There are many tools available for big data analytics, each with its own features, advantages, and limitations. Some of the most popular and widely used tools are:

- **Apache Hadoop:** Apache Hadoop is a Java-based open-source frame-

work that allows distributed processing of large datasets across clusters of computers. Hadoop consists of four main components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common. Hadoop can handle structured, semi-structured, and unstructured data, and supports various programming languages and applications. Hadoop is scalable, fault-tolerant, and cost-effective. However, Hadoop also has some drawbacks, such as complexity, security issues, and lack of real-time processing .

- **KNIME:** KNIME is an open-source platform for data integration, analysis, and exploration. KNIME provides a graphical user interface (GUI) that allows users to create workflows by dragging and dropping nodes. KNIME supports various data sources, formats, and types, and offers a wide range of data manipulation, visualization, and machine learning nodes. KNIME also has a marketplace where users can download extensions and plugins for additional functionality. KNIME is user-friendly, flexible, and extensible. However, KNIME also has some limitations, such as performance issues, memory consumption, and lack of documentation.
- **OpenRefine:** OpenRefine is an open-source tool for cleaning, transforming, and enriching data. OpenRefine allows users to import data from various sources, such as CSV, JSON, XML, and web APIs, and apply various operations, such as filtering, clustering, splitting, merging, and reconciling. OpenRefine also supports data quality assessment, data profiling, and data augmentation using external services, such as Google Refine Expression Language (GREL), Wikidata, and OpenStreetMap. OpenRefine is powerful, versatile, and interactive. However, OpenRefine also has some drawbacks, such as limited scalability, lack of automation, and steep learning curve .
- **Orange:** Orange is an open-source tool for data mining and machine learning. Orange provides a GUI that allows users to create workflows by connecting widgets. Orange supports various data sources, formats, and types, and offers a rich set of widgets for data preprocessing, visualization, modeling, evaluation, and interpretation. Orange also has a Python scripting environment that allows users to extend the functionality of the tool. Orange is intuitive, comprehensive, and customizable. However, Orange also has some limitations, such as memory issues, compatibility issues, and lack of advanced features .
- **RapidMiner:** RapidMiner is a platform for data science and machine learning. RapidMiner consists of three main components: RapidMiner Studio, RapidMiner Server, and RapidMiner Go. RapidMiner Studio is a GUI that allows users to create workflows by dragging and dropping operators. RapidMiner Server is a server-based component that allows users to deploy, manage, and share workflows. RapidMiner Go is a web-based component that allows users to create and run workflows without coding. RapidMiner supports various data sources, formats, and types, and offers a wide range of data preparation, analysis, and modeling operators. RapidMiner is easy to use, robust, and scalable. However, RapidMiner

also has some drawbacks, such as high cost, limited documentation, and steep learning curve .

- **R-programming:** R is a programming language and environment for statistical computing and graphics. R allows users to manipulate, analyze, and visualize data using various packages, functions, and libraries. R supports various data sources, formats, and types, and offers a rich set of tools for data exploration, modeling, testing, and reporting. R is powerful, flexible, and open-source. However, R also has some drawbacks, such as memory constraints, security risks, and lack of standardization .
- **Datawrapper:** Datawrapper is a web-based tool for creating interactive charts and maps. Datawrapper allows users to import data from various sources, such as CSV, Excel, Google Sheets, and web APIs, and choose from various chart and map types, such as bar, line,

Unit 2 - Hadoop and Map Reduce

- Hadoop is a platform that allows distributed storage and processing of large-scale data using clusters of commodity hardware .
- MapReduce is a programming model that enables massive scalability across hundreds or thousands of servers in a Hadoop cluster. It is the heart of Apache Hadoop.
- The term “MapReduce” refers to two separate and distinct tasks that Hadoop programs perform:
 - The map job: This is where a set of data is converted into another set of data, where individual elements are broken down into tuples (key/value pairs).
 - The reduce job: This is where the output of the map job is combined to produce a smaller set of tuples, which are the final result.
- During a MapReduce job, Hadoop sends the Map and Reduce tasks to the appropriate servers in the cluster. The framework manages all the details of data-passing such as issuing tasks, verifying task completion, and copying data around the cluster between the nodes.
- MapReduce can process vast amounts of data (multi-terabyte data-sets) in-parallel on large clusters (thousands of nodes) of commodity hardware in a reliable, fault-tolerant manner .
- MapReduce is suitable for applications that have the following characteristics :
 - The input data is large and can be split into independent chunks that can be processed by the map tasks in parallel.
 - The output data can be reduced to a smaller set of key/value pairs that can be combined by the reduce tasks in parallel.
 - The intermediate results of the map tasks can be grouped by key and shuffled across the network to the reduce tasks.
 - The logic of the application can be expressed as a map function and a reduce function.
- Some examples of applications that can use MapReduce are :

- Word count: Counting the frequency of words in a large collection of documents.
- Inverted index: Building an index of words and the documents that contain them for a search engine.
- PageRank: Computing the importance of web pages based on the links between them.
- K-means clustering: Finding groups of similar data points in a large data set.
- Matrix multiplication: Performing algebraic operations on large matrices.

Hadoop

- Hadoop is an open-source software framework for storing data and running applications on clusters of commodity hardware .
- Hadoop provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs .
- Hadoop allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage .
- Hadoop can efficiently store and process large datasets ranging in size from gigabytes to petabytes of data.
- Hadoop consists of several modules, such as Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, Hadoop YARN, and Hadoop Subprojects .
- Hadoop Common provides the common utilities and libraries that support other Hadoop modules.
- HDFS is a distributed file system that stores data on the cluster nodes, providing high-throughput access to the data.
- MapReduce is a programming model and software framework for writing applications that process large amounts of data in parallel on the cluster nodes.
- YARN is a resource management platform that manages the compute resources and schedules the applications running on the cluster.
- Hadoop Subprojects are additional components that extend the functionality of Hadoop, such as Hadoop ZooKeeper, Hadoop Hive, Hadoop HBase, Hadoop Spark, etc .

History of Hadoop

- Hadoop is an open-source software framework for storing and processing big data. It was created by Apache Software Foundation in 2006, based on a white paper written by Google in 2003 that described the Google

File System (GFS) and the MapReduce programming model.

- Hadoop was started with Doug Cutting and Mike Cafarella in the year 2002 when they both started to work on Apache Nutch project. Apache Nutch project was the process of building a search engine system that can index 1 billion pages.
- Hadoop was named after an extinct specie of mammoth, a so called Yellow Hadoop. The name was given by Doug Cutting's son, who had a toy elephant with that name.
- In 2008, Hadoop defeated the supercomputers and became the fastest system on the planet for sorting terabytes of data. It sorted 1 terabyte of data in 209 seconds, beating the previous record of 297 seconds.
- The very first design document for the Hadoop Distributed File System (HDFS) was written by Dhruba Borthakur in 2007. HDFS is a distributed and scalable file system that stores data across multiple nodes in a cluster.
- Hadoop has evolved over the years with the addition of various components and subprojects that form the Hadoop ecosystem. Some of the major components are HBase, Hive, Pig, Spark, Flume, Sqoop, Oozie, ZooKeeper, etc.

Apache Hadoop

- Apache Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation .
- Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models .
- Apache Hadoop is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
- Apache Hadoop consists of four main modules: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN.
- Hadoop Common contains the common utilities and libraries that support other Hadoop modules.
- HDFS is a distributed file system that provides high-throughput access to application data and can store large amounts of data across multiple machines.
- Hadoop MapReduce is a programming model and software framework for writing applications that process large amounts of data in parallel on clusters of nodes.
- Hadoop YARN is a resource management platform that manages the computing resources and schedules the execution of applications on the Hadoop cluster.

Hadoop Distributed File System

- Hadoop Distributed File System (HDFS) is a file system that provides scalable, reliable, and fault-tolerant storage for large-scale data processing applications.
- HDFS is designed to run on clusters of commodity hardware, and can store petabytes of data across thousands of nodes.
- HDFS follows a master-slave architecture, where a single NameNode manages the namespace and metadata of the file system, and multiple DataNodes store the actual data blocks.
- HDFS supports a write-once-read-many model, where files are split into fixed-size blocks (typically 128 MB) and replicated across multiple DataNodes for fault tolerance and high availability.
- HDFS provides a Java-based API for clients to interact with the file system, as well as a web-based interface and a command-line interface.
- HDFS supports several features, such as:
 - Rack awareness: HDFS can optimize the placement of data blocks based on the network topology of the cluster, and can tolerate rack failures by maintaining multiple replicas across different racks.
 - Snapshots: HDFS can create point-in-time copies of directories or files, which can be used for backup, disaster recovery, or testing purposes.
 - Federation: HDFS can scale horizontally by allowing multiple NameNodes to operate independently, each managing a separate namespace and a subset of DataNodes.
 - High Availability: HDFS can eliminate the single point of failure of the NameNode by using a pair of NameNodes in an active-standby configuration, and using a shared storage or a quorum journal manager to synchronize their states.
 - Erasure coding: HDFS can reduce the storage overhead of replication by using erasure coding, which splits data blocks into smaller fragments and encodes them with parity information, such that the original data can be recovered from a subset of the fragments.
 - Encryption: HDFS can encrypt data at rest and in transit, using keys managed by a key management server, and using transparent encryption zones to specify which directories or files should be encrypted.

Components of Hadoop

Hadoop is a framework for distributed processing of large-scale data sets using clusters of commodity hardware. It consists of the following core components:

- **Hadoop Distributed File System (HDFS):** This is the storage layer of Hadoop that provides a reliable and scalable file system for storing and accessing data across multiple nodes. HDFS has two main components: NameNode and DataNode. NameNode is the master node that manages the metadata and namespace of the file system, while DataNode is the

worker node that stores the actual data blocks. HDFS supports high availability, fault tolerance, and data replication.

- **Hadoop MapReduce:** This is the processing layer of Hadoop that provides a programming model for parallel processing of data using key-value pairs. MapReduce consists of two phases: Map and Reduce. Map phase takes the input data and transforms it into intermediate key-value pairs, while Reduce phase aggregates the intermediate key-value pairs based on the keys and produces the final output. MapReduce supports distributed execution, load balancing, and fault tolerance.
- **Hadoop YARN:** This is the resource management layer of Hadoop that provides a framework for scheduling and handling resource requests from distributed applications. YARN consists of two components: ResourceManager and NodeManager. ResourceManager is the master node that allocates and monitors the resources in the cluster, while NodeManager is the worker node that manages the resources and executes the tasks on each node. YARN supports multiple applications, dynamic resource allocation, and isolation.

Besides these core components, Hadoop also has a rich ecosystem of tools and applications that help collect, store, process, analyze, and manage big data. Some of these are:

- **Apache Pig:** This is a scripting language for data analysis that allows users to write complex data transformations using a high-level syntax. Pig runs on top of Hadoop and converts the scripts into MapReduce jobs.
- **Apache Hive:** This is a data warehouse system for data analysis that allows users to query and analyze structured and semi-structured data using a SQL-like language called HiveQL. Hive runs on top of Hadoop and converts the queries into MapReduce jobs.
- **Apache HBase:** This is a NoSQL database system for data storage that provides random access and real-time updates for large-scale data sets. HBase runs on top of HDFS and supports row-level transactions, versioning, and compression.
- **Apache Spark:** This is a fast and general-purpose engine for large-scale data processing that supports batch, streaming, interactive, and machine learning applications. Spark runs on top of Hadoop and provides an in-memory computation model that reduces the disk I/O and network overhead of MapReduce.
- **Presto:** This is a distributed SQL query engine for interactive data analysis that allows users to query data from various sources such as HDFS, Hive, HBase, Cassandra, MongoDB, etc. Presto runs on top of Hadoop and provides a low-latency and high-concurrency query execution.
- **Apache Zeppelin:** This is a web-based notebook for data exploration and visualization that allows users to write and execute code in various languages such as Scala, Python, SQL, etc. Zeppelin runs on top of Hadoop and integrates with various frameworks such as Spark, Hive, HBase, etc.

Data Format Co

- Data Format Co is a company that specializes in data conversion and transformation services.
- Data Format Co can help customers to convert their data from one format to another, such as CSV, JSON, XML, PDF, etc.
- Data Format Co can also help customers to transform their data according to their needs, such as filtering, sorting, aggregating, validating, etc.
- Data Format Co uses a cloud-based platform that allows customers to upload their data, choose the desired output format and transformation options, and download the converted and transformed data.
- Data Format Co offers a free trial for new customers, as well as various pricing plans based on the volume and complexity of the data.
- Data Format Co aims to provide fast, reliable, and secure data conversion and transformation services to customers across various industries and domains.

Analyzing data with Hadoop

Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. It consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.

- HDFS is a distributed file system that stores data in blocks across multiple nodes in a cluster. It provides high availability, fault tolerance, and scalability by replicating data blocks on different nodes and balancing the load among them. HDFS also supports various file formats and compression methods.
- MapReduce is a programming model that allows parallel processing of data on HDFS. It consists of two phases: map and reduce. In the map phase, each node in the cluster reads a block of data from HDFS and applies a user-defined function to transform it into key-value pairs. In the reduce phase, the key-value pairs are shuffled and sorted by key, and then another user-defined function is applied to aggregate the values for each key.
- Hadoop also provides other tools and libraries for data analysis, such as Hive, Pig, Spark, HBase, and Mahout. These tools and libraries offer different levels of abstraction and functionality for working with data on HDFS and MapReduce.
- Hadoop can be used for various types of data analysis, such as batch processing, stream processing, interactive querying, machine learning, and graph processing. Hadoop can handle structured, semi-structured, and unstructured data from different sources, such as web logs, social media, sensor data, text, images, and videos.
- Hadoop can be deployed on-premise, in the cloud, or in a hybrid mode.

Hadoop can also be integrated with other systems and platforms, such as relational databases, NoSQL databases, data warehouses, and business intelligence tools. Hadoop can be configured and customized according to the specific needs and requirements of the data analysis project.

Scaling out with Hadoop

- Scaling out is the ability to handle large amounts of data by distributing the workload across multiple machines in a cluster.
- Hadoop is an open-source framework that provides a distributed filesystem (HDFS) and a resource management system (YARN) for scaling out data processing applications.
- Hadoop allows the data to be stored in HDFS, which splits the data into blocks and replicates them across the cluster for fault tolerance and locality.
- Hadoop also allows the computation to be moved to the data, by using YARN to schedule and execute MapReduce jobs on the nodes that host the data blocks.
- Scaling out with Hadoop has several benefits, such as:
 - High performance: Hadoop can process large volumes of data in parallel, using the combined power and memory of the cluster nodes.
 - High scalability: Hadoop can easily add or remove nodes from the cluster, without affecting the data availability or the application logic.
 - High availability: Hadoop can handle node failures and data loss, by using replication and recovery mechanisms.
 - High flexibility: Hadoop can support various types of data (structured, semi-structured, unstructured) and various types of applications (batch, interactive, streaming, machine learning, etc.).
- Scaling out with Hadoop also has some challenges, such as:
 - High complexity: Hadoop requires a lot of configuration and tuning, as well as a good understanding of the underlying architecture and components.
 - High latency: Hadoop is not suitable for low-latency or real-time applications, as it involves a lot of disk and network I/O, as well as intermediate data shuffling and sorting.
 - High maintenance: Hadoop requires a lot of resources and skills to manage and monitor the cluster, as well as to troubleshoot and debug the applications.

Hadoop streaming

- Hadoop streaming is a utility that comes with the Hadoop distribution. It allows you to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer .
- Hadoop streaming works by passing the input data to the mapper as the standard input and collecting the output data from the standard output.

Similarly, the reducer receives the intermediate data from the standard input and writes the final output to the standard output .

- Hadoop streaming uses the Hadoop jar command with the `hadoop-streaming.jar` file to run the streaming jobs. You can specify the mapper and reducer scripts or executables using the `-mapper` and `-reducer` options. You can also specify the input and output directories using the `-input` and `-output` options .
- Hadoop streaming supports various options to customize the streaming jobs, such as specifying the input and output formats, the number of mappers and reducers, the partitioner class, the combiner class, the compression codec, the environment variables, etc .
- Hadoop streaming can be used to write MapReduce programs in any language that can read from the standard input and write to the standard output, such as Python, Ruby, Perl, Bash, etc. This makes Hadoop streaming a powerful and flexible feature for processing large-scale data .

Hadoop pipes

- Hadoop pipes is the name of the C++ interface to Hadoop MapReduce .
- Unlike Streaming, which uses standard input and output to communicate with the map and reduce code, Pipes uses sockets as the channel over which the tasktracker communicates with the process running the C++ map or reduce function . JNI is not used.
- Hadoop pipes is a SWIG-compatible C++ API to implement MapReduce applications (non JNITM based).
- Hadoop pipes can be used to run C++ code within the MapReduce framework in IBM Spectrum Symphony.
- Hadoop pipes requires building the Hadoop version for the target system and compiling the pipes libraries.
- Hadoop pipes operates exclusively on `<key, value>` pairs, that is, the framework views the input to the job as a set of `<key, value>` pairs and produces a set of `<key, value>` pairs as the output of the job.
- Hadoop pipes may have difficulty in passing large data records to map/reduce tasks as a contiguous block of bytes.

Hadoop Ecosystem

- The Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop.
- Hadoop is an open source software library that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage .
- Hadoop enables multiple types of analytic workloads to run on the same

data, at the same time, at massive scale on industry-standard hardware.

- The Hadoop Ecosystem consists of the following components:
 - HDFS: Hadoop Distributed File System, a distributed and fault-tolerant file system that stores data across multiple nodes.
 - YARN: Yet Another Resource Negotiator, a resource management layer that allocates and schedules resources for applications running on Hadoop.
 - MapReduce: A programming model and framework for parallel data processing using key-value pairs.
 - Spark: An in-memory data processing engine that supports batch, streaming, SQL, machine learning, and graph analytics.
 - PIG, HIVE: Query based processing of data services that provide a high-level abstraction for writing MapReduce programs using SQL-like syntax or scripting language.
 - HBase: A NoSQL database that provides random access and consistent read-write operations for large-scale structured and semi-structured data.
 - Sqoop, Flume, Kafka: Data ingestion tools that transfer data from external sources to Hadoop or vice versa.
 - Oozie, Zookeeper, Hue: Coordination and management tools that handle workflow scheduling, configuration, and monitoring of Hadoop applications.
 - Mahout, MLlib, TensorFlow: Machine learning and deep learning libraries that offer scalable and distributed algorithms for data analysis and modeling.

MapReduce

MapReduce is a framework for processing parallelizable problems across large datasets using a large number of computers (nodes), collectively referred to as a cluster or a grid. MapReduce is also the processing component of Apache Hadoop, a popular open-source platform for big data analytics.

The term “MapReduce” refers to two separate and distinct tasks that Hadoop programs perform:

- The **map** job, which takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs).
- The **reduce** job, which takes the output from the map job and combines those data tuples into a smaller set of tuples.

The main advantage of MapReduce is that it allows for distributed processing of large data sets across multiple nodes, thus increasing the speed and scalability of the analysis. MapReduce also abstracts the details of data distribution, load balancing, fault tolerance, and network communication, so that the programmer can focus on the core logic of the problem.

Some examples of problems that can be solved using MapReduce are:

- Word count: counting the number of occurrences of each word in a large text corpus.
- Inverted index: creating a list of all the documents that contain a given word or term.
- PageRank: computing the importance of web pages based on the number and quality of links to them.
- K-means clustering: grouping data points into clusters based on their similarity.

Map Reduce framework and basics

- MapReduce is a programming model or pattern within the Hadoop framework that is used to access big data stored in the Hadoop File System (HDFS).
- MapReduce consists of two distinct tasks – Map and Reduce. The map function takes input pairs, processes them, and produces another set of intermediate pairs as output. The reduce function takes the intermediate pairs, groups them by key, and performs some aggregation or computation on the values.
- The MapReduce framework consists of a single master ResourceManager, one worker NodeManager per cluster-node, and MRAppMaster per application. The ResourceManager allocates resources to the applications and monitors their progress. The NodeManager is responsible for launching and managing the containers that run the map and reduce tasks. The MRAppMaster coordinates the execution of the application and communicates with the ResourceManager and the NodeManager.
- MapReduce is used for various applications such as log analysis, full-text indexing, PageRank calculation, reverse web-link graph, etc.
- MapReduce is a scalable, fault-tolerant, and parallelizable framework that allows us to process large data sets in a distributed environment .

How MapReduce works

MapReduce is a programming model and a framework for processing large datasets in parallel and distributed manner across a cluster of nodes. It consists of two main phases: Map and Reduce. Here is how MapReduce works:

- The input data is split into smaller chunks and assigned to different nodes in the cluster. Each node applies a map function to its local data and produces a set of intermediate key-value pairs as output. The map function can perform any kind of computation on the data, such as filtering, transforming, aggregating, etc.
- The intermediate key-value pairs are then shuffled and sorted by their keys and sent to different nodes for the reduce phase. The reduce function takes a set of values associated with the same key and combines them

into a single output value. The reduce function can perform any kind of computation on the values, such as summing, averaging, counting, etc.

- The final output of the reduce phase is written to a file system or a database. The output can be further processed by other MapReduce jobs or applications.

Some of the features and benefits of MapReduce are:

- It abstracts the complexity of parallel and distributed computing from the programmers and allows them to focus on the logic of their applications.
- It scales well to handle large volumes of data and can run on commodity hardware.
- It is fault-tolerant and can handle node failures and network errors by re-executing the failed tasks on other nodes.
- It is flexible and can support various types of data formats and sources, such as text, binary, structured, unstructured, etc.
- It is compatible with other frameworks and tools in the Hadoop ecosystem, such as HDFS, Hive, Pig, Spark, etc.

Developing a Map Reduce application

MapReduce is a programming model for processing large-scale data sets in parallel and distributed manner. It consists of two phases: map and reduce. The map phase applies a user-defined function to each input record and produces a set of intermediate key-value pairs. The reduce phase applies another user-defined function to all the values associated with the same key and produces a set of output records.

To develop a MapReduce application, you need to follow these steps:

- Write your map and reduce functions in a programming language of your choice, such as Java, Python, or C++. You can use the Hadoop API or a higher-level framework such as Apache Pig or Apache Spark to simplify the development process.
- Test your map and reduce functions with a small subset of data using a local mode or a unit testing framework. You can use tools such as JUnit, MRUnit, or PyUnit to write and run your tests.
- Configure your development environment and set up the necessary parameters for your application, such as input and output paths, number of mappers and reducers, compression options, etc. You can use the Configuration API or the GenericOptionsParser, Tool, and ToolRunner classes to access and parse the configuration XML files and command-line arguments.
- Run your application on the full dataset using a Hadoop cluster or a cloud service such as Amazon EMR or Google Cloud Dataproc. You can use the Hadoop command-line interface or the Hadoop Streaming API to submit and monitor your job. You can also use the MapReduce Web UI or the Hadoop Logs to check the status and performance of your job.

- Tune your application to improve the performance and efficiency of your job. You can use techniques such as combiners, partitioners, counters, custom input and output formats, speculative execution, etc. to optimize your map and reduce functions and reduce the network and disk I/O. You can also use tools such as Hadoop Metrics, Ganglia, or Hadoop Vaidya to collect and analyze the metrics and statistics of your job.

Unit tests with MRUnit

- Unit tests are a way of verifying the correctness and functionality of individual components or classes of a software system.
- MRUnit is a Java library that helps in writing and running unit tests for MapReduce jobs in Hadoop.
- MRUnit provides a set of classes and methods that simulate the behavior of the MapReduce framework, such as `MapDriver`, `ReduceDriver`, and `MapReduceDriver`.
- MRUnit allows the developers to test their mappers, reducers, and combiners in isolation, without setting up a cluster or processing large data sets.
- MRUnit also supports testing other Hadoop components, such as `InputFormat`, `OutputFormat`, `Partitioner`, `Writable`, and `WritableComparable`.
- To use MRUnit, the developers need to add the MRUnit dependency to their project's `pom.xml` file, and import the relevant classes in their test classes.
- A typical MRUnit test case consists of the following steps:
 - Create an instance of the driver class corresponding to the component to be tested, such as `MapDriver` for testing a mapper.
 - Set up the configuration, input, and expected output for the test case using the driver's methods, such as `withInput`, `withOutput`, and `withConfiguration`.
 - Run the test case using the driver's `runTest` method, which will invoke the component under test and compare the actual output with the expected output.
 - Optionally, use other methods of the driver to verify other aspects of the test case, such as counters, order, or grouping of the output.
- An example of a MRUnit test case for testing a mapper that converts text to uppercase is shown below:

```
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.junit.Before;
import org.junit.Test;

public class UppercaseMapperTest {
```

```

private MapDriver<LongWritable, Text, LongWritable, Text> mapDriver;

@Before
public void setUp() {
    // Create an instance of the mapper to be tested
    UppercaseMapper mapper = new UppercaseMapper();
    // Create an instance of the MapDriver with the mapper
    mapDriver = new MapDriver<>(mapper);
}

@Test
public void testUppercaseMapper() {
    // Set up the input and expected output for the test case
    mapDriver.withInput(new LongWritable(1), new Text("hello"));
    mapDriver.withOutput(new LongWritable(1), new Text("HELLO"));
    // Run the test case
    mapDriver.runTest();
}
}

```

Test data and local tests in map reduce

- Map reduce is a programming model for processing large-scale data sets in parallel and distributed environments.
- Test data is a set of input data that is used to verify the correctness and performance of a map reduce program.
- Local tests are tests that run on a single machine, using a local file system and a local map reduce framework, such as Hadoop's LocalJobRunner.
- Local tests are useful for debugging and testing map reduce programs before deploying them to a cluster.
- Local tests have some advantages and disadvantages compared to cluster tests:
 - Advantages:
 - * Faster and easier to set up and run
 - * No need for network or cluster resources
 - * Easier to inspect intermediate and final outputs
 - * Easier to attach a debugger or a profiler
 - Disadvantages:
 - * Cannot test the scalability and reliability of the program
 - * Cannot test the interaction with distributed file systems, such as HDFS

- * Cannot test the behavior under different cluster configurations and parameters
- * May not catch some errors or bugs that only occur in a distributed environment
- To perform local tests, one needs to:
 - Generate or obtain test data that is representative of the real data
 - Write a driver class that configures and runs the map reduce job locally
 - Specify the input and output paths as local file system paths
 - Set the map reduce framework to local mode by setting the property `mapreduce.framework.name` to `local`
 - Run the driver class as a Java application
 - Examine the output files and logs for correctness and performance

Anatomy of a Map Reduce job run

- A Map Reduce job is a distributed computation that processes a large amount of data in parallel on a cluster of machines.
- A Map Reduce job consists of two phases: the map phase and the reduce phase.
- In the map phase, the input data is split into smaller chunks called input splits, and each input split is assigned to a mapper task.
- A mapper task reads the input split and applies a user-defined map function to each record in the input split, producing a set of intermediate key-value pairs.
- The intermediate key-value pairs are then partitioned and shuffled across the cluster, based on their keys, to the reducer tasks.
- In the reduce phase, each reducer task receives a subset of the intermediate key-value pairs that share the same key, and applies a user-defined reduce function to them, producing a set of final output key-value pairs.
- The final output key-value pairs are then written to the output files in the distributed file system.
- A Map Reduce job is coordinated by a master node called the JobTracker, which assigns tasks to the worker nodes called the TaskTrackers, and monitors their progress and status.
- A Map Reduce job can be configured with various parameters, such as the number of mapper and reducer tasks, the input and output formats, the compression and serialization options, the partitioning and sorting functions, and the combiner and counter functions.

Failures in MapReduce

MapReduce is a programming model and framework for processing large-scale data sets in parallel using clusters of commodity machines. However, MapReduce applications may encounter various types of failures during their execution,

which can affect their performance, correctness, and availability. Some of the common failures in MapReduce are:

- **Task failure:** This occurs when a map or reduce task fails due to a runtime exception in the user code, a bad input record, a corrupted intermediate file, or a hardware error. The task may also be killed by the user or the system for various reasons, such as exceeding the memory limit, running too slowly, or being preempted by a higher priority task. When a task fails, the tasktracker reports the error to the jobtracker, which then schedules a new attempt of the task on a different tasktracker. The jobtracker keeps track of the number of failed attempts for each task, and if it exceeds a certain threshold (usually four), the job is marked as failed and aborted.
- **Tasktracker failure:** This occurs when a tasktracker node becomes unavailable due to a network partition, a power outage, a hardware failure, or a software crash. When a tasktracker fails, the jobtracker detects the failure by a periodic heartbeat mechanism, and marks all the tasks that were running on the failed tasktracker as failed. The jobtracker then reassigns those tasks to other available tasktrackers, and also redistributes the map output files that were stored on the failed tasktracker to other tasktrackers. The jobtracker also updates the status of the failed tasktracker in the cluster, and removes it from the list of available tasktrackers.
- **Jobtracker failure:** This occurs when the jobtracker node becomes unavailable due to a similar reason as the tasktracker failure. When the jobtracker fails, the entire MapReduce cluster becomes inoperable, as the jobtracker is responsible for coordinating all the jobs and tasks in the cluster. The jobtracker failure also causes the loss of all the job and task metadata, such as the job configuration, the task assignments, the task progress, and the task output locations. To recover from the jobtracker failure, the jobtracker node needs to be restarted, and all the running jobs need to be resubmitted by the users. The jobtracker also needs to reinitialize the cluster state by communicating with all the tasktrackers and collecting their status.
- **References:**
 - : <https://sungsoo.github.io/2014/04/07/failures-in-classic-mapreduce.html>
https://www.researchgate.net/publication/320027419_Performance_Implications_of_Failures_on_MapReduce
 - <https://www.geeksforgeeks.org/how-mapreduce-completes-a-task/>
 - <https://stackoverflow.com/questions/32255196/error-handling-in-hadoop-map-reduce>
 - <https://stackoverflow.com/questions/40204001/how-does-mapreduce-recover-from-errors-if-failure-happens-in-an-intermediate-state>

Job scheduling in map reduce

- Job scheduling is the process of assigning tasks to workers in a distributed system, such as MapReduce, which is a popular framework for large-scale data processing in the cloud.
- MapReduce consists of two types of tasks: map tasks and reduce tasks. Map tasks read and process input data splits, and produce intermediate key-value pairs. Reduce tasks aggregate the intermediate values for each key, and produce the final output.
- The MapReduce framework consists of a single master node, called the JobTracker, and multiple slave nodes, called the TaskTrackers. The JobTracker is responsible for scheduling the jobs' component tasks on the slaves, monitoring them and re-executing the failed tasks. The TaskTrackers execute the tasks as directed by the JobTracker.
- The JobTracker maintains a queue of jobs submitted by the users, and runs them in order. For each job, the JobTracker splits the input data into fixed-size chunks, called input splits, and assigns a map task for each split. The JobTracker also determines the number of reduce tasks for each job, based on a user-specified parameter or a default value.
- The JobTracker assigns tasks to TaskTrackers based on their availability and locality. The JobTracker tries to schedule a map task on a TaskTracker that is close to the data source, such as the same node or the same rack, to minimize the network traffic. The JobTracker also tries to balance the load among the TaskTrackers, and avoid overloading or underutilizing any of them.
- The TaskTrackers periodically report their status and progress to the JobTracker, and send heartbeats to indicate that they are alive. If a TaskTracker fails or does not send a heartbeat for a long time, the JobTracker marks it as dead and re-schedules its tasks on other TaskTrackers. If a task fails or times out, the JobTracker also re-schedules it on another TaskTracker, up to a maximum number of retries.
- The JobTracker also coordinates the communication between the map and reduce tasks, using a mechanism called shuffle and sort. The map tasks partition their output by the key, and send each partition to the corresponding reduce task. The reduce tasks sort the incoming data by the key, and merge the values for each key. The JobTracker keeps track of the locations of the map output partitions, and provides them to the reduce tasks on demand.
- The JobTracker notifies the user when the job is completed, and provides statistics and logs for the job. The user can also monitor the status and progress of the job through a web interface or a command-line tool. The user can also cancel or kill a job if needed.

Shuffle and Sort in Map Reduce

- Map Reduce is a programming model for processing large-scale data sets

in parallel and distributed manner.

- Map Reduce consists of two phases: map and reduce. In the map phase, each input record is transformed into a key-value pair by a user-defined function. In the reduce phase, all the values associated with the same key are aggregated by another user-defined function.
- Shuffle and Sort is an intermediate phase between map and reduce that ensures that the input to every reducer is sorted by key. The process by which the system performs the sort and transfers the map outputs to the reducers as inputs is called shuffle .
- Shuffle and Sort consists of the following steps :
 - Partitioning: The map outputs are partitioned by a hash function based on the key. Each partition corresponds to a reducer and is stored in a separate buffer on the local disk of the mapper node.
 - Sorting: The map outputs within each partition are sorted by key. This can be done in memory if the partition is small enough, or using an external merge sort if the partition is large.
 - Combiner: An optional step that can be applied to perform a local aggregation of the map outputs with the same key. This can reduce the amount of data to be shuffled and improve the performance of the reduce phase.
 - Copying: The sorted partitions are copied from the mapper nodes to the reducer nodes over the network. The copying can be done in parallel with the map and sort phases to overlap the computation and communication.
 - Merging: The reducer node merges the sorted partitions from different mapper nodes to form a single sorted input for the reduce function. This can be done using a priority queue or a merge sort algorithm.
- Shuffle and Sort is a crucial phase in Map Reduce that affects the performance and scalability of the system. It involves a lot of disk I/O and network I/O, which can be the bottleneck of the system. Therefore, optimizing the shuffle and sort phase is important for improving the efficiency and reliability of Map Reduce applications.

Task execution in map reduce

Map reduce is a programming model that allows processing large amounts of data in parallel by dividing the job into several independent tasks. The tasks are of two types: map tasks and reduce tasks. Map tasks take an input key-value pair and produce a set of intermediate key-value pairs. Reduce tasks take the intermediate key-value pairs with the same key and combine them to produce an output key-value pair.

The execution of tasks is controlled by the map reduce framework, which consists of two types of entities: a job tracker and multiple task trackers. The job tracker is the master entity that coordinates the entire job execution. The task trackers

are the slave entities that run the map and reduce tasks on the nodes where the data is stored or available.

The task execution process in map reduce can be summarized as follows:

- The user submits a map reduce job to the job tracker, specifying the input and output locations, the map and reduce functions, and other configuration parameters.
- The job tracker splits the input data into fixed-size chunks called input splits, and assigns a map task for each input split to a task tracker. The input splits are usually the same as the blocks of the distributed file system, such as HDFS.
- The task tracker runs the map task in a separate process called a child process, and invokes the user-defined map function on each key-value pair in the input split. The map function emits intermediate key-value pairs, which are buffered in memory and periodically spilled to local disk.
- The task tracker partitions the intermediate key-value pairs by a hash function on the key, and sorts them by key within each partition. The partitions correspond to the number of reduce tasks for the job. The sorted partitions are called segments, and are merged into a single file called a map output file.
- The job tracker notifies the task trackers about the location of the map output files, and assigns a reduce task for each partition to a task tracker. The reduce task may run on a different node than the map task that produced the partition.
- The task tracker runs the reduce task in a separate child process, and fetches the map output files for its partition from the local or remote nodes. This process is called the shuffle phase, and may involve decompression and decryption of the map output files.
- The task tracker merges the map output files into a single sorted stream of key-value pairs, and invokes the user-defined reduce function on each group of values with the same key. The reduce function emits output key-value pairs, which are written to the output file in the distributed file system.
- The job tracker monitors the progress of the tasks, and re-executes any failed or slow tasks on different nodes. The job tracker also handles the speculative execution of tasks, which is a technique to improve the performance of the job by running backup copies of tasks on different nodes.
- The job tracker notifies the user when the job is completed, and provides statistics and information about the job execution. The user can access the output files from the distributed file system.

Map Reduce types in map reduce

MapReduce is a programming model for processing large-scale data sets in parallel and distributed manner. It consists of two main functions: map and reduce. The map function takes a key-value pair as input and produces a list of inter-

mediate key-value pairs as output. The reduce function takes an intermediate key and a list of values associated with that key as input and produces a list of final key-value pairs as output.

The types of the input and output key-value pairs for the map and reduce functions can be different, depending on the application logic and the data format. The general form of the map and reduce functions in Java is:

- map: $(K1, V1) \rightarrow \text{list}(K2, V2)$
- reduce: $(K2, \text{list}(V2)) \rightarrow \text{list}(K3, V3)$

where $K1$, $V1$, $K2$, $V2$, $K3$, and $V3$ are the types of the keys and values for the input and output of the map and reduce functions, respectively.

The types of the keys and values can be any Java class that implements the Writable interface, which defines methods for serializing and deserializing the objects. Hadoop provides several built-in classes that implement the Writable interface, such as IntWritable, LongWritable, Text, and so on. Users can also define their own custom classes that implement the Writable interface.

The types of the keys and values also determine the input and output formats for the MapReduce job. The input format defines how the input data is split into key-value pairs and distributed to the map tasks. The output format defines how the output key-value pairs are written to the output files. Hadoop provides several built-in input and output formats, such as TextInputFormat, KeyValueTextInputFormat, SequenceFileInputFormat, TextOutputFormat, SequenceFileOutputFormat, and so on. Users can also define their own custom input and output formats by implementing the InputFormat and OutputFormat interfaces, respectively.

Input Formats in MapReduce

- Input formats are the classes that define how the input data for a MapReduce job is split, read, and processed.
- Input formats are responsible for creating input splits, which are the logical units of work that are assigned to different mappers.
- Input formats also provide a record reader, which is the class that reads the input split and converts it into key-value pairs for the mapper.
- Hadoop provides several built-in input formats for different types of input data, such as text, binary, sequence, database, etc.
- Some of the common input formats are:
 - FileInputFormat: It is the base class for all file-based input formats. It handles common tasks such as splitting files, validating input paths, etc. It has two subclasses: TextInputFormat and KeyValueTextInputFormat.

- `TextInputFormat`: It is the default input format. It reads each line of a text file as a record, and assigns the byte offset of the line as the key, and the line content as the value.
- `KeyValueTextInputFormat`: It is similar to `TextInputFormat`, but it treats each line of a text file as a key-value pair, separated by a delimiter (default is tab).
- `SequenceFileInputFormat`: It is an input format that reads sequence files, which are binary files that store key-value pairs in a compressed and serialized format.
- `SequenceFileAsTextInputFormat`: It is a variant of `SequenceFileInputFormat`, that converts the binary key-value pairs into text and returns them as text key-value pairs.
- `SequenceFileAsBinaryInputFormat`: It is another variant of `SequenceFileInputFormat`, that returns the binary key-value pairs as bytes, without any conversion.
- `NLineInputFormat`: It is an input format that splits the input file based on the number of lines specified by the user. Each input split contains N lines of the input file, where N is a configurable parameter.
- `DBInputFormat`: It is an input format that reads data from a relational database, using JDBC. It requires the user to specify the database connection parameters, the query to execute, and the input class to map the query results.

Output Formats in Map Reduce

- `OutputFormat` is a class that describes the output specification for a MapReduce job.
- `OutputFormat` provides the `RecordWriter` implementation to write the output files of the job to a `FileSystem`.
- `OutputFormat` also validates the output specification of the job, such as checking that the output directory does not already exist.
- There are different types of `OutputFormat` in MapReduce, each with its own advantages and disadvantages.
- Some of the common types of `OutputFormat` are:
 - `TextOutputFormat`: This is the default `OutputFormat` that writes plain text files as output. Each record is a line of text. The key and the value are separated by a tab character. This `OutputFormat` is easy to read and process, but it may not be efficient for binary or complex data types.
 - `SequenceFileOutputFormat`: This `OutputFormat` writes sequence files as output. Sequence files are binary files that store key-value pairs in a compressed and serialized format. This `OutputFormat` is suitable for storing large and complex data types, such as images,

audio, video, etc. Sequence files can also be used as input for another MapReduce job.

- `SequenceFileAsBinaryOutputFormat`: This is a variant of `SequenceFileOutputFormat` that writes the key and the value as binary data, without any serialization or deserialization. This `OutputFormat` is useful for performance optimization, but it may not be compatible with other applications that expect a specific data format.
- `MapFileOutputFormat`: This `OutputFormat` writes map files as output. Map files are a special type of sequence files that support random access and lookup by key. This `OutputFormat` is useful for creating indexes or dictionaries that can be queried efficiently.
- `MultipleOutputs`: This is a utility class that allows writing to multiple output files from a single MapReduce job. This is useful for partitioning or separating the output data based on some criteria, such as the key, the value, or a custom function. `MultipleOutputs` can be used with any `OutputFormat`.
- `LazyOutputFormat`: This is a wrapper class that prevents the creation of empty output files. This is useful for reducing the number of output files and saving disk space. `LazyOutputFormat` can be used with any `OutputFormat`.
- `DBOutputFormat`: This `OutputFormat` writes the output data to a relational database table. This `OutputFormat` is useful for integrating MapReduce with existing database systems or applications. `DBOutputFormat` requires a JDBC driver and a database connection string to connect to the database.

Map Reduce features

- Map Reduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- Map Reduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase merges all intermediate values associated with the same intermediate key and produces the final output.
- Map Reduce can handle various types of data, such as structured, semi-structured, and unstructured data. It can also process data that is too large to fit in memory or disk.
- Map Reduce is fault-tolerant, scalable, and easy to use. It automatically handles the distribution of data and computation, the load balancing, the failure recovery, and the network communication. It also provides a simple interface for the user to specify the map and reduce functions.
- Map Reduce can be used for various applications, such as word count, inverted index, web link analysis, machine learning, data mining, and statistical analysis.

Real-world Map Reduce

MapReduce is a programming model for writing applications that can process large amounts of data in parallel on multiple nodes of a cluster. MapReduce provides analytical capabilities for analyzing huge volumes of complex data .

MapReduce consists of two phases: Map and Reduce. The Map phase takes an input dataset and transforms it into a set of key-value pairs. The Reduce phase takes the key-value pairs from the Map phase and combines them to produce a smaller set of output data .

Let us take a real-world example to comprehend the power of MapReduce. Twitter receives around 500 million tweets per day, which is nearly 3000 tweets per second. The following illustration shows how Twitter manages its tweets with the help of MapReduce:

Twitter MapReduce Example

- The tweets are stored in a distributed file system (such as HDFS) across multiple nodes of a cluster.
- The Map phase reads the tweets and extracts the hashtags from each tweet. The hashtags are the keys and the tweet IDs are the values. The Map phase outputs a set of key-value pairs for each hashtag and tweet ID.
- The key-value pairs are shuffled and sorted by the keys (hashtags) and sent to the Reduce phase.
- The Reduce phase receives the key-value pairs for each hashtag and counts the number of tweet IDs associated with each hashtag. The Reduce phase outputs a set of key-value pairs for each hashtag and its frequency.
- The output data can be used for various purposes, such as finding the most popular hashtags, analyzing the trends, or performing sentiment analysis.

One significant advantage of the MapReduce paradigm and its real-world implementations against traditional database systems is fault tolerance. Consider a traditional database system like Teradata. If one node fails, the entire system may become unavailable or slow down significantly. However, in MapReduce, if one node fails, the system can automatically reassign the tasks to other nodes and continue the processing without any interruption or loss of data. This makes MapReduce suitable for handling large-scale and dynamic data.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

- HDFS is a distributed file system that runs on a cluster of commodity hardware and provides high availability, fault tolerance, scalability, and reliability for storing and processing large-scale data sets.
- HDFS follows a master-slave architecture, where a single NameNode manages the namespace and metadata of the file system, and multiple DataNodes store the actual data blocks on local disks.

- HDFS exposes a POSIX-like interface for clients to interact with the file system, but does not support random writes or updates. Instead, HDFS is optimized for write-once, read-many times (WORM) workloads, such as batch processing, data analysis, and data warehousing.
- HDFS supports replication and erasure coding for data durability and availability. Replication is the default mode, where each data block is replicated across multiple DataNodes, typically three. Erasure coding is an optional mode, where each data block is split into smaller chunks and encoded with parity chunks, which can be used to reconstruct the original block in case of failures. Erasure coding reduces the storage overhead and network bandwidth, but increases the computational complexity and latency.
- HDFS provides several features and tools for managing and monitoring the file system, such as:
 - Web UI: A web interface that displays the status and statistics of the NameNode and DataNodes, as well as the file system namespace and block locations.
 - HDFS Shell: A command-line tool that allows users to perform basic file system operations, such as creating, deleting, copying, moving, and listing files and directories.
 - HDFS API: A Java-based application programming interface that allows developers to programmatically access and manipulate the file system.
 - HDFS Audit Log: A log file that records the file system operations performed by users and applications, such as opening, closing, reading, and writing files.
 - HDFS Balancer: A utility that balances the disk space usage across the DataNodes by moving data blocks from over-utilized nodes to under-utilized nodes.
 - HDFS Snapshot: A feature that allows users to create point-in-time copies of directories and files, which can be used for backup, recovery, and testing purposes.
 - HDFS Trash: A feature that allows users to recover deleted files and directories from a designated trash directory, which is periodically emptied by a configurable policy.
- Hadoop Environment is the set of software components and configurations that are required to run Hadoop applications on a cluster. It includes the following elements:
 - Hadoop Common: A collection of common libraries and utilities that are used by other Hadoop modules, such as HDFS, MapReduce, YARN, and HBase.
 - Hadoop MapReduce: A programming model and framework for processing large-scale data sets in parallel on a cluster of nodes, using a map and reduce function.
 - Hadoop YARN: A resource management and scheduling system that allocates and manages the compute resources (CPU, memory, disk,

network) for Hadoop applications on a cluster of nodes.

- Hadoop HBase: A distributed, column-oriented, NoSQL database that provides random, real-time read/write access to large-scale data sets on top of HDFS.
- Hadoop ZooKeeper: A distributed, highly available, and consistent coordination service that provides configuration management, naming service, synchronization, and group membership for Hadoop applications and services.
- Hadoop Oozie: A workflow scheduler that orchestrates and executes Hadoop jobs, such as MapReduce, Pig, Hive, and Sqoop, based on predefined dependencies and triggers.
- Hadoop Pig: A high-level scripting language and platform that allows users to analyze and process large-scale data sets on Hadoop, using a set of operators and functions.
- Hadoop Hive: A data warehouse system and query engine that allows users to store, manage, and analyze structured and semi-structured data on Hadoop, using a SQL-like language called HiveQL.
- Hadoop Sqoop: A tool that allows users to transfer data between Hadoop and relational databases, such as MySQL, Oracle, and PostgreSQL, using a command-line interface or a Java API.
- Hadoop Flume: A distributed, reliable, and scalable service that collects, aggregates, and moves large amounts of log data from various sources to HDFS or HBase.
- Hadoop Spark: A fast and general-purpose cluster computing system that supports in-memory processing, streaming, machine learning, graph processing, and SQL on Hadoop data.

HDFS

HDFS stands for Hadoop Distributed File System. It is a distributed file system that handles large data sets running on commodity hardware. It is one of the major components of Apache Hadoop, the others being MapReduce and YARN .

Some of the features and benefits of HDFS are:

- It is highly fault-tolerant and can handle failures of nodes, disks, or networks without losing data or interrupting the processing .
- It is designed to run on low-cost, commodity hardware, which reduces the cost of storage and maintenance .
- It can store and process very large files, up to petabytes or exabytes in size, by splitting them into smaller blocks and distributing them across multiple nodes .
- It supports high-throughput access to data by parallelizing the read and write operations across the nodes .
- It provides a simple and consistent interface for applications to access the data, regardless of the physical location or format of the data .

- It can be easily scaled up or down by adding or removing nodes from the cluster, without requiring any downtime or data migration .

Some of the challenges and limitations of HDFS are:

- It is not suitable for low-latency or random access to data, as it is optimized for batch processing and sequential access .
- It does not support concurrent write operations to the same file, as it follows a write-once-read-many model .
- It does not provide any built-in security or encryption mechanisms, and relies on external tools or frameworks for authentication and authorization .
- It does not support any transactional or ACID properties, and does not guarantee the consistency of the data across the nodes .
- It does not support any compression or deduplication techniques, and consumes more storage space than other file systems .

Design of HDFS

HDFS stands for Hadoop Distributed File System. It is a file system that is designed to store very large files across multiple machines in a cluster. HDFS has the following characteristics:

- It is based on the Google File System (GFS) .
- It was originally built for the Apache Nutch web search engine project .
- It is a member of the Hadoop Ecosystem, which includes other tools for data processing and analysis .
- It supports streaming data access patterns, meaning that it is optimized for reading large files sequentially rather than randomly .
- It runs on commodity hardware, meaning that it can use cheap and widely available machines rather than expensive and specialized ones .
- It has a master-slave architecture, meaning that it consists of a single NameNode that manages the metadata of the file system and multiple DataNodes that store the actual data blocks .
- It is fault-tolerant, meaning that it can handle failures of machines or network connections without losing data or functionality .
- It is scalable, meaning that it can grow or shrink in size and performance according to the demand and resources available .
- It is not POSIX-compliant, meaning that it does not follow the standard interface and semantics of a Unix-like file system . This allows it to achieve higher throughput and efficiency, but also imposes some limitations and trade-offs.

HDFS concepts

HDFS stands for Hadoop Distributed File System. It is a distributed file system that runs on a cluster of commodity hardware devices and provides high throughput access to large data sets. HDFS is one of the core components

of Apache Hadoop, an open source framework for distributed processing and analysis of big data.

Some of the key concepts of HDFS are:

- **Blocks:** HDFS divides each file into fixed-size blocks, usually 128 MB, and stores them across multiple data nodes in the cluster. Each block is replicated on a configurable number of data nodes (default is 3) to ensure fault tolerance and availability. HDFS does not store any metadata (such as file name, permissions, etc.) with the blocks. Instead, it maintains a separate namespace for the files and directories, which is stored on a dedicated node called the NameNode.
- **NameNode:** The NameNode is the master node of the HDFS cluster. It manages the namespace and the metadata of the file system, such as the file names, permissions, locations of the blocks, etc. The NameNode also coordinates the read and write operations from the clients to the data nodes. The NameNode keeps the entire file system metadata in memory for fast access and periodically saves a copy of it to a local disk, called the FsImage. The NameNode also maintains a log of the changes made to the file system, called the EditLog. The NameNode is a single point of failure in HDFS, so it is usually configured with a backup node or a high availability mode to ensure reliability.
- **DataNode:** The DataNode is the worker node of the HDFS cluster. It stores the actual data blocks of the files and serves the read and write requests from the clients. The DataNode also performs periodic block reports to the NameNode, informing it about the blocks it is storing and their health status. The DataNode also performs block replication, deletion, and rebalancing as instructed by the NameNode. The DataNode is designed to handle failures gracefully and can be added or removed from the cluster dynamically.
- **Client:** The client is the application or the user that interacts with the HDFS cluster. The client can perform various operations on the file system, such as creating, deleting, reading, writing, appending, and renaming files and directories. The client communicates with the NameNode to get the metadata of the file system and the locations of the blocks. The client then directly contacts the DataNodes to perform the actual data transfer. The client also handles the failures and retries of the DataNodes by consulting the NameNode for the latest block locations. The client can use various APIs or tools to access the HDFS cluster, such as Java, C, Python, WebHDFS, etc.

Benefits of HDFS

HDFS stands for Hadoop Distributed File System, which is a scalable and reliable file system for storing and processing large volumes of data. HDFS has several benefits, such as:

- **Distributed storage:** HDFS splits the data into blocks and distributes them across multiple nodes in a cluster, which enables parallel processing and reduces network congestion.
- **Fault tolerance:** HDFS replicates each block to multiple nodes, which ensures data availability and durability in case of node failures. HDFS also detects and recovers from faults automatically, without requiring human intervention.
- **Cost effectiveness:** HDFS runs on commodity hardware, which lowers the storage costs. HDFS is also an open-source software, which means there is no licensing or support fee.
- **Scalability:** HDFS can scale up or down by adding or removing nodes from the cluster, without affecting the data or the performance. HDFS can handle petabytes of data and thousands of nodes.
- **Speed:** HDFS can deliver more than 2 GB of data per second, thanks to its cluster architecture and data locality. Data locality means that the data is stored close to where it is processed, which reduces the data transfer time and improves the efficiency.

Challenges of HDFS

HDFS is a distributed file system that is designed to store and process large amounts of data across multiple nodes in a cluster. HDFS has many advantages, such as high scalability, fault tolerance, and parallel processing. However, it also faces some challenges, such as:

- **Issues with small files:** HDFS is not suitable for storing and accessing small files, as each file occupies a block of fixed size (typically 128 MB or 256 MB) and consumes a metadata entry in the NameNode. This can lead to inefficient disk space utilization and memory consumption in the NameNode. Moreover, HDFS does not support random access to files, which limits the performance of applications that need to read or write small portions of data .
- **Slow processing speed:** HDFS relies on MapReduce, a batch processing framework, to process the data stored in the file system. MapReduce has a high latency, as it involves multiple stages of computation, such as map, shuffle, sort, and reduce. Moreover, MapReduce does not support iterative processing, which is required for many machine learning and graph algorithms. MapReduce also creates intermediate files between each stage, which adds to the I/O overhead .
- **Support for batch processing only:** HDFS is designed for batch processing, where the data is ingested once and processed periodically. It is not suitable for streaming data, where the data is continuously generated and processed in real time. HDFS does not provide any mechanism to handle data updates, deletes, or appends, which limits the flexibility of data management. HDFS also does not support transactions, concurrency, or consistency guarantees, which are essential for many applications .

- **No real-time processing:** HDFS does not support real-time processing, as it has a high latency and does not support streaming data. HDFS also does not have an in-memory processing engine, which can speed up the data processing by avoiding disk I/O. HDFS users have to rely on other frameworks, such as Apache Spark, Apache Storm, or Apache Flink, to perform real-time processing on top of HDFS .
- **Latency:** HDFS has a high latency, as it involves multiple network hops and disk I/O operations to read or write data. HDFS also has a high replication factor, which increases the data transfer time and network bandwidth consumption. HDFS also has a single point of failure, which is the NameNode, which coordinates all the file system operations. If the NameNode fails, the entire file system becomes unavailable until it is recovered .
- **No ease of use:** HDFS is not easy to use, as it requires a lot of configuration and tuning to optimize the performance and reliability of the file system. HDFS also has a complex architecture and a low-level API, which makes it difficult for users to understand and interact with the file system. HDFS also does not have a graphical user interface, which can help users to monitor and manage the file system .
- **Security issue:** HDFS does not have a strong security mechanism, as it relies on the underlying operating system and network security. HDFS does not support encryption, authentication, authorization, or auditing of the data stored in the file system. HDFS also does not have a fine-grained access control mechanism, which can restrict the access to the data based on the user roles and privileges .

File sizes in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed storage system for large-scale data processing.
- HDFS is designed to support files that are **gigabytes to terabytes** in size. It provides high throughput and scalability by splitting files into **blocks** and distributing them across multiple nodes in a cluster.
- A block is a **fixed-sized** chunk of data that is stored as an independent unit in HDFS. The default block size in HDFS is **128 MB**, but it can be configured manually. Larger block sizes reduce the overhead of managing metadata and improve the network bandwidth utilization.
- To get the size of a file or a directory in HDFS, one can use the **hadoop fs -du** command. This command shows the base size of the file or directory before replication, which means the amount of space in bytes that have been used by the files that match the specified file pattern. For example, **hadoop fs -du input** will show the size of the input directory and its contents.
- To get the size of a specific file in HDFS, one can use the **hadoop fs -ls** command. This command lists the files in the current directory along with their details, such as permissions, owner, group, size, and modification

time. The fifth column in the command output contains the file size in bytes. For example, **hadoop fs -ls input/sou** will show the size of the sou file in the input directory.

- To get the summary of a directory or a file pattern in HDFS, one can use the **-s** option with the **hadoop fs -du** command. This option shows the aggregate size of the files that match the specified file pattern. For example, **hadoop fs -du -s input/count*** will show the sum of all files containing “count” in the input directory.

Block Sizes in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed file system that stores large amounts of data across multiple nodes in a cluster.
- HDFS divides a file into one or more blocks, which are fixed-sized chunks of data that are stored on different slave machines in the cluster.
- The default block size in HDFS is 64 MB for Hadoop 1.0 and 128 MB for Hadoop 2.0, but it can be configured manually by changing the `dfs.block.size` property in `hdfs-site.xml`.
- The advantage of using large block sizes in HDFS is that it reduces the overhead of managing metadata and improves the network bandwidth utilization by transferring large amounts of data in a single operation.
- The disadvantage of using large block sizes in HDFS is that it may cause internal fragmentation and waste disk space if the file size is not a multiple of the block size.
- HDFS supports variable length blocks, which means that the last block of a file can be smaller than the configured block size, and users can start a new block without filling out the last block.
- HDFS also supports replication, which means that each block is copied to a certain number of other nodes in the cluster to ensure fault tolerance and availability. The replication factor can be configured per file by changing the `dfs.replication` property in `hdfs-site.xml`.

Block abstraction in HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant file system for storing large amounts of data across multiple machines.
- HDFS has the concept of a **block**, which is a fixed-size unit of data that is stored as an independent unit on a DataNode. A DataNode is a machine that stores and serves blocks to clients.
- The default block size in HDFS is 64 MB or 128 MB, which is much larger than the typical block size of other file systems (e.g., 4 KB). This is because HDFS is designed for streaming large files, and having a large block size reduces the overhead of disk seeks and network transfers.
- A file in HDFS is split into one or more blocks, and each block is replicated

on multiple DataNodes for fault tolerance. The number of replicas for each block is configurable, and the default value is 3.

- The NameNode is the master node that manages the file system namespace and the metadata of blocks. It maintains a mapping of files to blocks, and blocks to DataNodes. It also handles file system operations such as opening, closing, renaming, and deleting files and directories.
- The NameNode does not store the actual data of blocks, but only the metadata. Therefore, it does not need to have a lot of storage space, but it needs to have enough memory to store the metadata in RAM. The metadata of a block includes its ID, size, location, checksum, and permissions.
- The NameNode periodically receives heartbeat and block report messages from each DataNode, which indicate the health and status of the DataNode and the blocks it stores. The NameNode uses this information to monitor the cluster and to balance the load and the replication of blocks.
- The NameNode also performs periodic checkpoints and backups of its metadata, which are stored on local or remote file systems. These are used to recover the NameNode in case of a failure or a corruption.
- The block abstraction in HDFS provides several benefits, such as:
 - It simplifies the storage and management of large files, as they are divided into manageable chunks that can be distributed and replicated across the cluster.
 - It improves the performance and scalability of the file system, as it reduces the disk seek and network transfer time, and allows parallel processing of blocks by different clients and applications.
 - It enhances the reliability and availability of the file system, as it allows the detection and recovery of corrupted or missing blocks, and the replication of blocks on different DataNodes.

Data replication in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed file system that stores large amounts of data across multiple nodes in a cluster.
- Data replication is a technique that creates multiple copies of the same data on different nodes, to ensure high availability, fault tolerance, and load balancing.
- HDFS follows a master-slave architecture, where the master node is called the NameNode, and the slave nodes are called the DataNodes.
- The NameNode is responsible for managing the metadata of the files and directories, such as the file name, size, location, permissions, etc. The NameNode also maintains the information about the replication factor and the block size of each file.
- The DataNodes are responsible for storing the actual data in the form of blocks, which are fixed-sized chunks of data. The DataNodes also send periodic heartbeats and block reports to the NameNode, to indicate their status and the blocks they have.

- The replication factor is a configurable parameter that specifies the number of copies of each block to be created. The default replication factor is 3, which means that each block will have 3 replicas on different DataNodes.
- The block size is another configurable parameter that specifies the size of each block. The default block size is 64 MB, which means that each file will be split into 64 MB blocks and stored on the DataNodes.
- The replication process in HDFS is as follows:
 - When a client wants to write a file to HDFS, it first contacts the NameNode and requests a list of DataNodes that can store the blocks of the file.
 - The NameNode allocates a unique block ID for each block of the file, and returns a list of DataNodes for each block, in the order of preference. The preference is based on the rack awareness policy, which tries to place the replicas on different racks to avoid data loss due to rack failure.
 - The client then contacts the first DataNode in the list for each block, and sends the data to it. The DataNode then replicates the data to the next DataNode in the list, and so on, until the replication factor is met.
 - The DataNode that receives the data from the client is called the pipeline leader, and the DataNodes that receive the data from the pipeline leader are called the pipeline followers. The pipeline leader is responsible for coordinating the replication process and sending acknowledgments to the client and the NameNode.
 - The client waits for the acknowledgments from the DataNodes before sending the next block of data. The client also sends a confirmation to the NameNode after writing the entire file.
 - The NameNode updates its metadata with the information about the file and its blocks, and commits the changes to a persistent storage.
- The replication process in HDFS is also dynamic, which means that it can adjust the number and location of the replicas based on the cluster conditions, such as node failure, node addition, node removal, etc.
- The NameNode periodically runs a process called the Replication Monitor, which checks the status of the blocks and the DataNodes, and initiates the replication or deletion of the blocks as needed.
- The Replication Monitor follows some rules to ensure the optimal replication of the blocks, such as:
 - If a block has less than the desired number of replicas, the NameNode will create new replicas on the available DataNodes, preferably on the same rack as the existing replicas, to reduce the network bandwidth consumption.
 - If a block has more than the desired number of replicas, the NameNode will delete the excess replicas, preferably on the same rack as the other replicas, to balance the disk space usage.
 - If a block has the desired number of replicas, but they are not distributed across different racks, the NameNode will move some repli-

- cas to other racks, to increase the data reliability.
- The Replication Monitor also takes into account the node capacity, node load, node distance, and node availability, to select the best DataNodes for the replication or deletion of the blocks.

How does HDFS store

- HDFS stands for Hadoop Distributed File System, which is the primary storage system used by Hadoop applications.
- HDFS stores data in a distributed manner, meaning that it divides the data into small pieces and stores them on different DataNodes in the cluster .
- Each piece of data is called a block, and the default size of each block is 128 MB, which can be configured in the `hdfs-site.xml` file.
- HDFS maintains multiple copies of each block, called replicas, for fault tolerance and high availability. The default replication factor is 3, which can also be configured in the `hdfs-site.xml` file.
- HDFS has a master-slave architecture, where the master node is called the NameNode and the slave nodes are called the DataNodes.
- The NameNode manages the file system namespace and the metadata of all the files and directories. It also coordinates the placement and replication of blocks across the DataNodes.
- The DataNodes store the actual blocks of data and serve read and write requests from the clients. They also send periodic reports to the NameNode about the status of the blocks they are holding.
- HDFS provides a way for MapReduce to process a subset of large data sets broken into blocks, parallelly on several nodes. This reduces the network overhead and increases the performance of the system.

Read operations in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS follows a master-slave architecture, where a single NameNode manages the metadata of the file system, and multiple DataNodes store the actual data blocks of the files.
- To read a file from HDFS, a client application first contacts the NameNode and obtains the list of DataNodes that store the replicas of the blocks of the file.
- The client then contacts one of the DataNodes directly and requests the transfer of the desired block. The DataNode sends the block to the client as a stream of bytes over a TCP connection.
- The client reads the data from the stream and verifies the checksum of the block. If the checksum matches, the client is done with the block. If the checksum does not match, the client reports the error to the NameNode and tries to read the block from another DataNode that has a replica of

the same block.

- The client repeats this process until it has read all the blocks of the file. The client can also read the file in parallel from multiple `DataNodes` to improve the read performance.

Write operations in HDFS

- Write operations in HDFS are used to store data in the distributed file system by creating new files or appending to existing ones.
- Write operations in HDFS follow these steps:
 1. The client initiates the write operation by calling the `create()` method of the `DistributedFileSystem` object, which is a wrapper over the `FileSystem` class. This method takes the file name, replication factor, block size, and other parameters as arguments.
 2. The `DistributedFileSystem` object communicates with the `NameNode` to check if the file already exists, if the user has the permission to write, and if there is enough space in the cluster. If all the conditions are met, the `NameNode` grants the write permission and returns a list of `DataNode` addresses where the file blocks can be stored.
 3. The client then creates a `DFSOutputStream` object, which is responsible for splitting the data into packets, adding checksums, and sending them to the `DataNode` pipeline. The `DFSOutputStream` also maintains a buffer to store the packets before sending them to the pipeline.
 4. The client writes the data to the `DFSOutputStream`, which divides the data into fixed-size blocks (default 128 MB) and assigns a sequence number to each block. The `DFSOutputStream` then creates a packet for each block and adds it to the buffer.
 5. The `DFSOutputStream` periodically flushes the packets from the buffer to the `DataNode` pipeline, which consists of a chain of `DataNodes` that are responsible for storing the replicas of the blocks. The first `DataNode` in the pipeline is the primary `DataNode`, which receives the packets from the client and forwards them to the next `DataNode` in the pipeline. The last `DataNode` in the pipeline sends an acknowledgment to the previous `DataNode`, and so on, until the acknowledgment reaches the primary `DataNode`, which then sends it back to the client.
 6. The client receives the acknowledgment from the primary `DataNode` and updates the file metadata in the `NameNode` by calling the `close()` method of the `DFSOutputStream`. The `NameNode` then commits the file creation or append operation in the file system namespace.

Java interfaces to HDFS

- HDFS is the Hadoop Distributed File System, which is a scalable and

fault-tolerant storage system for large-scale data processing.

- HDFS can be accessed and manipulated through various interfaces, such as Java API, command-line interface, web interface, etc.
- The Java interface of HDFS is based on the abstract `FileSystem` class, which defines the common operations for all file systems, such as create, delete, rename, copy, etc.
- HDFS provides a concrete implementation of the `FileSystem` class, called `DistributedFileSystem`, which extends the `FileSystem` class and implements the specific features of HDFS, such as replication, block size, checksum, etc.
- To use the Java interface of HDFS, one needs to create a `FileSystem` object with a `Configuration` object that specifies the HDFS configuration parameters, such as the namenode address, the default block size, the replication factor, etc.
- The `FileSystem` object can then be used to perform various operations on HDFS files and directories, such as opening, reading, writing, appending, listing, etc.
- The `FileSystem` object also provides methods to get the status, metadata, and location of HDFS files and directories, such as `getFileStatus`, `listStatus`, `getContentSummary`, `getFileBlockLocations`, etc.
- The `FileSystem` object also supports the `PathCapabilities` interface, which allows checking the capabilities of a given path, such as whether it supports append, truncate, concat, etc.
- The `FileSystem` object also implements the `DelegationTokenIssuer` interface, which allows obtaining and renewing delegation tokens for HDFS authentication and authorization.
- The `FileSystem` object also provides methods to access the Trash feature of HDFS, which allows moving deleted files and directories to a user-specific trash directory, and restoring them from there.
- The `FileSystem` object also provides methods to access the Snapshot feature of HDFS, which allows creating point-in-time copies of files and directories, and restoring them from there.
- The `FileSystem` object also provides methods to access the Encryption Zone feature of HDFS, which allows encrypting and decrypting files and directories using keys managed by a Key Management Server (KMS).
- The `FileSystem` object also provides methods to access the Erasure Coding feature of HDFS, which allows encoding and decoding files and directories using a specified erasure coding policy, such as Reed-Solomon or XOR.
- The `FileSystem` object also provides methods to access the Quota feature of HDFS, which allows setting and getting the limits on the disk space and the number of files and directories for a given path.
- The `FileSystem` object also provides methods to access the Storage Policy feature of HDFS, which allows setting and getting the storage policy for a given path, such as hot, warm, cold, etc.
- The `FileSystem` object also provides methods to access the ACL feature of HDFS, which allows setting and getting the access control lists for a

given path, such as user, group, and other permissions.

- The `FileSystem` object also provides methods to access the `XAttr` feature of HDFS, which allows setting and getting the extended attributes for a given path, such as user-defined metadata, tags, etc.

Command Line Interface to HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for big data processing.
- Command Line Interface (CLI) is one of the simplest ways to interact with HDFS. CLI has support for filesystem operations like reading, writing, creating, moving, deleting, and listing files and directories.
- To use CLI, we need to have a Hadoop cluster running and access it through SSH or a terminal.
- The basic syntax of CLI commands is `hdfs dfs -command [options] [arguments]`, where `hdfs` is the command name, `dfs` is the subcommand name, `-command` is the specific operation, `[options]` are the optional parameters, and `[arguments]` are the required arguments.
- Some of the common CLI commands are:
 - `hdfs dfs -ls [path]`: List the contents of a directory or file.
 - `hdfs dfs -mkdir [path]`: Create a directory or directories.
 - `hdfs dfs -put [local_path] [hdfs_path]`: Copy a file or directory from the local filesystem to HDFS.
 - `hdfs dfs -get [hdfs_path] [local_path]`: Copy a file or directory from HDFS to the local filesystem.
 - `hdfs dfs -cat [path]`: Display the contents of a file or files.
 - `hdfs dfs -rm [path]`: Delete a file or directory.
 - `hdfs dfs -mv [source_path] [destination_path]`: Move or rename a file or directory.
 - `hdfs dfs -cp [source_path] [destination_path]`: Copy a file or directory within HDFS.
 - `hdfs dfs -du [path]`: Display the disk usage of a file or directory.
 - `hdfs dfs -help [command]`: Display the help information for a specific command or all commands.
- To get more details and examples of CLI commands, we can refer to the official documentation or run `hdfs dfs -help` in the terminal.

Hadoop file system interfaces

- Hadoop file system interfaces are the Java abstract classes and interfaces that define the client API to interact with various file systems in Hadoop.
- The main interface is `org.apache.hadoop.fs.FileSystem`, which represents a generic file system that can be implemented by different concrete classes.

- Hadoop uses the URI scheme to select the appropriate file system instance to communicate with, such as `hdfs://`, `file://`, `s3://`, etc.
- Hadoop file system interfaces provide methods for creating, deleting, renaming, reading, writing, and listing files and directories, as well as accessing file system metadata and configuration.
- Hadoop file system interfaces also support streaming access to file system data, which allows applications to process large amounts of data efficiently without loading the entire file into memory.
- Hadoop file system interfaces are designed to be highly fault-tolerant, scalable, and compatible with existing distributed file systems.

Data flow in HDFS

HDFS stands for Hadoop Distributed File System, which is a storage layer of Hadoop that stores data in a distributed and reliable manner. HDFS splits a file into one or more blocks and stores them across a cluster of DataNodes. The NameNode manages the file system namespace and the metadata of the blocks. The client interacts with the NameNode and the DataNodes to perform read and write operations on HDFS files.

The data flow in HDFS can be summarized as follows:

- **Write operation:** The client creates a file by calling `create()` on the DistributedFileSystem (DFS) object, which is an instance of Hadoop FileSystem. DFS makes a remote procedure call (RPC) to the NameNode to create a new file in the file system's namespace, with no blocks associated with it. The NameNode checks if the file already exists or if the parent directory is valid, and returns an exception if any of these conditions are not met. The NameNode also grants a lease to the client to create the file, which prevents other clients from writing to the same file. The client then writes data to the file by calling `write()` on the DFSOutputStream object, which is a wrapper of the Java OutputStream. The DFSOutputStream splits the data into packets, which it writes to an internal queue called the data queue. The data queue is consumed by the DataStreamer, which is a thread that asks the NameNode to allocate new blocks by picking a list of suitable DataNodes to store the replicas. The list of DataNodes forms a pipeline, and the DataStreamer streams the packets to the first DataNode in the pipeline. The first DataNode stores the packet and forwards it to the second DataNode, and so on until the last DataNode. The DataStreamer also maintains an internal queue of packets that are waiting for an acknowledgement from the DataNodes, called the ack queue. The last DataNode in the pipeline sends back an acknowledgement to the previous DataNode, which relays it back to the DataStreamer. The DataStreamer removes the acknowledged packets from the ack queue. If a packet fails to be acknowledged by a certain timeout or a DataNode fails, the pipeline is broken and a new pipeline is constructed by the DataStreamer. The write operation is complete when the client calls `close()` on the DFSOut-

putStream, which flushes all the remaining packets to the DataNodes and tells the NameNode to complete the file. The NameNode commits the file creation by removing the lease and adding the file to the namespace.

- **Read operation:** The client opens a file by calling `open()` on the DFS object, which returns a `DFSInputStream` object, which is a wrapper of the Java `InputStream`. The client then reads data from the file by calling `read()` on the `DFSInputStream`, which contacts the NameNode to fetch the locations of the blocks of the file. The NameNode returns a list of DataNodes for each block, sorted by their proximity to the client. The client then contacts the closest DataNode for the first block of the file and starts reading data from it. If the read fails or the DataNode is not available, the client tries the next DataNode in the list. The client can also perform a seek operation to move to a different position in the file, which may involve contacting the NameNode again to get the new block locations. The read operation is complete when the client has read all the data from the file or calls `close()` on the `DFSInputStream`.

Data Ingest with Flume and Sqoop in HDFS

- Data ingestion is the process of collecting, transferring, and loading data from various sources into a data store, such as HDFS, for analysis and processing.
- Data ingestion is important in any big data project because the volume of data is generally in petabytes or exabytes, and the data may come from different types of sources, such as structured, unstructured, or semi-structured data.
- Hadoop provides two tools for data ingestion: Flume and Sqoop, which have different use cases and features.

Flume

- Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of streaming data into HDFS.
- Flume can handle data from multiple sources, such as web servers, application servers, social media, sensors, etc., and can perform data filtering, transformation, and enrichment before loading the data into HDFS.
- Flume has a flexible and scalable architecture, which consists of three main components: sources, channels, and sinks.
 - Sources are the components that consume data from external sources and pass them to channels.
 - Channels are the components that store the data temporarily and provide a reliable data transfer between sources and sinks.
 - Sinks are the components that deliver the data to the destination, such as HDFS, HBase, Kafka, etc.

- Flume supports a variety of sources, channels, and sinks, and also allows custom plugins to be developed for specific needs.
- Flume is a better choice when moving bulk streaming data from various sources that need to be processed in near real-time.

Sqoop

- Sqoop is a tool that allows bulk import and export of data between structured or relational data sources, such as RDBMS, and Hadoop storage architectures, such as HDFS or Hive.
- Sqoop can handle data from databases that support JDBC, such as Oracle, MySQL, PostgreSQL, Teradata, etc., and can perform parallel data transfer, compression, and incremental updates.
- Sqoop has a command-line interface, which allows users to specify the source and destination of the data transfer, as well as the mapping and transformation rules.
- Sqoop can also generate Java code to create and populate Hive tables or HBase tables from the imported data.
- Sqoop is an ideal fit if the data is sitting in databases that need to be moved into Hadoop for batch processing or analysis.

Hadoop archives in HDFS

- Hadoop archives or HAR files are a file archiving facility that packs files into HDFS blocks more efficiently, thereby reducing Name Node memory usage while still allowing transparent access to files .
- Hadoop archives can be used as input to MapReduce jobs by specifying a different input filesystem than the default file system. For example, if you have a hadoop archive stored in HDFS in /user/zoo/foo.har then for using this archive for MapReduce input, all you need to specify the input directory as har:///user/zoo/foo.har .
- Hadoop archives are created from a collection of files and the archiving tool (a simple command) will run a MapReduce job to process the input files in parallel and create an archive file. The command syntax is:

```
hadoop archive -archiveName name -p <parent path> <src>* <dest>
```

- Hadoop archives have a hierarchical structure that consists of an index file, a master index file and data files . The index file contains the metadata of the files in the archive, such as name, size, modification time, etc. The master index file contains the offsets of the index files in the data files. The data files contain the actual data of the files in the archive.
- Hadoop archives can be accessed using the har:// scheme or using the `hadoop fs -lsr` command . The har:// scheme allows the user to access the files in the archive as if they were normal files in HDFS. The `hadoop fs -lsr` command lists the files in the archive recursively.

Hadoop I/O

- Hadoop I/O is a set of primitives for data input and output in Hadoop.
- Hadoop I/O includes techniques such as data integrity, compression, serialization, and file formats that are suitable for large-scale data processing.
- Data integrity is the ability to detect and correct errors in data. Hadoop I/O uses checksums to verify the integrity of data blocks stored in HDFS and transferred across the network.
- Compression is the process of reducing the size of data to save storage space and network bandwidth. Hadoop I/O supports various compression codecs, such as gzip, bzip2, LZO, and Snappy. Compression can be applied at different levels, such as file, block, or record.
- Serialization is the process of converting data structures into a binary format that can be stored or transmitted. Hadoop I/O uses Writable and Avro as the main serialization frameworks for MapReduce programs. Writable is a simple and fast serialization format that implements the Writable interface. Avro is a schema-based serialization format that supports schema evolution and data interoperability.
- File formats are the ways of organizing and storing data in files. Hadoop I/O provides several file formats for different types of data, such as text, binary, and structured. Some of the common file formats are SequenceFile, MapFile, SetFile, ArrayFile, and AvroFile. These file formats provide features such as splitting, compression, and random access.

Compression in Hadoop io

- Compression is a technique to reduce the size of data by encoding it in a different format that uses fewer bits.
- Compression is useful for Hadoop because it can save storage space, network bandwidth, and disk I/O time.
- Hadoop supports various compression codecs, such as DEFLATE, gzip, bzip2, LZO, LZ4, and Snappy.
- Only bzip2 is splittable, meaning that it can be processed by multiple map tasks in parallel without reading the entire file.
- Hadoop provides a CompressionCodecFactory class that can detect the compression format of an input file based on its extension and return the appropriate CompressionCodec object.
- A CompressionCodec object can create an InputStream or an OutputStream that can compress or decompress data on the fly.
- Some compression codecs can also work with direct bytebuffers, which are faster and more efficient than regular byte arrays.
- Compression can be applied to different stages of Hadoop processing, such as input, output, intermediate, and shuffle.
- Compression can improve the performance and scalability of Hadoop applications, but it also has some trade-offs, such as CPU overhead, memory usage, and compatibility issues.

Serialization in Hadoop io

- Serialization is the process of converting structured data into a byte stream for transmission over the network or storage on disk.
- Deserialization is the reverse process of converting a byte stream back into structured data.
- Hadoop uses serialization for interprocess communication between nodes in the system using remote procedure calls (RPCs).
- Hadoop also uses serialization for storing data in HDFS and for passing data between map and reduce tasks.
- Hadoop provides a mechanism for using different serialization frameworks by defining a list of `Serialization` classes in the property `io.serializations`.
- A `Serialization` class knows how to create `Serializer` and `Deserializer` objects for a given type of data.
- A `Serializer` object can serialize an object of a given type into an `OutputStream`.
- A `Deserializer` object can deserialize an object of a given type from an `InputStream`.
- Hadoop provides a default `Serialization` class for `Writable` objects, which delegates to the `Writable.write(DataOutput)` and `Writable.readFields(DataInput)` methods.
- Hadoop also supports other serialization frameworks, such as Avro, Thrift, and Protocol Buffers, by providing corresponding `Serialization` classes.

Avro and file based data structures in Hadoop io

- Avro is a data serialization framework that is widely supported throughout Hadoop and its ecosystem .
- Avro creates a data file where it keeps data along with schema in its metadata section, which makes the file self-describing .
- Avro data files support compression and are splittable, which is crucial for a MapReduce data input format.
- Avro data files are line-oriented, meaning the value of each row in the file are stored consecutively.
- Avro provides a rich data structure which makes it more popular than other similar solutions.
- Avro is the best fit for Big Data processing and is quite popular in Hadoop and Kafka world for its faster processing.
- Avro is best suited to Spark processing, whereas Hadoop supports other file formats such as ORC and Parquet.
- Avro supports schema evolution, which means changing the schema without breaking the compatibility with older data.

Hadoop Environment

- Hadoop is an open source software framework that allows for the distributed storage and processing of large datasets across clusters of computers using simple programming models .
- Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN.
- Hadoop Common provides the common utilities and libraries that are used by other Hadoop modules.
- HDFS is a distributed file system that stores data across multiple nodes in a cluster, providing high availability, fault tolerance, and scalability.
- Hadoop MapReduce is a programming model and a software framework for writing applications that process large amounts of data in parallel on clusters of nodes using a map and reduce function.
- Hadoop YARN is a resource management platform that manages the compute resources and the scheduling of applications running on Hadoop clusters.
- Hadoop requires the Java Runtime Environment (JRE) 1.6 or higher to run. The standard startup and shutdown scripts require that Secure Shell (SSH) be set up between nodes in the cluster.
- Hadoop can run on various types of hardware, from commodity computers to cloud platforms, such as Google Cloud .
- Hadoop can also integrate with other software tools and frameworks, such as Apache Hive, Apache Pig, Apache Spark, Apache HBase, Apache ZooKeeper, and Apache Flume, to provide additional functionality and features for data analysis, processing, and management.

Setting up a Hadoop cluster in Hadoop Environment

A Hadoop cluster is a collection of machines that run the Hadoop software and store and process large amounts of data. A Hadoop cluster can be configured in different modes, such as standalone, pseudo-distributed, or fully distributed. In this section, we will focus on the fully distributed mode, which is the most common and scalable way to run Hadoop in a production environment.

To set up a Hadoop cluster in a fully distributed mode, you will need to follow these steps:

1. Install and configure the required software on all the nodes in the cluster, such as Java, SSH, and Hadoop. You can download the latest version of Hadoop from <https://hadoop.apache.org/releases.html> and follow the installation instructions from <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/SingleCluster.html#Installation>.
2. Configure the environment variables for the Hadoop daemons, such as `HADOOP_HOME`, `HADOOP_CONF_DIR`, `HADOOP_LOG_DIR`, `HADOOP_PID_DIR`, etc. You can edit the `hadoop-env.sh` file in the Hadoop configuration directory to set these variables. You can also set

the `JAVA_HOME` variable to point to the Java installation directory.

3. Configure the Hadoop parameters for the Hadoop daemons, such as the NameNode, DataNode, ResourceManager, NodeManager, etc. You can edit the `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, and `yarn-site.xml` files in the Hadoop configuration directory to set these parameters. You can refer to the Hadoop documentation for the details of each parameter and the recommended values for different cluster sizes and workloads. <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/ClusterSetup.html#Configuration>
4. Configure the list of nodes in the cluster, such as the master node, the worker nodes, and the secondary NameNode. You can edit the `masters` and `slaves` files in the Hadoop configuration directory to specify the hostnames or IP addresses of the nodes. The `masters` file should contain the hostname of the master node, which runs the NameNode and the ResourceManager. The `slaves` file should contain the hostnames of the worker nodes, which run the DataNode and the NodeManager. The secondary NameNode, which performs periodic checkpoints of the HDFS metadata, can be run on any node, but it is recommended to run it on a separate node from the master node.
5. Format the HDFS filesystem on the master node, which will initialize the NameNode and create the HDFS directory structure. You can run the command `hdfs namenode -format` as the HDFS user to format the filesystem. You should only do this once, when you set up the cluster for the first time, or when you want to erase all the data on the cluster.
6. Start the Hadoop daemons on all the nodes in the cluster, using the scripts provided by the Hadoop distribution. You can run the command `start-dfs.sh` as the HDFS user to start the NameNode, the DataNode, and the secondary NameNode. You can run the command `start-yarn.sh` as the YARN user to start the ResourceManager and the NodeManager. You can also use the `stop-dfs.sh` and `stop-yarn.sh` scripts to stop the daemons.
7. Verify the status of the Hadoop cluster, using the web interfaces or the command-line tools. You can access the web interface of the NameNode at `http://:9870`, which will show you the overview of the HDFS filesystem, such as the number of files, blocks, and nodes, the disk usage, the replication factor, etc. You can access the web interface of the ResourceManager at `http://:8088`, which will show you the overview of the YARN cluster, such as the number of applications, containers, and nodes, the resource usage, the queues, etc. You can also use the command `hdfs dfsadmin -report` as the HDFS user to get the report of the HDFS cluster, and the command `yarn node -list` as the YARN user to get the list of the YARN nodes.

Cluster Specification in Hadoop Environment

- A Hadoop cluster is a special type of computational cluster designed specif-

ically for storing and analyzing huge amounts of unstructured data in a distributed computing environment .

- A Hadoop cluster consists of a network of master and slave nodes that are connected to each other and run Hadoop's open source distributed processing software.
- The master nodes are responsible for managing the cluster resources, coordinating the data processing tasks, and maintaining the metadata. The slave nodes are responsible for storing the data and executing the data processing tasks .
- The main components of a Hadoop cluster are:
 - Hadoop Distributed File System (HDFS): A distributed file system that provides high-throughput access to the data stored in the cluster. HDFS consists of a NameNode that manages the file system metadata and a number of DataNodes that store the file blocks .
 - MapReduce: A programming model and framework for processing large-scale data sets in parallel. MapReduce consists of a JobTracker that schedules and monitors the jobs and a number of TaskTrackers that run the map and reduce tasks on the DataNodes .
 - YARN: A resource management layer that allocates the cluster resources to different applications. YARN consists of a Resource Manager that arbitrates the resources among the applications and a number of Node Managers that monitor the resource usage and report to the Resource Manager.
- To configure a Hadoop cluster, you will need to configure the environment in which the Hadoop daemons execute as well as the configuration parameters for the Hadoop daemons. The configuration files are stored in the \$HADOOP_CONF_DIR directory and include core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml .
- There are different types of Hadoop clusters based on the size, purpose, and architecture. Some common types are:
 - Single-node cluster: A cluster with only one node that runs all the Hadoop daemons. This is useful for testing and development purposes.
 - Multi-node cluster: A cluster with more than one node that runs the Hadoop daemons. This is useful for production and deployment purposes.
 - Pseudo-distributed cluster: A cluster with only one node that runs all the Hadoop daemons but simulates a distributed environment by using different ports and processes. This is useful for testing and debugging purposes.
 - Fully-distributed cluster: A cluster with multiple nodes that runs the Hadoop daemons in a distributed manner. This is useful for scaling and performance purposes.

Cluster Setup and Installation in Hadoop Environment

A Hadoop cluster is a collection of machines that run the Hadoop software and store and process large amounts of data. A Hadoop cluster can have one or more master nodes and many worker nodes. The master nodes run the Hadoop daemons that coordinate and manage the cluster, such as the NameNode, the SecondaryNameNode, the ResourceManager, and the JobTracker. The worker nodes run the Hadoop daemons that perform the actual data processing and storage, such as the DataNode and the TaskTracker.

To set up and install a Hadoop cluster, you need to follow these steps:

- Install the required software on all the nodes in the cluster, such as Java, SSH, and Hadoop.
- Configure the environment variables and the Hadoop configuration files on all the nodes, such as the `core-site.xml`, the `hdfs-site.xml`, the `mapred-site.xml`, and the `yarn-site.xml`.
- Start the Hadoop daemons on the master and the worker nodes, such as the NameNode, the DataNode, the ResourceManager, the NodeManager, and the JobTracker.
- Verify the status and the functionality of the cluster, such as the Hadoop web interface, the HDFS file system, and the MapReduce jobs.

The following points provide more details on each step:

- Installing the required software: You can install the Hadoop software by downloading the binary distribution from the official website and unpacking it on all the nodes, or by using a package manager such as `apt` or `yum`, depending on your operating system. You also need to install Java and SSH on all the nodes, as they are prerequisites for Hadoop. You can use the official guides for installing Java and SSH for your operating system, or follow the instructions from the Hadoop documentation .
- Configuring the environment variables and the Hadoop configuration files: You need to set the `JAVA_HOME` and `HADOOP_HOME` environment variables on all the nodes, as well as the `PATH` variable to include the Hadoop bin directory. You also need to edit the Hadoop configuration files on all the nodes, such as the `core-site.xml`, the `hdfs-site.xml`, the `mapred-site.xml`, and the `yarn-site.xml`, to specify the cluster settings, such as the hostnames and the ports of the master and the worker nodes, the HDFS replication factor, the YARN resource allocation, and the MapReduce framework. You can use the Hadoop documentation or the online tutorials for guidance on how to configure the Hadoop cluster.
- Starting the Hadoop daemons: You need to start the Hadoop daemons on the master and the worker nodes, using the `start-dfs.sh` and the `start-yarn.sh` scripts, or the service command if you installed Hadoop using a package manager. You can also use the `start-all.sh` script to start all the daemons at once, but this is not recommended for production clusters. You can use the `jps` command to check if the Hadoop daemons are running on each node.
- Verifying the status and the functionality of the cluster: You can use

the Hadoop web interface to monitor the cluster status and the performance metrics, such as the HDFS usage, the YARN applications, and the MapReduce jobs. You can access the web interface by using the URLs `http://:50070` for the NameNode, `http://:8088` for the ResourceManager, and `http://:19888` for the JobTracker. You can also use the Hadoop commands to interact with the HDFS file system and the MapReduce framework, such as the `hdfs dfs`, the `hadoop fs`, the `hadoop jar`, and the `yarn` commands. You can use the Hadoop documentation or the online tutorials for more information on how to use the Hadoop commands.

Hadoop configuration in Hadoop Environment

Hadoop configuration is the process of setting up the parameters and properties for the Hadoop daemons and services that run in a Hadoop cluster. Hadoop configuration can be done at different levels, such as HDFS, YARN, and Oozie. Hadoop configuration can affect the performance, security, and functionality of the Hadoop cluster.

Some of the common steps for Hadoop configuration are:

- Configuring the environment of Hadoop daemons using the scripts in the `etc/hadoop` directory, such as `hadoop-env.sh`, `mapred-env.sh`, and `yarn-env.sh`. These scripts can be used to set the `JAVA_HOME`, `HADOOP_CONF_DIR`, `HADOOP_LOG_DIR`, and other environment variables for the Hadoop daemons.
- Configuring the `core-site.xml` file in the `etc/hadoop` directory, which contains the common configuration properties for the Hadoop cluster, such as `fs.defaultFS`, `hadoop.tmp.dir`, and `io.file.buffer.size`.
- Configuring the `hdfs-site.xml` file in the `etc/hadoop` directory, which contains the configuration properties for the HDFS daemons, such as `dfs.namenode.name.dir`, `dfs.datanode.data.dir`, `dfs.replication`, and `dfs.blocksize`.
- Configuring the `yarn-site.xml` file in the `etc/hadoop` directory, which contains the configuration properties for the YARN daemons, such as `yarn.resourcemanager.address`, `yarn.nodemanager.resource.memory-mb`, `yarn.scheduler.minimum-allocation-mb`, and `yarn.application.classpath`.
- Configuring the `mapred-site.xml` file in the `etc/hadoop` directory, which contains the configuration properties for the MapReduce framework, such as `mapreduce.framework.name`, `mapreduce.jobtracker.address`, `mapreduce.map.memory.mb`, and `mapreduce.reduce.memory.mb`.
- Configuring the `oozie-site.xml` file in the `etc/oozie` directory, which contains the configuration properties for the Oozie service, such as `oozie.service.AuthorizationService.security.enabled`, `oozie.service.HadoopAccessorService.kerberos.enabled`, and `oozie.service.JPAService.jdbc.url`.

Hadoop configuration can be done manually by editing the XML files or using the Hadoop command-line interface. Hadoop configuration can also be done us-

ing tools such as Ambari, Cloudera Manager, or Hue, which provide graphical user interfaces for managing and monitoring the Hadoop cluster. Hadoop configuration should be done carefully and consistently across all the nodes in the cluster, as any misconfiguration can cause errors or failures in the Hadoop cluster. Hadoop configuration should also be updated whenever there are changes in the hardware, software, or requirements of the Hadoop cluster. Hadoop configuration is an essential part of Hadoop cluster setup and administration.

Security in Hadoop in Hadoop Environment

Security in Hadoop is the process of protecting the data and services of a Hadoop cluster from unauthorized access, modification, or disclosure. Security in Hadoop consists of four main aspects :

- **Authentication:** This is the process of verifying the identity of a user or a service before allowing them to access the Hadoop cluster. Authentication can be done using Kerberos, a network protocol that uses tickets to establish secure communication between the client and the server. Kerberos authentication ensures that only authorized users and services can access the Hadoop cluster and prevents impersonation attacks .
- **Authorization:** This is the process of granting or denying permissions to a user or a service to perform certain actions on the Hadoop cluster. Authorization can be done using HDFS file permissions, which control the read, write, and execute access to the files and directories in the Hadoop Distributed File System (HDFS). Authorization can also be done using Service Level Authorization, which controls the access to the Hadoop services such as MapReduce, YARN, Hive, HBase, etc. Service Level Authorization can be configured using Access Control Lists (ACLs) or Apache Ranger, a framework that provides centralized security administration and auditing for Hadoop services .
- **Auditing:** This is the process of recording and monitoring the activities and events that occur on the Hadoop cluster. Auditing can help to detect and prevent security breaches, identify and resolve performance issues, and comply with regulatory requirements. Auditing can be done using Apache Knox, a gateway that provides a single point of access and authentication for the Hadoop services and enables logging and auditing of the requests and responses. Auditing can also be done using Apache Atlas, a metadata management and governance solution that tracks the lineage and provenance of the data and the processes in the Hadoop cluster .
- **Data Confidentiality:** This is the process of protecting the data stored in the Hadoop cluster from unauthorized access or disclosure. Data confidentiality can be achieved using encryption, which transforms the data into an unreadable form that can only be decrypted by authorized parties. Encryption can be done at different levels, such as data at rest, data in transit, or data in use. Data at rest encryption refers to encrypting the data stored in HDFS or other storage systems using tools such

as HDFS Encryption Zones, Apache Hadoop Key Management Server (KMS), or Apache Hadoop Crypto Filesystem (CFS). Data in transit encryption refers to encrypting the data transferred between the nodes or the services in the Hadoop cluster using protocols such as SSL/TLS or SASL. Data in use encryption refers to encrypting the data processed by the Hadoop services using tools such as Apache Accumulo, a distributed key-value store that supports cell-level encryption .

Security in Hadoop is a complex and challenging task that requires careful planning, configuration, and monitoring. However, it is also essential to ensure the reliability, availability, and integrity of the data and the services in the Hadoop cluster. By following the best practices and using the available tools and frameworks, security in Hadoop can be achieved and maintained in a Hadoop environment.

Administering Hadoop in Hadoop Environment

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.
- HDFS is a distributed file system that stores data in blocks on multiple nodes in a cluster, providing high availability, fault tolerance, and scalability.
- MapReduce is a programming model that allows parallel processing of data on HDFS using two types of functions: map and reduce.
- Administering Hadoop involves configuring, monitoring, and managing the Hadoop cluster and its components, such as HDFS, MapReduce, YARN, Hive, HBase, etc.
- Some of the tasks involved in administering Hadoop are:
 - Installing and setting up Hadoop on a cluster of nodes, using tools such as Apache Ambari, Cloudera Manager, or Hortonworks Data Platform.
 - Configuring Hadoop parameters, such as memory, CPU, disk, network, and security settings, using configuration files, such as `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, etc.
 - Managing Hadoop services, such as starting, stopping, restarting, and checking the status of HDFS, MapReduce, YARN, etc., using commands, such as `hadoop`, `hdfs`, `yarn`, etc.
 - Monitoring Hadoop performance, such as resource utilization, job execution, data throughput, etc., using tools, such as Ganglia, Nagios, or Hadoop Metrics.

- Troubleshooting Hadoop issues, such as log analysis, error handling, debugging, etc., using tools, such as Hadoop logs, Hadoop shell, or Hadoop web UI.
- Securing Hadoop cluster, such as enabling authentication, authorization, encryption, and auditing, using tools, such as Kerberos, ACLs, SSL, or Ranger.
- Optimizing Hadoop cluster, such as tuning Hadoop parameters, balancing HDFS load, optimizing MapReduce jobs, etc., using tools, such as Hadoop balancer, Hadoop scheduler, or Hadoop profiler.
- Scaling Hadoop cluster, such as adding or removing nodes, upgrading or downgrading Hadoop versions, etc., using tools, such as Hadoop decommission, Hadoop upgrade, or Hadoop distcp.
- Backing up and restoring Hadoop data, such as creating snapshots, replicating data, or recovering data, using tools, such as HDFS snapshot, HDFS distcp, or HDFS fsck.

HDFS monitoring & maintenance in Hadoop Environment

- HDFS (Hadoop Distributed File System) is a distributed file system that stores large data sets across multiple nodes in a cluster. HDFS consists of two core components: NameNode and DataNode .
- NameNode is the master node that manages the file system metadata, such as the directory tree, file permissions, and block locations. DataNode is the worker node that stores the actual data blocks on the local disks.
- HDFS monitoring is the process of checking the health and performance of the HDFS cluster, such as the disk usage, memory usage, network traffic, file operations, replication, and availability of the nodes.
- HDFS maintenance is the process of performing regular tasks to ensure the optimal operation of the HDFS cluster, such as balancing the data distribution, cleaning the trash, upgrading the software, and recovering from failures.
- HDFS monitoring and maintenance can be done using various tools and commands, such as:
 - HDFS commands: These are the command-line tools that can be used to interact with the HDFS file system, such as **hdfs dfs**, **hdfs fsck**, **hdfs balancer**, **hdfs fetchdt**, etc. These commands can be used to perform basic file operations, check the file system status, balance the data blocks, get delegation tokens, and more.
 - HDFS web interface: This is the web-based user interface that can be accessed through the NameNode's URL, such as **http://namenode:50070/**. This interface can be used to view the file system information, browse the files and directories, monitor the cluster metrics, and perform administrative tasks.
 - HDFS JMX interface: This is the Java Management Extensions (JMX) interface that can be used to access the management beans (MBeans) that expose the internal state and statistics of the HDFS

components, such as the NameNode, DataNode, JournalNode, etc. These MBeans can be accessed through the JMX URL, such as `http://namenode:8000/jmx`. These MBeans can be used to monitor the cluster metrics, such as the number of files, blocks, transactions, etc.

- **HDFS monitoring tools:** These are the third-party tools that can be used to collect, analyze, and visualize the HDFS metrics, such as the disk usage, memory usage, network traffic, file operations, replication, and availability of the nodes. Some examples of these tools are Ganglia, Nagios, Zabbix, Grafana, etc. These tools can be used to set up alerts, dashboards, and reports for the HDFS cluster.

Hadoop benchmarks in Hadoop Environment

Hadoop benchmarks are tools or applications that can be used to measure the performance of a Hadoop cluster in terms of various aspects, such as data processing, data storage, data transfer, etc. Hadoop benchmarks can help users to evaluate the efficiency, scalability, reliability, and suitability of their Hadoop cluster for different use cases and workloads.

Some of the common Hadoop benchmarks are:

- **TestDFSIO:** This benchmark tests the read and write performance of the Hadoop Distributed File System (HDFS). It generates a number of files of a given size and writes them to HDFS using one map task per file. Then, it reads the same files from HDFS using one map task per file and reports the throughput and latency metrics.
- **TeraSort:** This benchmark tests the sorting performance of Hadoop MapReduce. It consists of three components: TeraGen, TeraSort, and TeraValidate. TeraGen generates a large amount of random data and writes it to HDFS. TeraSort sorts the data using MapReduce and writes the sorted output to HDFS. TeraValidate verifies that the output is correctly sorted and reports the number of errors.
- **WordCount:** This benchmark tests the basic functionality of Hadoop MapReduce. It counts the number of occurrences of each word in a given input file or directory and writes the results to HDFS. It is a simple and widely used example of a MapReduce application.
- **PiEstimator:** This benchmark tests the computation performance of Hadoop MapReduce. It estimates the value of pi using a Monte Carlo method. It launches a number of map tasks, each of which generates a number of random points and counts how many of them fall inside a unit circle. Then, it launches a single reduce task, which calculates the ratio of the points inside the circle to the total number of points and multiplies it by four to get an approximation of pi.
- **Nutch:** This benchmark tests the web crawling and indexing performance of Hadoop MapReduce. It uses the Nutch open source project, which is a web crawler and search engine based on Hadoop. It crawls a set of web

pages and indexes them using MapReduce. It reports the number of pages crawled, the size of the index, and the time taken.

Hadoop benchmarks can be run using the `hadoop jar` command with the appropriate parameters and options. For example, to run the TestDFSIO benchmark with 10 files of 1 GB each, the command would be:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-jobclient-tests.jar
TestDFSIO -write -nrFiles 10 -fileSize 1000
```

Hadoop benchmarks can also be customized or modified to suit different scenarios and requirements. For example, one can change the input size, the number of map and reduce tasks, the compression codec, the replication factor, the block size, etc. to see how they affect the performance.

Hadoop benchmarks are useful for testing and tuning a Hadoop cluster, but they are not sufficient to capture the real-world complexity and diversity of Hadoop applications. Therefore, users should also run their own benchmarks or use other benchmarks that are more relevant to their specific domain and workload.

Hadoop in the cloud

- Hadoop is a software framework that allows users to process large data sets in a distributed environment using a cluster of computers.
- Hadoop consists of four main modules: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common.
- Hadoop was designed for physical data centers, where data is stored and processed on the same machines.
- Cloud computing is a model of delivering computing resources as a service over the internet, where users can access and scale them on demand.
- Cloud computing offers several benefits for Hadoop users, such as lower upfront costs, faster provisioning, elasticity, and availability.
- However, running Hadoop on the cloud also poses some challenges, such as data transfer costs, security, performance, and compatibility.
- There are different ways to run Hadoop on the cloud, such as using a cloud provider's managed service, deploying Hadoop on virtual machines, or using a hybrid approach.
- Some examples of cloud providers that offer Hadoop services are Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure.

Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

- Hadoop is a framework and suite of tools that tackle the many challenges in dealing with big data. It consists of several components, such as HDFS, MapReduce, YARN, Hive, Pig, Spark, Oozie, Zookeeper, Mahout, etc.

- HDFS is the distributed file system that stores large volumes of data across multiple nodes in a cluster. It provides high availability, fault tolerance, scalability and reliability.
- MapReduce is the programming model that allows parallel processing of large data sets using a map function and a reduce function. It divides the data into smaller chunks and assigns them to different nodes for processing. The results are then aggregated and returned.
- YARN (Yet Another Resource Negotiator) is the resource management layer that allocates and monitors the resources (CPU, memory, disk, network) for the applications running on the cluster. It also provides scheduling, security and isolation features. YARN is also called as the operating system of Hadoop.
- Hive is a data warehouse tool that provides SQL-like interface to query and analyze data stored in HDFS. It supports various data formats, such as text, JSON, ORC, Parquet, etc. It also allows creating tables, views, partitions, indexes and user-defined functions.
- Pig is a scripting language that allows data analysis and transformation using a set of operators, such as load, store, filter, join, group, etc. It can run on top of MapReduce or Tez engines and can handle structured, semi-structured and unstructured data.
- Spark is a fast and general-purpose engine for large-scale data processing. It supports batch, streaming, interactive and machine learning applications. It can run on top of Hadoop, Mesos, Kubernetes or standalone clusters and can access data from various sources, such as HDFS, Hive, HBase, Cassandra, MongoDB, etc.
- Scala is a programming language that combines object-oriented and functional paradigms. It is concise, expressive and interoperable with Java. It is one of the main languages used to write Spark applications, along with Python, R and Java.
- NoSQL databases are databases that do not use SQL as the query language and do not follow the relational model. They are designed to handle large volumes of data with high performance, scalability and availability. They typically use key-value, document, columnar, graph or hybrid data models.
- MongoDB is a popular NoSQL database that uses the document data model. It stores data as JSON-like documents with dynamic schemas. It supports various features, such as indexing, aggregation, replication, sharding, transactions, etc. It can be integrated with Hadoop and Spark for data analysis and processing.

Hadoop Eco System and YARN

- Hadoop is an open source framework that allows for distributed processing of large and complex data sets across clusters of computers.
- Hadoop consists of four main modules: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce and Hadoop YARN.
- Hadoop Common provides the common utilities and libraries that are used

by other Hadoop modules.

- HDFS is a distributed file system that stores data across multiple nodes in a cluster, providing high availability, fault tolerance and scalability.
- MapReduce is a programming model and execution engine that enables parallel processing of data using key-value pairs.
- YARN is the resource management and job scheduling layer of Hadoop that allocates and manages the resources and keeps all things working as they should.
- YARN stands for Yet Another Resource Negotiator. It was introduced in Hadoop 2.0 to overcome the limitations of MapReduce in Hadoop 1.0, such as low resource utilization, lack of support for non-MapReduce applications and fixed job execution model.
- YARN splits up the functionalities of resource management and job scheduling/monitoring into separate daemons: a global ResourceManager (RM) and per-application ApplicationMaster (AM).
- The RM is responsible for managing the cluster resources, such as memory, CPU, disk and network bandwidth. It also arbitrates resource requests from competing applications.
- The AM is responsible for coordinating the execution of a specific application, such as a MapReduce job or a Spark application. It requests resources from the RM, communicates with the NodeManagers (NMs) that run the application tasks and monitors the progress and status of the application.
- The NM is a daemon that runs on each node in the cluster and manages the containers that run the application tasks. A container is a unit of resource allocation that consists of a certain amount of memory, CPU, disk and network bandwidth. The NM also monitors the health and performance of the node and reports to the RM.
- YARN enables the Hadoop ecosystem to be more flexible, efficient and scalable. It allows for multiple types of applications to run on the same cluster, such as MapReduce, Spark, Hive, Pig, HBase, etc. It also supports dynamic resource allocation, high availability, security and isolation.

Hadoop ecosystem components

The Hadoop ecosystem is a collection of software components and tools that work together to provide a framework for distributed data processing and storage. The Hadoop ecosystem consists of four main components: data storage, data processing, data access, and data management.

- Data storage: This component is responsible for storing large volumes of data across multiple nodes in a cluster. The most common data storage component in the Hadoop ecosystem is Hadoop Distributed File System (HDFS), which is a distributed file system that runs on Java and splits data into blocks and distributes them across the cluster. Another data storage component is HBase, which is a NoSQL database that stores data in key-value pairs and supports random access and real-time analysis.

- **Data processing:** This component is responsible for processing the data stored in the cluster using parallel and distributed algorithms. The most common data processing component in the Hadoop ecosystem is MapReduce, which is a programming model that divides a task into smaller sub-tasks (map) and then combines the results (reduce) . Another data processing component is Spark, which is a fast and general-purpose engine that supports batch, streaming, SQL, machine learning, and graph processing .
- **Data access:** This component is responsible for providing access to the data stored and processed in the cluster using various tools and languages. The most common data access component in the Hadoop ecosystem is Hive, which is a data warehouse system that allows querying and analyzing large datasets using a SQL-like language called HiveQL . Another data access component is Pig, which is a scripting language that allows performing complex data transformations and analysis using a high-level syntax .
- **Data management:** This component is responsible for managing the data lifecycle, workflow, security, and governance in the cluster. The most common data management component in the Hadoop ecosystem is YARN, which is a resource manager that allocates and schedules resources for different applications running on the cluster . Another data management component is Oozie, which is a workflow scheduler that coordinates and executes jobs based on predefined dependencies and triggers .

Some other components of the Hadoop ecosystem are:

- **Sqoop,** which is a tool that transfers data between Hadoop and relational databases .
- **Flume,** which is a tool that collects and transports streaming data from various sources to HDFS .
- **Zookeeper,** which is a service that provides coordination, synchronization, and configuration management for distributed applications .

The Hadoop ecosystem is constantly evolving and adding new components and features to meet the growing demands of big data analytics. The Hadoop ecosystem components work together to provide a scalable, reliable, and flexible platform for data storage and processing.

Schedulers in Hadoop Ecosystem

Schedulers are algorithms that manage the execution of tasks in a Hadoop cluster. They help in optimizing the utilization of resources and ensuring fair access to different users and applications. There are mainly three types of schedulers in Hadoop:

- **FIFO Scheduler:** This is the default scheduler in Hadoop. It places the applications in a queue and executes them in the order of their submission (first in, first out). It does not consider the priority or the resource require-

ments of the applications. It is simple and suitable for small clusters with homogeneous workloads .

- **Capacity Scheduler:** This scheduler allows multiple queues with different capacities and priorities. Each queue can have a minimum and a maximum capacity, and a weight to determine its share of resources. The scheduler allocates resources to the queues based on their capacities and weights, and within each queue, it uses FIFO scheduling. It also supports preemption, which means that it can reclaim resources from low-priority queues if they exceed their maximum capacity and give them to high-priority queues. This scheduler is designed for large clusters with multiple tenants and heterogeneous workloads .
- **Fair Scheduler:** This scheduler aims to provide fair and equal access to resources for all users and applications. It creates a separate queue for each user or group, and allocates resources to the queues based on their demand and the available resources. It also supports preemption, which means that it can balance the resource allocation among the queues if some of them are underutilized or overutilized. It also supports hierarchical queues, weight-based shares, and pool-specific settings. This scheduler is flexible and adaptable for various scenarios and workloads .

Fair and Capacity in Hadoop Ecosystem

- Hadoop is a batch processing ecosystem that can handle large-scale data analysis using distributed computing.
- Hadoop has a resource management layer called YARN (Yet Another Resource Negotiator) that allocates resources to different applications running on the cluster.
- Hadoop also has a scheduling layer that decides the order and priority of the applications that request resources from YARN.
- There are mainly three types of schedulers in Hadoop: FIFO, Capacity, and Fair.
- FIFO (First In First Out) Scheduler is the simplest and default scheduler that assigns resources to applications based on their submission time. It does not consider the priority, size, or resource requirements of the applications. It is suitable for small clusters with homogeneous and predictable workloads.
- Capacity Scheduler is a more advanced scheduler that allows multiple queues to be created, each with a fixed percentage of the cluster capacity. Each queue can have its own access control, priority, and resource limits. The Capacity Scheduler can handle heterogeneous and dynamic workloads, and can support multi-tenancy and preemption.
- Fair Scheduler is another advanced scheduler that aims to provide fair and equal share of resources to all applications over time. It does not require predefined queues or capacities, but dynamically balances the resources among the running applications. The Fair Scheduler can also handle heterogeneous and dynamic workloads, and can support multi-tenancy,

preemption, and weight-based priorities.

Hadoop 2.0 New Features - NameNode high availability

- NameNode is the master node in HDFS that maintains the filesystem tree and the metadata of all the files and directories.
- In Hadoop 1.x, NameNode was a single point of failure (SPOF) in an HDFS cluster. If the NameNode failed or became unavailable, the entire cluster would be inaccessible until the NameNode was restored or replaced.
- Hadoop 2.0 overcomes this SPOF problem by providing support for multiple NameNodes. It introduces Hadoop 2.0 High Availability feature that brings in an extra NameNode (Passive Standby NameNode) to the Hadoop Architecture which is configured for automatic failover .
- The Active NameNode and the Standby NameNode are connected to a shared storage system (such as NFS or Quorum Journal Manager) that stores the edit logs of the HDFS namespace. The Standby NameNode constantly reads the edit logs from the shared storage and applies them to its own namespace in memory, so that it is always synchronized with the Active NameNode .
- The DataNodes in the cluster send block reports and heartbeats to both the Active and the Standby NameNodes, so that they are aware of the location and health of the blocks.
- The clients of the HDFS cluster are configured with the logical URI of the cluster, which maps to the addresses of both the NameNodes. The clients can transparently access the HDFS cluster even if one of the NameNodes fails, as the failover process is handled by the Hadoop framework.
- The Hadoop 2.0 High Availability feature enables the HDFS cluster to run 24/7 without any downtime or data loss due to NameNode failure. It also improves the performance and scalability of the HDFS cluster by reducing the load on the single NameNode.

HDFS federation in Hadoop ecosystem

- HDFS federation is a feature introduced in Hadoop 2 that enhances the existing HDFS architecture by adding multiple NameNode/namespaces support to HDFS .
- This allows the use of more than one NameNode/namespace, each managing a separate namespace volume, which is a self-contained unit of directories, files and blocks .
- The benefits of HDFS federation are:
 - It overcomes the isolation, scalability and performance limitations of the prior HDFS architecture, which had a single NameNode/namespace for the entire cluster .
 - It improves the availability and reliability of the system by allowing the failure of one NameNode/namespace to be isolated from the others.

- It increases the throughput and efficiency of the system by distributing the workload and metadata across multiple NameNodes/namespaces.
- It enables the use of heterogeneous storage types and policies for different namespaces, such as SSD, HDD, RAM, etc.
- It opens up the architecture for future innovations and extensions.
- The main components of HDFS federation are:
 - NameNode: It is the master node that manages the metadata and namespace operations for a namespace volume. It also communicates with the DataNodes and clients.
 - DataNode: It is the slave node that stores the data blocks for one or more namespace volumes. It reports the block locations and status to the NameNodes and serves the read/write requests from the clients.
 - Block Pool: It is the set of blocks that belong to a namespace volume. It is identified by a unique ID and is managed by a NameNode. A DataNode can store blocks from multiple block pools.
 - Namespace ID: It is a unique identifier for a namespace volume. It is used to associate a block pool with a NameNode.
 - Client: It is the application that accesses the data stored in HDFS. It interacts with the NameNodes to perform namespace operations and with the DataNodes to perform data operations.

MRv2 in Hadoop ecosystem

- MRv2 stands for MapReduce version 2, which is an application framework that runs within YARN (Yet Another Resource Negotiator) .
- YARN is a new component in Hadoop 2 that separates the resource management and scheduling tasks from the MapReduce layer .
- YARN allows other application engines to utilize Hadoop and its distributed file system (HDFS), while also improving the performance and scalability of MapReduce .
- MRv2 is backward compatible with the org.apache.hadoop.mapred APIs of Hadoop 1, which means that the compiled binaries can run without any modification on the new framework .
- MRv2 also supports a new set of org.apache.hadoop.mapreduce APIs that offer more flexibility and functionality .
- MRv2 can run on different schedulers, such as the Capacity Scheduler, the Fair Scheduler, and the FIFO Scheduler, which provide different ways of allocating resources and prioritizing jobs .
- MRv2 enables multi-tenancy support, which means that different users and applications can share the same cluster and data nodes without interfering with each other .

YARN

- Yarn is a long continuous length of interlocked fibres, used in sewing, cro-

cheting, knitting, weaving, embroidery, ropemaking, and the production of textiles.

- Thread is a type of yarn intended for sewing by hand or machine.
- Yarn can be made from natural or synthetic materials, such as cotton, wool, silk, acrylic, nylon, polyester, etc.
- Yarn can vary in thickness, weight, texture, color, and ply.
- Ply refers to the number of strands twisted together to make a single yarn.
- Yarn can be classified into different categories based on the yarn weight system, which measures the thickness of the yarn and the gauge (number of stitches per inch) it produces when knitted or crocheted.
- Some common yarn weight categories are lace, fingering, sport, DK, worsted, bulky, and super bulky.
- Yarn can also be classified into different types based on the way it is spun, such as woolen, worsted, semi-worsted, and novelty.
- Woolen yarns are spun from short and fuzzy fibres, creating a soft and airy yarn that traps air and provides warmth.
- Worsted yarns are spun from long and smooth fibres, creating a dense and strong yarn that has good stitch definition and drapes well.
- Semi-worsted yarns are a blend of woolen and worsted spinning techniques, creating a yarn that has some characteristics of both.
- Novelty yarns are yarns that have special effects, such as texture, color, or shape, that make them stand out from regular yarns.
- Some examples of novelty yarns are bouclé, eyelash, ribbon, chenille, metallic, and self-striping yarns.
- Yarn is also the name of a package manager that doubles down as project manager for JavaScript projects.
- Yarn allows users to install, update, and manage dependencies for their projects, as well as run scripts, create workspaces, and publish packages.
- Yarn is compatible with the npm registry and can work alongside npm or replace it.
- Yarn is faster, more reliable, and more secure than npm, according to its developers.
- Yarn can be installed on various operating systems, such as Windows, macOS, Linux, and BSD, using different methods, such as npm, Homebrew, Chocolatey, Scoop, etc.
- Yarn can be used with various tools and frameworks, such as React, Angular, Vue, Next.js, Gatsby, etc.

Running MRv1 in YARN

- MRv1 is the original MapReduce framework that runs on Hadoop 1.x. It can also run on YARN, which is the resource management layer introduced in Hadoop 2.x.
- To run MRv1 applications on YARN, you need to use the `yarn` command in the Hadoop-YARN bin folder rather than the `hadoop` command. For

example, to run a word count job, you can use the following command:

```
yarn jar /usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar  
wordcount input output
```

- You can also use the `hadoop` command to run MRv1 applications on YARN, but it is not recommended as it may cause compatibility issues.
- To monitor MRv1 applications on YARN, you can use the ResourceM-anager web interface, which shows the basic cluster metrics, list of ap-plications, and nodes associated with the cluster. You can access the ResourceManager web interface by using the following URL:

```
http://<ResourceManager-Host>:8088
```

- The ResourceManager web interface shows the application ID, name, type, user, queue, state, final status, progress, and tracking URL for each appli-cation. You can click on the application ID to see more details about the application, such as the application master, containers, logs, and counters.
- You can also use the `yarn` command to monitor MRv1 applications on YARN. For example, you can use the following command to list all the applications:

```
yarn application -list
```

- You can use the following command to get the status of a specific appli-cation:

```
yarn application -status <application_id>
```

- You can use the following command to kill a specific application:

```
yarn application -kill <application_id>
```

NoSQL Databases

- NoSQL databases are a type of database that do not use the relational model or SQL (Structured Query Language) to store and manipulate data.
- NoSQL databases are designed to handle large volumes of unstructured, semi-structured, or dynamic data that may not fit well in a rigid schema of tables and columns.
- NoSQL databases are often used for applications that require high scala-bility, availability, performance, or flexibility, such as web, social media, big data, or real-time analytics.
- NoSQL databases can be classified into four main categories based on their data model: key-value, document, column, and graph.
 - Key-value databases store data as pairs of keys and values, where the key is a unique identifier and the value can be any type of data. Examples of key-value databases are Redis, DynamoDB, and Riak.

- Document databases store data as documents, which are collections of fields and values that can be nested and have different structures. Examples of document databases are MongoDB, CouchDB, and Elasticsearch.
- Column databases store data as columns, which are groups of values that share the same attribute. Columns are organized into column families, which are similar to tables in relational databases. Examples of column databases are Cassandra, HBase, and Bigtable.
- Graph databases store data as nodes and edges, which represent entities and relationships in a graph structure. Nodes and edges can have properties and labels that describe their attributes and types. Examples of graph databases are Neo4j, OrientDB, and ArangoDB.

Introduction to NoSQL databases

- NoSQL databases are databases that do not use the relational model or SQL (Structured Query Language) to store and manipulate data.
- NoSQL databases can handle large volumes of unstructured, semi-structured, or structured data that may change rapidly or frequently.
- NoSQL databases are designed to be scalable, distributed, and fault-tolerant, and to provide high performance and availability.
- NoSQL databases can be classified into four main types based on their data model: document, key-value, wide-column, and graph.
- Document databases store data as documents, which are collections of key-value pairs that can have nested structures. Each document has a unique identifier and can have different fields and schemas. Examples of document databases are MongoDB, CouchDB, and Elasticsearch.
- Key-value databases store data as pairs of keys and values, where the key is a unique identifier and the value can be any type of data. Key-value databases are simple and fast, but they do not support complex queries or relationships. Examples of key-value databases are Redis, DynamoDB, and Couchbase.
- Wide-column databases store data as rows and columns, but unlike relational databases, the columns can vary for each row and can be nested. Wide-column databases are suitable for storing sparse and dynamic data, such as web logs or sensor data. Examples of wide-column databases are Cassandra, HBase, and Bigtable.
- Graph databases store data as nodes and edges, where the nodes represent entities and the edges represent relationships. Graph databases are ideal for modeling complex and interconnected data, such as social networks or recommendation systems. Examples of graph databases are Neo4j, OrientDB, and ArangoDB.

MongoDB

- MongoDB is an open source, nonrelational database management system (DBMS) that uses flexible documents instead of tables and rows to process and store various forms of data .
- MongoDB is a document database, which means it stores data in JSON-like documents with optional schemas . This allows for more flexibility and scalability than traditional relational databases that use fixed schemas and tables.
- MongoDB is a distributed database at its core, so high availability, horizontal scaling, and geographic distribution are built in and easy to use. MongoDB can run on multiple servers and replicate data across them for fault tolerance and load balancing.
- MongoDB supports ad-hoc queries, secondary indexing, and real-time aggregations to provide powerful ways to access and analyze your data. MongoDB also supports various programming languages and frameworks through its drivers and APIs.
- MongoDB is free to use under the Server Side Public License (SSPL), which is a copyleft license that requires users to make their source code available if they offer MongoDB as a service . MongoDB also offers a cloud-based version called MongoDB Atlas, which is a fully managed and secure database service .

Introduction to MongoDB

MongoDB is a cross-platform, document-oriented database that works on the concept of collections and documents. MongoDB is a NoSQL database that stores data in JSON-like documents with dynamic schemas, meaning that documents in the same collection do not need to have the same structure or fields. MongoDB offers high performance, high availability, and easy scalability for data storage and manipulation.

Some of the key features of MongoDB are:

- MongoDB supports a rich query language that allows users to perform complex queries, aggregations, and text searches on the data.
- MongoDB supports various data types, such as strings, numbers, dates, arrays, embedded documents, and binary data.
- MongoDB provides various indexing options, such as single-field, compound, multi-key, geospatial, text, and hashed indexes, to optimize the performance of queries.
- MongoDB supports replication, which is the process of synchronizing data across multiple servers, to provide high availability and data redundancy.
- MongoDB supports sharding, which is the process of distributing data across multiple servers, to provide horizontal scalability and handle large volumes of data.
- MongoDB supports transactions, which are a sequence of operations that

are executed atomically and in isolation, to ensure data consistency and integrity.

- MongoDB provides various tools and drivers, such as MongoDB Compass, MongoDB Atlas, MongoDB Shell, and MongoDB Drivers, to help users interact with the database and perform various tasks.

Data Types in MongoDB

MongoDB is a document-oriented database that stores data in BSON format, which is a binary-encoded version of JSON. BSON supports various data types, some of which are common to other programming languages, and some of which are specific to MongoDB. Here are some of the data types in MongoDB:

- **String:** This is the most commonly used data type to store text data. Strings in MongoDB must be UTF-8 valid. Example: `{"name": "Alice"}`
- **Integer:** This type is used to store numerical values that can be either 32-bit or 64-bit, depending on the server. Example: `{"age": 25}`
- **Double:** This type is used to store floating-point values or decimal numbers. Example: `{"price": 9.99}`
- **Boolean:** This type is used to store a logical value of either true or false. Example: `{"active": true}`
- **Date:** This type is used to store a date or a timestamp. MongoDB stores dates as milliseconds since the Unix epoch (January 1, 1970). Example: `{"createdAt": new Date()}`
- **ObjectId:** This type is used to store a unique identifier for each document in a collection. MongoDB automatically generates an ObjectId for each document if not specified. Example: `{"_id": ObjectId("61c9a0a7f0f4f8c7f0a7f0f4")}`
- **Array:** This type is used to store an ordered list of values, which can be of any data type. Example: `{"colors": ["red", "green", "blue"]}`
- **Object:** This type is used to store a nested document or a subdocument, which can have its own fields and values. Example: `{"address": {"street": "Main St", "city": "New York", "zip": "10001"}}`
- **Null:** This type is used to store a null value or a missing field. Example: `{"middleName": null}`
- **Binary Data:** This type is used to store binary data or byte arrays, such as images or files. Example: `{"photo": BinData(0, "iVBORw0KGgoAAAANSUhEUgAAABwAAAACCAIAAADYTZjNAAAACXBIWXMAAAAsTAAALEw")}`

Creating documents in MongoDB

- A document is a basic unit of data in MongoDB. It is a JSON-like object that can have fields of different types and values.
- To create documents in MongoDB, you can use one of the following methods: `insertOne()`, `insertMany()`, or `insert()`.
- The `insertOne()` method inserts a single document into a collection. The syntax is:

`db.collection.insertOne(document, options)`

- The **document** parameter is the document to insert. The **options** parameter is an optional object that can specify write concern, bypass document validation, etc.
- The **insertMany()** method inserts an array of documents into a collection. The syntax is:

`db.collection.insertMany(documents, options)`

- The **documents** parameter is an array of documents to insert. The **options** parameter is similar to the one for **insertOne()**.
- The **insert()** method inserts one or more documents into a collection. The syntax is:

`db.collection.insert(documents, options)`

- The **documents** parameter can be either a single document or an array of documents. The **options** parameter is similar to the one for **insertOne()** and **insertMany()**.
- All these methods return a **WriteResult** object that contains information about the operation, such as the number of documents inserted, the **_id** values of the inserted documents, etc.
- If an inserted document does not have an **_id** field, MongoDB will automatically generate an **ObjectId** value for it. The **_id** field is the primary key of the document and must be unique within the collection.
- To create a collection and its first document at the same time, you can use the **create()** command or the MongoDB Compass GUI. The syntax for the **create()** command is:

`db.createCollection(name, options)`

- The **name** parameter is the name of the collection. The **options** parameter is an optional object that can specify the collection properties, such as the validation rules, the storage engine, the capped size, etc.

Updating documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To update documents in MongoDB, we can use the **updateOne()**, **updateMany()**, or **replaceOne()** methods of the **db.collection** object.
- The **updateOne()** method updates the first document that matches the filter condition with the specified update operation.
- The **updateMany()** method updates all the documents that match the filter condition with the specified update operation.
- The **replaceOne()** method replaces the first document that matches the filter condition with the specified replacement document.
- The update operation can use the following operators:

- **\$set**: sets the value of a field or creates a new field if it does not exist.
- **\$unset**: removes the specified field from the document.
- **\$inc**: increments the value of a field by a specified amount.
- **\$mul**: multiplies the value of a field by a specified amount.
- **\$rename**: renames a field.
- **\$min**: updates the value of a field to the specified value if it is less than the current value.
- **\$max**: updates the value of a field to the specified value if it is greater than the current value.
- **\$currentDate**: sets the value of a field to the current date, either as a Date or a Timestamp.
- **\$addToSet**: adds a value to an array field only if it does not already exist in the array.
- **\$push**: appends a value to an array field.
- **\$pop**: removes the first or last element of an array field.
- **\$pull**: removes all instances of a value from an array field.
- **\$pullAll**: removes all instances of the specified values from an array field.
- The update methods return a **WriteResult** object that contains information about the number of matched and modified documents, as well as any errors or warnings.
- Example: To update the name and age of the first document in the **users** collection where the email is “user@example.com”, we can use the following command:

```
db.users.updateOne(
  { email: "user@example.com" }, // filter condition
  { $set: { name: "John", age: 25 } } // update operation
)
```

Deleting documents in MongoDB

MongoDB is a document-oriented database that stores data in collections of JSON-like documents. To delete documents from a collection, you can use one of the following methods:

- The **db.collection.remove()** method: This method takes a query filter as a parameter and deletes all the documents that match the filter. You can also specify an optional second parameter to limit the deletion to just one document. For example, **db.users.remove({name: "Alice"})** will delete all the documents in the **users** collection where the **name** field is “Alice”. **db.users.remove({name: "Alice"}, true)** will delete only one document that matches the filter.
- The **delete** command: This command takes a query filter as a parameter and deletes all the documents that match the filter. You can also specify an optional second parameter to limit the deletion to just one document. For

example, `db.runCommand({delete: "users", deletes: [{q: {name: "Alice"}}, {limit: 0}]})` will delete all the documents in the `users` collection where the `name` field is "Alice". `db.runCommand({delete: "users", deletes: [{q: {name: "Alice"}}, {limit: 1}]})` will delete only one document that matches the filter.

- The `db.collection.deleteOne()` method: This method takes a query filter as a parameter and deletes the first document that matches the filter. For example, `db.users.deleteOne({name: "Alice"})` will delete the first document in the `users` collection where the `name` field is "Alice".
- The `db.collection.deleteMany()` method: This method takes a query filter as a parameter and deletes all the documents that match the filter. For example, `db.users.deleteMany({name: "Alice"})` will delete all the documents in the `users` collection where the `name` field is "Alice".

Some important points to note about deleting documents in MongoDB are:

- Deleting documents does not affect the indexes on the collection. The index entries for the deleted documents are removed when the collection is compacted or reindexed.
- Deleting documents does not free up disk space immediately. The deleted documents are marked as deleted and the space they occupy can be reused by new documents. To reclaim the disk space, you can use the `db.collection.reIndex()` method or the `compact` command.
- Deleting documents does not affect the `_id` field of the remaining documents. The `_id` field is immutable and unique for each document in a collection.

Querying documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To query documents in MongoDB, you can use the `find()` method on a collection, which returns a cursor to the matching documents.
- The `find()` method accepts two optional parameters: a query filter and a projection.
- The query filter is an object that specifies the conditions for selecting documents. You can use various operators to construct complex queries, such as `$eq`, `$gt`, `$in`, `$and`, `$or`, etc.
- The projection is an object that specifies which fields to include or exclude from the returned documents. You can use `1` or `true` to include a field, and `0` or `false` to exclude a field. You can also use the `$` operator to project specific elements from an array field.
- For example, the following query finds all documents in the `users` collection where the `age` is greater than 25 and the `name` starts with "A", and returns only the `name` and `email` fields:

```
db.users.find(
```

```

{ age: { $gt: 25 }, name: { $regex: "^A" } },
{ name: 1, email: 1, _id: 0 }
)

```

- You can also use the `findOne()` method to return only the first matching document, or the `count()` method to return the number of matching documents.
- To sort the results, you can use the `sort()` method on the cursor, which accepts an object that specifies the sort order for each field. You can use 1 for ascending order and -1 for descending order.
- To limit the number of results, you can use the `limit()` method on the cursor, which accepts a positive integer as the maximum number of documents to return.
- To skip some results, you can use the `skip()` method on the cursor, which accepts a positive integer as the number of documents to skip.
- For example, the following query finds the 10 youngest users in the `users` collection, sorted by age in ascending order, and skips the first 5 results:

```

db.users.find().sort({ age: 1 }).limit(10).skip(5)

```

Indexing in MongoDB

- Indexing is a technique that improves the efficiency of query processing in MongoDB by storing some information related to the documents in a special data structure .
- Indexes are ordered by the value of the field or fields specified in the index, which supports fast equality matches and range-based queries.
- Indexes can also be used to sort the query results by using the ordering in the index.
- MongoDB provides a method called `createIndex()` to create an index and a method called `dropIndex()` to drop an index .
- The syntax of `createIndex()` is `db.collection.createIndex(keys, options)`, where `keys` is a document that specifies the field or fields to index and the index type, and `options` is a document that specifies the additional properties of the index.
- The syntax of `dropIndex()` is `db.collection.dropIndex(index)`, where `index` is the name or specification of the index to drop.
- MongoDB supports various types of indexes, such as:
 - Single field index: an index on a single field of a document.
 - Compound index: an index on multiple fields of a document.
 - Multikey index: an index on a field that holds an array value, which creates separate index entries for every element of the array.
 - Text index: an index that supports text search queries on string content.
 - Hashed index: an index that computes the hash of the field value and stores the hash as the index entry.

- Indexes can improve the performance of queries, but they also have some drawbacks, such as:
 - Indexes consume disk space and memory.
 - Indexes slow down write operations, such as insert, update, and delete, because they need to be updated whenever the data changes.
 - Indexes need to be carefully designed and maintained to avoid unnecessary or inefficient indexes.
- Therefore, it is important to choose the appropriate index type and fields for the query patterns and data model of the application.

Aggregation in MongoDB

Aggregation in MongoDB is the process of selecting, grouping, and transforming data from a collection to produce computed results. Aggregation can be used for various purposes, such as:

- Calculating statistics and summaries of data
- Performing complex queries and filters
- Joining data from multiple collections
- Shaping and transforming data for different needs

MongoDB provides two methods to perform aggregation:

- Single-purpose aggregation operations: These are collection methods that perform a specific aggregation task, such as counting documents, finding the minimum or maximum value, or calculating the average. Examples of single-purpose aggregation operations are `count()`, `distinct()`, and `group()`.
- Aggregation pipeline: This is a more flexible and powerful way of performing aggregation. It allows you to create a sequence of stages that process documents in a collection. Each stage can perform a different operation on the input documents, such as filtering, grouping, sorting, projecting, or joining. The output of one stage becomes the input of the next stage, until the final result is produced. Examples of aggregation pipeline stages are `$match`, `$group`, `$sort`, and `$lookup`.

To use aggregation in MongoDB, you can use the `aggregate()` method on a collection. This method accepts an array of stages as an argument, and returns a cursor that iterates over the aggregated documents. You can also use the `db.collection.aggregate()` command in the MongoDB shell to run an aggregation pipeline.

Here is an example of using aggregation in MongoDB to find the average rating of movies by genre:

```
// Create a collection of movies with title, genre, and rating fields
db.movies.insertMany([
  {title: "The Godfather", genre: "Crime", rating: 9.2},
  {title: "The Shawshank Redemption", genre: "Drama", rating: 9.3},
```

```

    {title: "The Dark Knight", genre: "Action", rating: 9.0},
    {title: "The Matrix", genre: "Sci-Fi", rating: 8.7},
    {title: "The Lord of the Rings: The Return of the King", genre: "Fantasy", rating: 8.9},
    {title: "The Lion King", genre: "Animation", rating: 8.5},
    {title: "Toy Story", genre: "Animation", rating: 8.3},
    {title: "The Silence of the Lambs", genre: "Thriller", rating: 8.6},
    {title: "The Sixth Sense", genre: "Thriller", rating: 8.1},
    {title: "Inception", genre: "Sci-Fi", rating: 8.8}
  ])

  // Use aggregation to find the average rating of movies by genre
  db.movies.aggregate([
    // Group the documents by genre and calculate the average rating
    {$group: {_id: "$genre", avg_rating: {$avg: "$rating"}}},
    // Sort the results by average rating in descending order
    {$sort: {avg_rating: -1}}
  ])

```

The output of the aggregation pipeline is:

```

{ "_id" : "Drama", "avg_rating" : 9.3 }
{ "_id" : "Crime", "avg_rating" : 9.2 }
{ "_id" : "Action", "avg_rating" : 9 }
{ "_id" : "Fantasy", "avg_rating" : 8.9 }
{ "_id" : "Sci-Fi", "avg_rating" : 8.75 }
{ "_id" : "Thriller", "avg_rating" : 8.35 }
{ "_id" : "Animation", "avg_rating" : 8.4 }

```

This shows that the highest average rating belongs to the drama genre, followed by crime and action. The lowest average rating belongs to the thriller genre.

Capped Collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order .
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection .
- You must create capped collections explicitly using the `db.createCollection()` method, which is a mongosh helper for the create command .
- When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection .
- You can also optionally specify the maximum number of documents that the collection can store.
- Capped collections are typically used to store log information, high volume of data, and cache information .

- Capped collections have the following advantages and limitations :
 - Advantages:
 - * Capped collections guarantee preservation of the insertion order. As a result, queries do not need an index to return documents in insertion order. Without this indexing overhead, capped collections can support higher insertion throughput.
 - * Capped collections automatically remove the oldest documents in the collection without requiring scripts or explicit remove operations.
 - * Capped collections support `createIndex ()` operations to create indexes on any field of the documents in the collection.
 - * Capped collections support tailable cursors, which can remain open after returning the final result of the initial query and continue to return new documents as they are inserted into the collection.
 - Limitations:
 - * You cannot delete documents from a capped collection. To remove all documents from a capped collection, use the `drop ()` method to drop the collection and recreate the collection.
 - * You cannot update documents in a capped collection if the update operation causes the document to grow beyond its original size. However, you can perform update operations that do not increase the size of the document, such as using the `$inc` operator to increment a field value.
 - * You cannot shard a capped collection. However, you can create a capped collection on a shard by connecting directly to the shard mongod instance and using the `db.createCollection ()` method.
 - * You cannot change the size of a capped collection after creation. However, you can use the `collMod` command with the `convertToCapped` option to convert a non-capped collection to a capped collection and specify a new size. This operation will delete any documents that do not fit in the new size.

Spark

Spark is a word that has two distinct meanings. As a noun, it means a small fiery particle thrown off from a fire, alight in ashes, or produced by striking together two hard surfaces such as stone or metal. Or as a verb, it means emit sparks of fire or electricity.

Some of the features of spark as a noun are:

- It can be used to describe a small flash of light produced by a sudden disruptive electrical discharge through the air.
- It can also refer to an electrical discharge serving to ignite the explosive mixture in an internal combustion engine.
- It can indicate a trace of a specified quality or intense feeling, such as

interest, anger, or excitement.

- It can denote a sense of liveliness and excitement, or a lively young fellow.

Some of the features of spark as a verb are:

- It can mean to produce sparks at the point where an electric circuit is interrupted.
- It can also mean to ignite something, such as a fire, a fuse, or a conflict.
- It can imply to provide the stimulus for something, such as an idea, a movement, or a reaction.
- It can signify to engage in courtship or flirtation.

Some examples of spark in a sentence are:

- A log fire was sending sparks onto the rug.
- Angry sparks were flashing in her eyes.
- There was a spark of light.
- A tiny spark of anger flared within her.
- There was a spark between them at their first meeting.
- The ignition sparks as soon as the gas is turned on.
- The explosion sparked a fire.
- The severity of the plan sparked off street protests.
- He went a sparking among the rosy country girls.

: OxfordDictionaries, “Spark”, <http://www.oxforddictionaries.com/us/>

Installing spark

Spark is an open-source distributed computing framework that can process large-scale data sets using in-memory caching and parallel processing. Spark can run on various platforms, such as Hadoop, Mesos, Kubernetes, standalone, or in the cloud. To install Spark, you need to follow these steps:

- Download the latest version of Spark from the official website: <https://spark.apache.org/downloads.html>. Choose a package type that matches your cluster manager and a compatible version of Scala.
- Extract the downloaded file to a location of your choice, such as `/opt/spark`.
- Set the environment variables `SPARK_HOME` and `PATH` to point to the Spark installation directory and its `bin` subdirectory, respectively. For example, if you are using bash, you can add these lines to your `.bashrc` file:

```
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin
```

- Optionally, you can also configure some Spark properties by editing the `conf/spark-defaults.conf` file. For example, you can set the amount of memory to use for each executor, the number of cores to use, the log level, etc. You can find more details about the available properties

- To verify that Spark is installed correctly, you can run the `spark-shell` command, which will launch an interactive Scala shell with Spark. You should see something like this:

```

  /---/
 \V_--V_--\---/
/---/ .---\ , / / / \ \
 /- /

```

version 3.2.0

```
scala>
```

- You can also run the `pyspark` command, which will launch an interactive Python shell with Spark. You should see something like this:



93

SparkSession available as 'spark'.

>>>

- You can now use Spark to perform various data analysis tasks, such as reading and writing data, applying transformations and actions, creating SQL queries, using machine learning libraries, etc. You can find more details about the Spark programming guide here: <https://spark.apache.org/docs/latest/programming-guide.html>.

Spark Applications

Spark applications are programs that run on a cluster of nodes using the Spark framework. They can perform various tasks such as processing streaming data, machine learning, fog computing, and more. Some of the features and benefits of Spark applications are:

- They consist of a driver process and a set of executor processes that communicate and coordinate with each other.
- The driver process runs the main function of the program, maintains information about the application, and distributes and schedules work across the executors.
- The executor processes run the tasks assigned by the driver and store and cache data in memory or disk.
- Spark applications can run on various cluster managers such as YARN, Mesos, Kubernetes, or standalone mode.
- Spark applications can use various APIs and libraries such as Spark SQL, Spark Streaming, MLlib, GraphX, and more to perform different types of analytics and computations.
- Spark applications can handle large-scale, distributed, and complex data processing with high performance, scalability, and fault tolerance.

Jobs in Spark

Spark is a distributed computing framework that allows users to process large-scale data using various programming languages and libraries. Spark can run on different cluster managers, such as Hadoop YARN, Apache Mesos, or Kubernetes, and can also run in standalone mode.

There are different types of jobs in Spark, depending on the context and the purpose of the user. Here are some examples:

- **Spark jobs in Azure Machine Learning (preview):** This is a feature that allows users to submit Spark jobs to Azure Machine Learning, a cloud-based platform for building and deploying machine learning models. Users can define a Spark job as a YAML specification file, which can be used in the `az ml job create` command, with the `--file` parameter. Users can also use the Azure Machine Learning Python SDK or the Azure Machine Learning Studio to create and manage Spark jobs.

- **Spark jobs in Spark applications:** A Spark application is a user program that runs on a Spark cluster and uses the Spark API to perform various operations on data. A Spark job is a parallel computation of tasks, triggered by an action such as `count()`, `foreachRdd()`, `collect()`, `read()` or `write()`. Each Spark job will be converted to a DAG (directed acyclic graph) which includes one or more stages. A stage is a set of tasks that can run in parallel on different executors. A task is a unit of work that runs on a single executor and processes a partition of data.
- **Spark jobs in the job market:** There are also various job opportunities for people who have skills and experience in working with Spark. For example, some of the job titles that are related to Spark are: Spark engineer, Spark developer, Spark architect, Spark data scientist, Spark data engineer, Spark analyst, etc. Some of the companies that are hiring Spark professionals are: Omega Solutions, Spark Education, Spark Program, Spark Power, Spark Transport, etc. Some of the skills and qualifications that are required or preferred for Spark jobs are: proficiency in Scala, Python, Java, or R, familiarity with Spark SQL, Spark Streaming, Spark MLlib, Spark GraphX, or other Spark libraries, experience in developing, testing, debugging, and deploying Spark applications, knowledge of distributed systems, cloud computing, data structures, algorithms, etc.

Stages and Tasks in Spark

- Spark is a distributed computing framework that allows users to process large-scale data using various transformations and actions.
- Spark applications are composed of one or more jobs, which are triggered by actions such as `count()`, `save()`, `show()` etc.
- Each job is divided into one or more stages, which are sets of parallel tasks that operate on a subset of the data.
- Each stage is further divided into one or more tasks, which are the smallest unit of execution in Spark.
- Tasks are executed by executors, which are processes that run on different nodes in the Spark cluster.
- The stages and tasks of a Spark job are determined by the DAG (Directed Acyclic Graph) scheduler, which analyzes the logical plan of the Spark application and optimizes it for performance and efficiency.
- The DAG scheduler also handles the shuffle operations, which are the data transfers between stages that require repartitioning of the data.
- The stages and tasks of a Spark job can be visualized using the Spark UI, which provides useful information such as the number of tasks, the duration of each task, the input and output data size, the shuffle read and write size, the executor memory and CPU usage, and the task errors and failures.

Resilient Distributed Databases in spark

- Resilient Distributed Databases (RDDs) are the primary data structure in Spark .
- RDDs are immutable distributed collections of objects that can be operated on in parallel .
- Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster .
- RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes .
- RDDs support two types of operations: transformations and actions .
 - Transformations create a new RDD from an existing one, such as map, filter, join, etc .
 - Actions return a value to the driver program or write data to an external storage system, such as count, collect, save, etc .
- RDDs are fault-tolerant, meaning they can recover from failures or errors by using lineage information .
 - Lineage is the sequence of transformations that produced an RDD.
 - Spark can recompute the lost partitions of an RDD by using the lineage information.
- RDDs are memory-efficient, meaning they can cache or persist data in memory or disk for faster access .
 - Spark can cache an RDD in memory across the nodes in the cluster.
 - Spark can persist an RDD on disk or a combination of memory and disk.
 - Spark can specify the storage level of an RDD, such as MEMORY_ONLY_ONLY, DISK_ONLY, MEMORY_AND_DISK, etc.
- RDDs are lazy-evaluated, meaning they are only computed when an action is performed on them .
 - Spark can optimize the execution plan of an RDD by applying transformations lazily.
 - Spark can avoid unnecessary computations by applying transformations lazily.
- RDDs are the core of Spark’s programming model and enable parallel and distributed processing of large-scale data .

Anatomy of a Spark job run

- A Spark job is a parallel computation that runs on a cluster of nodes and consists of one or more stages.
- A stage is a set of tasks that perform the same computation on different partitions of the input data.
- A task is a unit of work that runs on a single executor and processes a single partition of the input data.
- An executor is a process that runs on a worker node and executes tasks assigned by the driver.
- A driver is the process that coordinates the execution of a Spark job and runs the user-defined main function.

- A Spark job is created when the driver submits a Spark application to the cluster manager, which allocates resources for the job.
- A Spark job is divided into stages based on the dependencies between the RDDs (Resilient Distributed Datasets) that are involved in the computation.
- A stage can be either a shuffle stage or a result stage, depending on whether it involves shuffling data across the network or not.
- A shuffle stage is a stage that requires data to be transferred from one or more parent stages to child stages, such as when performing a join or a group-by operation.
- A result stage is a stage that computes the final result of the job, such as when performing an action like collect or save.
- A stage is further divided into tasks based on the number of partitions of the RDD that is being computed.
- A task is executed by an executor on a worker node and reads data from one or more partitions of the input RDD, applies the user-defined function, and writes the output to a partition of the output RDD or to an external storage system.
- A task can also read and write data from a shuffle file, which is a temporary file that stores the intermediate data produced by a shuffle stage.
- A task can also communicate with the driver through an accumulator, which is a shared variable that can be updated by tasks and read by the driver.
- A task can also report metrics to the driver, such as the number of records processed, the amount of memory used, and the duration of the task.
- A task can also fail due to various reasons, such as an exception in the user code, a network error, a disk error, or a timeout.
- A task failure can trigger a task retry, a stage retry, or a job failure, depending on the type and frequency of the failure and the `spark.task.maxFailures` configuration parameter.
- A task retry is a re-execution of the same task on the same or a different executor, which can be done up to `spark.task.maxFailures` times before failing the stage.
- A stage retry is a re-computation of the entire stage and its parent stages, which can be done up to `spark.stage.maxFailures` times before failing the job.
- A job failure is a termination of the Spark job, which can be handled by the user code or by the cluster manager, depending on the `spark.submit.deployMode` configuration parameter.
- A job failure can also trigger a job resubmission, which is a re-submission of the same Spark application to the cluster manager, which can be done up to `spark.yarn.maxAppAttempts` times before giving up.

Spark on YARN

- Spark on YARN is a mode of running Spark applications on a cluster of

machines managed by YARN (Yet Another Resource Negotiator), which is a resource manager for Hadoop.

- Spark on YARN requires a binary distribution of Spark that is built with YARN support. Binary distributions can be downloaded from the [downloads page](#) of the project website. To build Spark yourself, refer to [Building Spark](#).
- To make Spark runtime jars accessible from YARN side, you can specify `spark.yarn.archive` or `spark.yarn.jars` in the `spark-defaults.conf` file or as command-line options. These options point to a compressed archive or a list of jars that contain the Spark classes and dependencies.
- Spark on YARN supports two deployment modes: `cluster` and `client`. In `cluster` mode, the Spark driver runs on a YARN container on one of the cluster nodes. In `client` mode, the Spark driver runs on the machine that launches the Spark application, and the YARN resource manager allocates containers for the Spark executors.
- Spark on YARN can use dynamic allocation to scale the number of executors up and down based on the workload. This feature requires an external shuffle service to be enabled on each node in the cluster, which preserves the shuffle files of the executors that are removed. The external shuffle service can be configured by setting `spark.shuffle.service.enabled` to `true` and `yarn.nodemanager.aux-services` to `spark_shuffle`.
- Spark on YARN can also use the YARN timeline server to store the application history. This feature can be enabled by setting `spark.yarn.historyServer.address` to the URL of the timeline server and `spark.eventLog.enabled` to `true`. The timeline server can be accessed through the YARN web UI or the Spark history server UI.

SCALA

- Scala is a general-purpose, high-level, multi-paradigm programming language that supports both object-oriented and functional programming .
- Scala is designed to be concise, expressive, and interoperable with Java . It can run on the JVM (Java Virtual Machine) and JavaScript runtimes.
- Scala has a strong static type system that helps avoid bugs and enforce contracts between components. It also supports advanced features such as higher-order types, implicit parameters, and type classes.
- Scala has a rich collection of immutable and mutable data structures, as well as powerful methods for manipulating them. It also supports pattern matching, algebraic data types, and case classes for concise and elegant code.
- Scala has a flexible syntax that allows multiple ways of expressing the same logic, such as infix notation, operator overloading, and string interpolation. It also supports syntactic sugar for common idioms, such as lambda expressions, for-comprehensions, and traits.
- Scala has a modular architecture that allows code reuse and extensibility through mixins, traits, and implicits. It also supports concurrency and

parallelism through futures, promises, actors, and streams.

- Scala has a vibrant and growing community of developers, researchers, and enthusiasts who contribute to its development and evolution. It also has a large and diverse ecosystem of libraries and frameworks for various domains and applications.

Introduction to Scala

- Scala is a general-purpose, multi-paradigm programming language that integrates both object-oriented and functional features.
- Scala runs on the Java Virtual Machine (JVM) and is compatible with existing Java code and libraries.
- Scala was designed to address some of the limitations and complexities of Java, such as verbosity, null pointers, and concurrency issues.
- Scala supports multiple programming styles, such as imperative, declarative, concurrent, and reactive.
- Scala has a concise and expressive syntax that allows programmers to write less code and avoid boilerplate.
- Scala has a powerful type system that supports static typing, type inference, generics, variance, and higher-kinded types.
- Scala has a rich set of built-in data structures and collections, such as lists, arrays, sets, maps, tuples, and options.
- Scala has a number of advanced features, such as pattern matching, case classes, traits, implicits, and macros.
- Scala has a flexible and extensible syntax that allows programmers to define their own operators, keywords, and DSLs.
- Scala has a vibrant and growing community of developers, researchers, and enthusiasts who contribute to its development and ecosystem.

Classes and Objects in Scala

- Classes in Scala are blueprints for creating objects. They can contain methods, values, variables, types, objects, traits, and classes which are collectively called members .
- Objects in Scala are instances of classes. They can be created using the `new` keyword or using companion objects.
- A minimal class definition is simply the keyword `class` and an identifier (name) of the class. For example:

```
class Person
```

- A class can have a primary constructor that takes parameters and initializes the fields of the object. The parameters are defined in the class header after the name of the class . For example:

```
class Person(firstName: String, lastName: String)
```

- A class can also have methods that define the behavior of the object. Methods are defined using the `def` keyword followed by the name, parameters, return type, and body of the method . For example:

```
class Person(firstName: String, lastName: String) {
  def fullName: String = firstName + " " + lastName
}
```

- A class can also have values and variables that store the state of the object. Values are defined using the `val` keyword and are immutable, meaning they cannot be reassigned. Variables are defined using the `var` keyword and are mutable, meaning they can be reassigned . For example:

```
class Person(firstName: String, lastName: String) {
  val age: Int = 0 // a value that cannot be changed
  var email: String = "" // a variable that can be changed
}
```

- A class can also have types, objects, traits, and classes as its members, which will be covered later in the tour.
- A companion object is an object that has the same name as a class and is defined in the same file. A companion object can access the private members of the class and can provide a factory method for creating objects without using the `new` keyword. For example:

```
class Person(firstName: String, lastName: String) {
  // class definition
}

object Person {
  // companion object definition
  def apply(firstName: String, lastName: String): Person = {
    new Person(firstName, lastName)
  }
}
```

- To create an object of a class, you can use the `new` keyword followed by the name of the class and the arguments for the constructor. Alternatively, you can use the companion object's `apply` method if it is defined . For example:

```
val p1 = new Person("John", "Doe") // using the new keyword
val p2 = Person("Jane", "Doe") // using the companion object
```

- To access the members of an object, you can use the dot (`.`) notation followed by the name of the member. For example:

```
val name = p1.fullName // accessing the fullName method
p2.email = "jane.doe@example.com" // assigning a new value to the email variable
```

Basic types and operators in Scala

- Scala has a rich set of built-in types, such as `Int`, `Double`, `Boolean`, `String`, etc.
- Scala also supports user-defined types, such as classes, traits, and objects.
- Scala has a unified type system, which means that every value is an object and every operation is a method call.
- Scala has two kinds of numeric types: integral and fractional. Integral types include `Byte`, `Short`, `Int`, `Long`, and `Char`. Fractional types include `Float` and `Double`.
- Scala also has a special type called `Unit`, which corresponds to `void` in Java. It is used to indicate that a function or expression does not return any meaningful value.
- Scala has a type called `Any`, which is the supertype of all types. It has two direct subtypes: `AnyVal` and `AnyRef`. `AnyVal` is the supertype of all value types, such as `Int` and `Boolean`. `AnyRef` is the supertype of all reference types, such as `String` and `Array`.
- Scala has a type called `Nothing`, which is the subtype of all types. It is used to indicate that a function or expression never returns normally, such as by throwing an exception or entering an infinite loop.
- Scala has a type called `Null`, which is the subtype of all reference types. It is used to represent the absence of a value. It is compatible with `null` in Java, but it is rarely used in Scala.
- Scala has a rich set of operators, such as arithmetic, relational, logical, bitwise, and assignment operators. Operators are actually methods that can be defined or overloaded for user-defined types.
- Scala has operator precedence rules that determine the order of evaluation of expressions. The precedence is based on the first character of the operator, with some exceptions. For example, `*` has higher precedence than `+`, and `&&` has higher precedence than `||`.
- Scala has operator associativity rules that determine how operators of the same precedence are grouped. The associativity is based on the last character of the operator, with some exceptions. For example, `+` and `-` are left-associative, meaning that `a + b - c` is equivalent to `(a + b) - c`. However, `=` and `:` are right-associative, meaning that `a = b = c` is equivalent to `a = (b = c)`.
- Scala has a special operator called `apply`, which is used to invoke a function or access an element of a collection. For example, `f(x)` is equivalent to `f.apply(x)`, and `a(i)` is equivalent to `a.apply(i)`. This operator can be defined or overloaded for user-defined types.

Built-in control structures in Scala

Scala has several built-in control structures that can be used to write concise and expressive code. Some of the most common ones are:

- **if-else:** This is a conditional expression that evaluates a boolean con-

dition and returns one of two possible values depending on whether the condition is true or false. For example:

```
val x = 10
val y = if (x > 0) "positive" else "negative"
// y is "positive"
```

- **while**: This is a loop that executes a block of code repeatedly as long as a boolean condition is true. For example:

```
var i = 0
while (i < 10) {
  println(i)
  i += 1
}
// prints 0 to 9
```

- **for**: This is a loop that iterates over a collection or a range of values and executes a block of code for each element. For example:

```
val names = List("Alice", "Bob", "Charlie")
for (name <- names) {
  println(name)
}
// prints Alice, Bob, Charlie
```

- **match**: This is a pattern matching expression that compares a value with a series of cases and returns the result of the first case that matches. For example:

```
val color = "red"
val message = color match {
  case "red" => "stop"
  case "green" => "go"
  case "yellow" => "slow down"
  case _ => "unknown color"
}
// message is "stop"
```

- **try-catch-finally**: This is a control structure that handles exceptions that may occur during the execution of a block of code. The **try** block contains the code that may throw an exception, the **catch** block contains the code that handles the exception, and the **finally** block contains the code that is always executed regardless of whether an exception occurs or not. For example:

```
try {
  val n = 10 / 0 // throws an ArithmeticException
  println(n)
} catch {
```

```

    case e: ArithmeticException => println("cannot divide by zero")
  } finally {
    println("this is always executed")
  }
  // prints "cannot divide by zero" and "this is always executed"

```

Functions and Closures in Scala

- A function in Scala is a piece of code that takes some input, performs some computation, and returns some output.
- A function can be defined using the **def** keyword, followed by the name, parameters, return type, and body of the function.
- A function can also be defined as an anonymous function, or a function literal, using the **=>** syntax. An anonymous function can be assigned to a variable or passed as an argument to another function.
- A closure in Scala is a special type of function that uses one or more free variables, which are not defined as parameters or local variables of the function, but are defined in some outer scope.
- A closure captures the value or reference of the free variables at the time of its creation, and can access or modify them even if they are not in the current scope.
- A closure is useful for creating higher-order functions, which are functions that take other functions as arguments or return other functions as results.
- A closure can be pure or impure, depending on whether it modifies the free variables or not. A pure closure does not change the value of the free variables, while an impure closure does.
- An example of a pure closure is:

```

val x = 10 // a free variable
val addX = (y: Int) => x + y // a closure that uses x
println(addX(5)) // prints 15

```

- An example of an impure closure is:

```

var x = 10 // a free variable
val addX = (y: Int) => {
  x = x + 1 // modifies x
  x + y // returns x + y
}
println(addX(5)) // prints 16
println(x) // prints 11

```

- Closures are part of Scala and any functional programming language because they allow creating functions that can access and manipulate data from different scopes, without passing them explicitly as parameters. This makes the code more concise, expressive, and reusable.

Inheritance in Scala

- Inheritance is an important pillar of OOP (Object Oriented Programming).
- It is the mechanism in Scala by which one class is allowed to inherit the features (fields and methods) of another class.
- The class whose features are inherited is known as superclass (or a base class or a parent class).
- The class that inherits the features is known as subclass (or a derived class or a child class).
- We use the **extends** keyword to indicate that a class inherits from another class .
- Scala supports various types of inheritance, such as single, multilevel, multiple, and hierarchical .
- Single inheritance is when one class inherits from a single other class.
- Multilevel inheritance is when one class extends another, which in turn extends another.
- Hierarchical inheritance is when more than one class inherits from a single class.
- Multiple inheritance is when one class inherits from more than one class .
- Hybrid inheritance is a combination of multiple and hierarchical inheritance .
- Scala does not support multiple and hybrid inheritance directly in classes, but it can be achieved by using traits .
- Traits are abstract types that can contain fields and methods, and can be mixed in with classes using the **with** keyword .
- Traits can also extend other traits or classes, and can be used to implement multiple interfaces .
- Inheritance allows us to reuse code, avoid duplication, and achieve polymorphism .
- Polymorphism is the ability of an object to behave differently depending on its type or context .
- Scala supports two types of polymorphism: static (or compile-time) and dynamic (or run-time) .
- Static polymorphism is when the method to be invoked is determined at compile time, based on the type of the reference variable .
- Dynamic polymorphism is when the method to be invoked is determined at run time, based on the type of the object referred by the variable .
- Static polymorphism can be achieved by using method overloading, which is when a class has more than one method with the same name but different parameters .
- Dynamic polymorphism can be achieved by using method overriding, which is when a subclass redefines a method inherited from a superclass .
- Scala also supports abstract classes and methods, which are classes and methods that are declared but not defined .
- Abstract classes and methods are used to specify the general behavior of a class, but leave the implementation details to the subclasses .
- Abstract classes and methods are marked with the **abstract** keyword .

- Abstract classes cannot be instantiated, but can be extended by concrete classes .
- Abstract methods must be overridden by concrete methods in the subclasses .
- Scala also supports final classes and methods, which are classes and methods that cannot be extended or overridden .
- Final classes and methods are marked with the `final` keyword .
- Final classes and methods are used to prevent inheritance and polymorphism, and to improve performance .
- Inheritance in Scala is a powerful feature that allows us to design and organize our code in a modular and reusable way .

Hadoop Eco System Frameworks , Pig , Hive and HBase

- Hadoop is a software library that provides a framework for distributed storage and processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of four core components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common.
- HDFS is a distributed file system that stores data across multiple nodes in a cluster, providing high availability, fault tolerance, and scalability.
- MapReduce is a programming model and an execution engine for parallel processing of large data sets on HDFS.
- YARN is a resource management layer that allocates and schedules computing resources (such as CPU and memory) for applications running on Hadoop.
- Hadoop Common is a set of utilities and libraries that support the other Hadoop components.
- Hadoop also includes several additional modules that provide additional functionality, such as:
 - Hive: a data warehouse infrastructure that provides data summarization and ad-hoc querying using a SQL-like query language called HiveQL.
 - Pig: a high-level platform for creating MapReduce programs using a data-flow language called Pig Latin.
 - HBase: a non-relational, distributed database that supports structured data storage for large tables, and provides random, real-time read/write access to the data.
 - Oozie: a workflow scheduler that manages and coordinates the execution of Hadoop jobs.
 - Zookeeper: a coordination service that maintains configuration information, naming, and synchronization for distributed systems.
 - Spark: a fast and general engine for large-scale data processing, supporting batch, streaming, and interactive analytics.
 - Mahout: a machine learning library that provides scalable algorithms for clustering, classification, recommendation, and dimensionality re-

- duction.
- Sqoop: a tool that transfers data between Hadoop and relational databases.
- The Hadoop ecosystem is a framework and suite of tools that tackle the many challenges in dealing with big data, such as scalability, reliability, performance, and analysis.

Hadoop Eco System Frameworks

- Hadoop is a framework that enables processing of large data sets which reside in the form of clusters.
- Hadoop is made up of several modules that are supported by a large ecosystem of technologies.
- The core component of the Hadoop ecosystem is a Hadoop distributed file system (HDFS), which is a distributed file system that has the capability to store a large stack of data sets.
- HDFS is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
- HDFS provides high availability, fault tolerance, and reliability by replicating data blocks across multiple nodes.
- Another core component of the Hadoop ecosystem is MapReduce, which is a programming model that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- MapReduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same intermediate key and produces the final output.
- MapReduce can handle structured, semi-structured, and unstructured data, and supports various data formats, such as text, binary, XML, JSON, etc.
- A third core component of the Hadoop ecosystem is YARN, which stands for Yet Another Resource Negotiator. YARN is a resource management layer that allocates and schedules resources (such as CPU, memory, disk, network, etc) to different applications running on top of Hadoop.
- YARN enables multiple applications to share the same cluster resources, and supports various types of applications, such as batch, interactive, streaming, graph, machine learning, etc.
- YARN consists of two main components: a Resource Manager, which is responsible for managing the cluster resources and assigning them to differ-

ent applications, and a Node Manager, which is responsible for managing the resources and tasks on each node.

- A fourth core component of the Hadoop ecosystem is Hadoop Common, which is a set of common utilities and libraries that support the other Hadoop modules.
- Hadoop Common provides the basic functionality, such as configuration, logging, security, serialization, etc, that are required by the other Hadoop modules.
- In addition to the core components, the Hadoop ecosystem also includes various other tools, libraries, and frameworks that help you build applications on top of Hadoop.
- Some of the popular tools, libraries, and frameworks in the Hadoop ecosystem are:
 - Apache Hive, which is a data warehouse system that provides a SQL-like interface to query and analyze data stored in HDFS.
 - Apache Pig, which is a scripting language that allows for the analysis and transformation of large data sets using a high-level abstraction.
 - Apache Spark, which is a fast and general-purpose engine for large-scale data processing, supporting batch, streaming, SQL, machine learning, and graph processing.
 - Apache HBase, which is a distributed and scalable NoSQL database that provides random access and strong consistency for large amounts of sparse and structured data.
 - Apache Kafka, which is a distributed and fault-tolerant messaging system that enables high-throughput and low-latency data ingestion and processing.
 - Apache Flume, which is a service that collects, aggregates, and moves large amounts of log data from various sources to HDFS.
 - Apache Sqoop, which is a tool that transfers data between Hadoop and relational databases.
 - Apache Oozie, which is a workflow scheduler that coordinates and executes Hadoop jobs.
 - Apache Mahout, which is a library that provides scalable and distributed machine learning algorithms for Hadoop.
 - Apache ZooKeeper, which is a service that provides coordination, configuration, and synchronization for distributed systems.
 - Apache Cassandra, which is a distributed and scalable NoSQL database that provides high availability and performance for large amounts of unstructured data.
 - Apache Storm, which is a distributed and real-time computation system that processes streams of data in parallel.
 - Apache Flink, which is a stream and batch processing framework that provides high performance and low latency for large-scale data

analysis.

- Apache Solr, which is a distributed and scalable search platform that provides full

Applications on Big Data using Pig

- Apache Pig is a high-level platform or tool that is used to process large datasets on Hadoop. It provides a high level of abstraction for processing over MapReduce. It provides a high-level scripting language, known as Pig Latin, which is used to develop the data analysis codes .
- Some of the applications of Pig in big data are :
 - It is used by telecom companies to de-identify the customer call data information.
 - It processes time-sensitive data loads, such as web logs and click-stream data.
 - It explores large datasets for ad-hoc queries and prototyping of data processing algorithms.
 - It collects large amounts of datasets in the form of web crawls and search logs for web analytics and search engines.
 - It supports user-defined functions (UDFs) in other programming languages like Java and Python and embeds or invokes them in Pig scripts.
 - It handles a wide range of data, both structured and unstructured, such as text, images, videos, etc.
 - It performs complex data transformations and manipulations, such as filtering, grouping, joining, sorting, etc. with ease and efficiency.

Applications on Big Data using Hive

Hive is a software application that allows users to perform data analysis on large-scale datasets using a SQL-like language called HiveQL. Hive is built on top of Apache Hadoop, which is a framework for distributed storage and processing of big data. Hive can handle petabytes of data and supports various data formats, such as text, JSON, XML, ORC, Parquet, and Avro. Some of the applications of Hive in big data are:

- Data encapsulation: Hive provides a schema on read approach, which means that the data is stored in its original format and the schema is applied only when the data is queried. This allows users to access and analyze data from different sources without modifying the underlying data structure. Hive also supports partitioning and bucketing, which are techniques to organize data into logical subsets for faster query processing.
- Ad-hoc queries: Hive allows users to perform interactive queries on big data using HiveQL, which is similar to SQL. HiveQL supports various features, such as joins, aggregations, subqueries, window functions, and user-defined functions. Hive also provides a command-line interface and a web-based user interface for executing queries and viewing results.

- Analysis of huge datasets: Hive can perform complex data analysis on big data using various tools and frameworks, such as MapReduce, Spark, Tez, and Presto. Hive can also integrate with other big data technologies, such as Kafka, Flume, Sqoop, and Pig, to ingest, transform, and export data. Hive can also connect with various data visualization and reporting tools, such as Tableau, Power BI, and Excel, to present the insights derived from the data.

Applications on Big Data using HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop and HDFS or Amazon S3 using Amazon EMR.
- HBase is used to have random, real-time read/write access to Big Data. It can handle large, sparse datasets that are common in many Big Data use cases.
- HBase works well with Hive, a query engine for batch processing of Big Data, to enable fault-tolerant Big Data applications.
- HBase can be used to manage structured and semi-structured data and it has many built-in features such as scalability, compression, replication, and consistency.
- Some of the applications of HBase are:
 - Medical: HBase is used for storing genome sequences and running MapReduce on them, storing the disease history of people or an area, and many others.
 - Sports: HBase is used for storing and analyzing sports statistics, such as player performance, team rankings, and game outcomes.
 - Social Media: HBase is used for storing and processing large volumes of user-generated content, such as posts, comments, likes, and shares.
 - E-commerce: HBase is used for storing and querying product catalogs, customer reviews, recommendations, and transactions.
 - Search: HBase is used for storing and retrieving web pages, documents, and metadata for search engines.

Pig

- A pig is a mammal that belongs to the family Suidae, which includes eight genera and 16 species.
- Pigs are also known as hogs or swine. Male pigs of any age are called boars; female pigs are called sows.
- Pigs are found and raised all over the world, and provide valuable products to humans, including pork, lard, leather, glue, fertilizer, and a variety of medicines.
- Pigs have the intelligence of a human toddler and are ranked as the fifth most intelligent animal in the world. They can learn their names, come when they are called, and even play video games better than some primates.

- Pigs have a keen sense of smell and can detect odors up to seven miles away and 20 feet underground. They use their snouts for digging, rooting, and communicating.
- Pigs do not have many sweat glands, so they cool themselves by wallowing in mud or water. This also helps them to protect their skin from parasites and sunburn.
- Pigs are social and playful animals that form strong bonds with each other and other species. They enjoy cuddling, napping, and listening to music.
- Pigs are omnivorous and will eat almost anything, including plants, insects, worms, eggs, carrion, and even feces. They have a simple stomach and a long digestive tract that allows them to digest fibrous foods.
- Pigs have four toes on each foot, but only use the middle two for walking. These are called trotters. Pigs can run up to 11 miles per hour.
- Pigs are one of the most populous mammals on earth, with about one billion pigs alive at any given time. They are also one of the oldest domesticated animals, dating back to 7000 BC in China.

: **Pig Facts - Animal Facts Encyclopedia** Pig - Animals
 10 Facts About Pigs | World Animal Protection
 Pig Facts - Animal Facts Encyclopedia
 10 Facts About Pigs - WorldAtlas

Pig - Introduction to PIG

- Pigs are mammals of the family Suidae, which includes eight genera and 16 species of wild and domestic pigs .
- Pigs are also known as hogs or swine. Male pigs of any age are called boars; female pigs are called sows.
- Pigs are native to the Old World, but have been introduced to many parts of the world by humans .
- Pigs are omnivorous animals, meaning they eat both plants and animals . They have a well-developed sense of smell and can locate food underground.
- Pigs are social animals that live in groups called sounders. They communicate with each other using a variety of vocalizations and body language.
- Pigs are intelligent and can learn new skills quickly . They can also recognize themselves in mirrors and show empathy for other pigs.
- Pigs are raised by humans for various purposes, such as food, leather, glue, fertilizer, and medicine. They are also kept as pets by some people .
- Pigs have a significant impact on the environment, both positive and negative. They can help control pests and weeds, but they can also damage crops and habitats, spread diseases, and compete with native wildlife.

Execution Modes of Pig

Apache Pig is a high-level platform for analyzing large data sets using a scripting language called Pig Latin. Pig can run on a single machine or on a distributed

environment like Hadoop. Depending on where the Pig script is executed and where the data is stored, Pig has different execution modes. These are:

- **Local mode:** In this mode, Pig runs in a single Java Virtual Machine (JVM) and accesses the local file system. This mode is useful for development, testing and prototyping. To run Pig in local mode, use the `-x local` option in the command line or the `pig -x local` command in the Grunt shell .
- **MapReduce mode:** In this mode, Pig runs on a Hadoop cluster and accesses the Hadoop Distributed File System (HDFS). This mode is used for processing large data sets in parallel. To run Pig in MapReduce mode, use the `-x mapreduce` option in the command line or the `pig -x mapreduce` command in the Grunt shell .
- **Tez mode:** In this mode, Pig runs on a Hadoop cluster and uses Apache Tez as the execution engine. Tez is a framework for building high-performance data processing applications on Hadoop. Tez optimizes the execution plan of Pig scripts and reduces the number of MapReduce jobs. To run Pig in Tez mode, use the `-x tez` option in the command line or the `pig -x tez` command in the Grunt shell.
- **Tez local mode:** In this mode, Pig runs in a single JVM and uses Apache Tez as the execution engine. This mode is similar to local mode, but with the benefits of Tez optimization. To run Pig in Tez local mode, use the `-x tez_local` option in the command line or the `pig -x tez_local` command in the Grunt shell.
- **Spark mode:** In this mode, Pig runs on a Hadoop cluster and uses Apache Spark as the execution engine. Spark is a fast and general engine for large-scale data processing. Spark can run Pig scripts in memory or on disk, and supports advanced analytics such as machine learning and graph processing. To run Pig in Spark mode, use the `-x spark` option in the command line or the `pig -x spark` command in the Grunt shell.
- **Embedded mode:** In this mode, Pig can be embedded in a Java application and run as a library. This mode allows the user to define custom functions, load and store functions, and user-defined data types in Java. To run Pig in embedded mode, use the `PigServer` class in the Java code .

Comparison of Pig with Databases

- Pig is a high-level scripting platform that runs on top of Hadoop and allows users to process and analyze large datasets using a language called Pig Latin .
- Databases are structured collections of data that can be queried and manipulated using a standard query language such as SQL.
- Some of the main differences between Pig and databases are:
 - Pig can handle both structured and semi-structured data, while

databases are designed for structured data only .

- Pig is a dataflow language that specifies the steps of data transformation and analysis, while databases are declarative languages that specify the desired result of a query .
 - Pig is optimized for parallel processing and scalability on Hadoop clusters, while databases are optimized for performance and consistency on single or distributed servers .
 - Pig can integrate with other Hadoop components such as Sqoop, Flume, and Hive, while databases are standalone systems that may require additional tools or connectors to interact with Hadoop .
 - Pig is more flexible and expressive than databases, but it also requires more coding and debugging skills, while databases are more user-friendly and familiar to data analysts, but they also have more limitations and restrictions .
- Therefore, Pig and databases have different strengths and weaknesses, and the choice of which one to use depends on the type, size, and complexity of the data, as well as the user's preference and experience.

Grunt in Pig

- Grunt is a term that can refer to two different things in the context of Pig: a type of sound that pigs make, and a shell that is used to write Pig Latin scripts .
- Grunt as a sound:
 - Pigs use grunts to communicate with each other and express their emotions .
 - Grunts can vary in frequency, duration, intensity, and pitch depending on the pig's personality, environment, and situation .
 - Grunts can indicate positive or negative emotions, such as happiness, curiosity, fear, or aggression .
 - Grunts can also convey information about the pig's social status, reproductive state, and health condition .
- Grunt as a shell:
 - Grunt is the default shell of Apache Pig, which is a platform for analyzing large data sets using a high-level language called Pig Latin.
 - Grunt can be used to write Pig Latin scripts, load and store data, and execute Pig Latin commands.
 - Grunt can also invoke shell commands using `sh` and `fs`, which can be useful for performing operations on the local file system or the Hadoop file system.
 - Grunt can be launched in interactive mode, batch mode, or embedded mode, depending on the user's preference and needs.

Pig Latin

- Pig Latin is a language game or argot in which words in English are altered,

usually by adding a suffix or by moving the onset or initial consonant or consonant cluster of a word to the end of the word and adding a vocalic syllable to create such a suffix.

- The objective is to conceal the meaning of the words from others not familiar with the rules.
- The reference to Latin is a deliberate misnomer, as it is simply a form of jargon, used only for its English connotations as a strange and foreign-sounding language.
- Some examples of Pig Latin are:
 - “Pig” becomes “igpay”
 - “Latin” becomes “atinlay”
 - “Banana” becomes “ananabay”
 - “Happy” becomes “appyhay”
 - “Duck” becomes “uckday”
 - “Hello” becomes “ellohay”
 - “Smile” becomes “ilesmay”
 - “Eat” becomes “eatway” or “yay”
 - “Question” becomes “estionquay”
 - “Star” becomes “arstay”
 - “I” becomes “Iyay”
 - “You” becomes “ouyay”
 - “Me” becomes “emay”
 - “He” becomes “ehay”
 - “She” becomes “eshay”
 - “We” becomes “eway”
 - “They” becomes “eythay”
 - “It” becomes “itway” or “ityay”
 - “Is” becomes “isway” or “isyay”
 - “Are” becomes “areway” or “aryay”
 - “Am” becomes “amway” or “amyay”
 - “Be” becomes “ebay”
 - “Do” becomes “odway” or “odyay”
 - “Have” becomes “avehay”
 - “Can” becomes “ancay”
 - “Will” becomes “illway”
 - “Would” becomes “ouldway”
 - “Should” becomes “ouldshay”
 - “Could” becomes “ouldcay”
 - “And” becomes “andway” or “andyay”
 - “But” becomes “utbay”
 - “Or” becomes “orway” or “oryay”
 - “For” becomes “orfay”
 - “Of” becomes “ofway” or “ofyay”
 - “The” becomes “ethay”

- “A” becomes “away” or “ayay”
- “An” becomes “anway” or “anyay”
- “To” becomes “otway” or “otyay”
- “In” becomes “inway” or “inyay”
- “On” becomes “onway” or “onyay”
- “At” becomes “atway” or “atyay”
- “By” becomes “ybay”
- “From” becomes “omfray”
- “With” becomes “ithway”
- “As” becomes “asway” or “asyay”
- “If” becomes “ifway” or “ifyay”
- “That” becomes “atthay”
- “This” becomes “isthay”
- “These” becomes “esethay”
- “Those” becomes “osethay”
- “What” becomes “atwhay”
- “When” becomes “enwhay”
- “Where” becomes “erewhay”
- “Why” becomes “ywhay”
- “How” becomes “owhay”
- “Who” becomes “oowhay”
- “Whom” becomes “oomwhay”
- “Whose” becomes “ooswhay”
- “Which” becomes “ichwhay”
- “Here” becomes “erehay”
- “There” becomes “erethay”
- “Now” becomes “ownay”
- “Then” becomes “enthay”
- “Than” becomes “anthay”
- “More” becomes “oremay”
- “Less” becomes “esslay”
- “Much” becomes “uchmay”
- “Many” becomes “anymay”
- “Some” becomes “omesay”
- “Any” becomes “anyay”
- “No” becomes “onay”
- “Yes” becomes “esyay”
- “Please” becomes “easeplay”
- “Thank” becomes “

User Defined Functions in Pig

- User Defined Functions (UDFs) are custom functions that can be written in Java, Python, or other languages to extend the functionality of Pig.
- UDFs can be used to perform complex data transformations, custom aggregations, filtering, or other operations that are not supported by the

built-in Pig functions.

- UDFs can be registered in a Pig script using the REGISTER statement, which specifies the path to the JAR file or the Python script that contains the UDFs.
- UDFs can be invoked in a Pig script using the DEFINE statement, which assigns an alias to the UDF and specifies its input and output types.
- UDFs can be used in any Pig expression, such as in a FOREACH, FILTER, GROUP, or JOIN statement.
- UDFs can also be used to create custom load and store functions, which allow Pig to read and write data from various sources and formats.
- UDFs can be tested and debugged using the PigUnit framework, which provides a way to write unit tests for Pig scripts and UDFs.

Data Processing Operators in Pig

Data processing operators are the main tools that Pig Latin provides to operate on the data. They allow you to transform the data by sorting, grouping, joining, projecting, and filtering. A Pig Latin statement is an operator that takes a relation as input and produces another relation as output .

Some of the common data processing operators in Pig are:

- **LOAD:** This operator is used to load the data from the file system or other sources into a relation. The syntax is:

```
relation_name = LOAD 'file_path' [USING function] [AS schema];
```

- **STORE:** This operator is used to store the data from a relation into the file system or other destinations. The syntax is:

```
STORE relation_name INTO 'file_path' [USING function];
```

- **FILTER:** This operator is used to filter the tuples from a relation based on a condition. The syntax is:

```
relation_name = FILTER relation_name BY condition;
```

- **FOREACH:** This operator is used to generate data transformations based on the columns of data. The syntax is:

```
relation_name = FOREACH relation_name GENERATE expression [,  
expression ...];
```

- **MAPREDUCE:** This operator is used to run a MapReduce job from Pig. The syntax is:

```
relation_name = MAPREDUCE 'jar_path' [STORE relation_name  
INTO 'input_path' [USING function]] [LOAD 'output_path'  
[USING function] AS schema];
```

- **GROUP:** This operator is used to group the data in one or more relations based on a key or a set of keys. The syntax is:

```
relation_name = GROUP relation_name [ALL] BY expression [,
expression ...] [USING 'collected' | 'merge'];
```

- **COGROUP:** This operator is similar to the GROUP operator, but it can group the data from two or more relations at the same time. The syntax is:

```
relation_name = COGROUP relation_name [ALL] BY expression [,
expression ...] [USING 'collected' | 'merge'], relation_name
[ALL] BY expression [, expression ...] [USING 'collected' |
'merge'], ...;
```

- **JOIN:** This operator is used to join two or more relations based on a common field or a set of fields. The syntax is:

```
relation_name = JOIN relation_name BY expression [, expression
...] [USING 'replicated' | 'skewed' | 'merge' | 'repljoin'],
relation_name BY expression [, expression ...] [USING 'replicated'
| 'skewed' | 'merge' | 'repljoin'], ...;
```

- **CROSS:** This operator is used to produce the cross product of two or more relations. The syntax is:

```
relation_name = CROSS relation_name, relation_name [, relation_name
...];
```

- **DISTINCT:** This operator is used to remove the duplicate tuples from a relation. The syntax is:

```
relation_name = DISTINCT relation_name;
```

- **LIMIT:** This operator is used to limit the number of tuples in a relation. The syntax is:

```
relation_name = LIMIT relation_name number;
```

- **ORDER:** This operator is used to sort the tuples in a relation based on one or more fields. The syntax is:

```
relation_name = ORDER relation_name BY expression [ASC |
DESC] [, expression [ASC | DESC] ...];
```

- **UNION:** This operator is used to combine the tuples from two or more relations into a single relation. The syntax is:

```
relation_name = UNION relation_name, relation_name [, relation_name
...];
```

- **SPLIT:** This operator is used to split a relation into two or more relations based on some conditions. The syntax is:

```
SPLIT relation_name INTO relation_name IF condition, relation_name
IF condition [, relation_name IF condition ...];
```

- **DUMP**: This operator is used to display the contents of a relation on the screen. The syntax is:

```
DUMP relation_name;
```

- **DESCRIBE**: This operator is used to display the schema of a relation on the screen. The syntax is:

```
DESCRIBE relation_name;
```

- **EXPLAIN**: This operator is used to display the execution plan of a relation on the screen. The syntax is:

```
EXPLAIN relation_name;
```

- **ILLUSTRATE**: This operator is used to display an example of how a relation is computed on the screen. The syntax is:

```
ILLUSTR
```

Hive

- Hive is a data warehouse software that facilitates querying and managing large datasets residing in distributed storage.
- Hive provides a SQL-like interface called HiveQL to access structured and semi-structured data in various formats.
- Hive allows users to project structure on largely unstructured data and perform analytical queries on it.
- Hive is built on top of Apache Hadoop and supports storage on various file systems such as HDFS, S3, ADLS, GS, etc.
- Hive can also integrate with other data processing tools such as Spark, Pig, and MapReduce.
- Hive has several components, such as:
 - Hive Metastore: a central repository of metadata that stores the schema and location of the tables and partitions.
 - Hive Driver: a component that receives the HiveQL queries from the users and compiles, optimizes, and executes them.
 - Hive Compiler: a component that parses the HiveQL queries and generates an execution plan in the form of a directed acyclic graph (DAG) of stages.
 - Hive Optimizer: a component that applies various optimizations to the execution plan, such as predicate pushdown, column pruning, join reordering, etc.
 - Hive Executor: a component that executes the stages of the execution plan using the appropriate execution engine, such as MapReduce, Tez, or Spark.
 - Hive Server: a component that provides a Thrift-based API for clients to connect and submit queries to Hive.
 - Hive CLI: a command-line interface for users to interact with Hive.

- Hive Web UI: a web-based interface for users to monitor and debug the queries running on Hive.

Apache Hive architecture

Apache Hive is a data warehouse system that enables analytics at a massive scale using SQL-like queries. It runs on top of the Hadoop distributed file system (HDFS) and supports various data formats, such as text, ORC, Parquet, JSON, and Avro. Hive can also integrate with other data sources, such as HBase, Cassandra, MongoDB, and JDBC.

The key components of the Apache Hive architecture are:

- **Hive clients:** These are the applications or tools that allow users to interact with Hive, such as the Hive command line interface (CLI), the Hive web interface (HWI), the Hive Beeline shell, or any third-party applications that use the Hive JDBC or ODBC drivers. Users can submit queries and other operations to Hive through these clients.
- **Hive services:** These are the components that process the requests from the Hive clients and communicate with the Hadoop cluster. The main Hive services are:
 - **HiveServer2:** This is the service that accepts incoming requests from the Hive clients and creates an execution plan and auto-generates a YARN job to process SQL queries. HiveServer2 also provides authentication, authorization, and session management for the users and applications.
 - **Hive Metastore:** This is the service that stores the metadata of the tables, partitions, columns, schemas, and other Hive objects. The Hive Metastore can be either embedded within the HiveServer2 process or run as a separate service that can be accessed by multiple HiveServer2 instances. The Hive Metastore uses a relational database, such as MySQL, PostgreSQL, or Oracle, to store the meta-data.
 - **Hive WebHCat:** This is the service that provides a REST API for accessing the Hive functionality. WebHCat allows users to create, drop, and alter tables and partitions, as well as submit Hive queries and Hadoop MapReduce jobs.
- **Processing framework and resource management:** This is the component that executes the queries and jobs submitted by the Hive services and manages the resources of the Hadoop cluster. Hive supports two processing frameworks: MapReduce and Tez. MapReduce is the default framework that runs the queries as a series of map and reduce tasks. Tez is an alternative framework that runs the queries as a directed acyclic graph (DAG) of tasks, which can improve the performance and efficiency

of complex queries. Hive also supports two resource management systems: YARN and Mesos. YARN is the default system that allocates the resources and schedules the tasks for the Hadoop cluster. Mesos is an alternative system that can manage multiple clusters and frameworks.

- **Distributed storage:** This is the component that stores the data of the Hive tables and partitions. Hive uses HDFS as the default storage system, which provides high availability, scalability, and fault tolerance. Hive can also use other storage systems, such as Amazon S3, Azure Blob Storage, or Google Cloud Storage, by using custom SerDes (serializers and deserializers) and InputFormat and OutputFormat classes.

Installing Hive

- Hive is a data warehouse system that runs on top of Hadoop, a distributed file system that stores large amounts of data.
- Hive provides a SQL-like interface to query and analyze data stored in Hadoop.
- To install Hive, you need to have Java, Hadoop and MySQL installed on your system.
- The steps to install Hive are as follows:
 1. Download the latest version of Hive from the Apache website and extract it to a desired location.
 2. Set the environment variables `HIVE_HOME` and `PATH` to point to the Hive directory and its bin subdirectory respectively.
 3. Create a MySQL database and user for Hive metastore, which stores the metadata of Hive tables and partitions.
 4. Copy the MySQL JDBC driver jar file to the Hive lib directory.
 5. Edit the Hive configuration file `hive-site.xml` and set the properties for the metastore connection, such as `javax.jdo.option.ConnectionURL`, `javax.jdo.option.ConnectionDriverName`, `javax.jdo.option.ConnectionUserName` and `javax.jdo.option.ConnectionPassword`.
 6. Initialize the Hive metastore schema by running the command `schematool -initSchema -dbType mysql`.
 7. Start the Hive shell by running the command `hive` and verify that you can execute Hive queries.

Hive shell

- Hive shell is a command-line interface that allows users to interact with Hive and run Hive queries.
- Hive shell can be launched by typing `hive` in the terminal or by specifying a script file with the `-f` option.
- Hive shell supports various options and commands to configure and control the execution of Hive queries. For example, `-e` option can be used to

execute a single query, `-v` option can be used to enable verbose mode, and `set` command can be used to change Hive configuration variables.

- Hive shell also supports some built-in functions and operators to manipulate data and expressions. For example, `concat` function can be used to concatenate strings, `+` operator can be used to add numbers, and `substr` function can be used to extract a substring.
- Hive shell can be used to create, alter, and drop tables, partitions, and views, as well as to load, insert, and query data. Hive shell supports both HiveQL and SQL syntax for most of the operations.
- Hive shell can be connected to different Hive servers, such as HiveServer2 or HiveServer1, by specifying the `-h` or `--hiveconf` options. HiveServer2 is the preferred server as it offers more features and security than HiveServer1.
- Hive shell can also be accessed remotely using SSH clients or Hive Shell service. Hive Shell service is a feature of Hivion OS that provides remote access to workers using the Hivion OS network infrastructure. Hive Shell service also allows console sharing and session management.

Hive services

Hive services are the components that perform client interactions with Hive. They allow users to submit queries and commands to Hive and receive the results. Some of the main Hive services are:

- **HiveServer2:** This is the main service that provides a JDBC/ODBC interface for clients to connect to Hive and execute queries. It also supports multiple concurrent users and sessions, security features, and a web UI for monitoring and debugging.
- **Beeline:** This is a command-line shell that connects to HiveServer2 and allows users to submit queries and commands to Hive. It is based on the SQLLine tool and supports various options and configurations.
- **Metastore:** This is a service that stores the metadata of the tables, partitions, columns, and schemas created in Hive. It also provides a thrift API for other components to access and manipulate the metadata. The metastore can use different backends, such as MySQL, PostgreSQL, or Derby, to store the metadata.
- **WebHCat:** This is a service that provides a REST API for users to submit and monitor Hive jobs. It also supports other Hadoop components, such as Pig and MapReduce. WebHCat acts as a proxy between the clients and the Hadoop cluster, and handles the job submission, configuration, and status checking.
- **Hive CLI:** This is a deprecated service that provides a command-line interface for users to interact with Hive. It is not recommended to use this service, as it has several limitations and issues, such as lack of security, concurrency, and logging. Users should use HiveServer2 and Beeline instead.

Hive metastore

- Hive metastore is a service that stores metadata related to Apache Hive and other services, such as Impala, Spark, etc. in a backend relational database, such as MySQL or PostgreSQL .
- Metadata includes information about the tables, partitions, columns, data types, locations, etc. of the data stored in Hive or other services.
- Hive metastore provides a common interface for clients to access and manipulate the metadata using the metastore service API.
- Hive metastore enables analytics at a massive scale by allowing users to query data from different sources and formats using SQL-like syntax .
- Hive metastore supports storage on various file systems, such as HDFS, S3, ADLS, GS, etc. through Hive.

Comparison of Hive with traditional databases

Hive is a data warehouse software system that provides data query and analysis on large datasets stored in Hadoop. Hive supports a SQL-like interface called HiveQL, which allows users to perform various operations on the data, such as creating tables, loading data, querying, aggregating, and transforming. However, Hive is not a full-fledged database, and it has some differences from traditional relational databases (RDBMS) in terms of architecture, functionality, and performance. Some of the main differences are:

- **Schema on read vs schema on write:** Hive applies schema on read, which means that the data is not validated or enforced when it is loaded into the tables, but only when it is queried. This allows users to store data in various formats and structures, and to change the schema as needed. RDBMS applies schema on write, which means that the data is validated and enforced when it is inserted or updated in the tables, and the schema is fixed and predefined. This ensures data consistency and integrity, but also limits the flexibility and scalability of the data.
- **Scalability and cost:** Hive is very easily scalable at low cost, as it leverages the distributed and parallel processing capabilities of Hadoop. Hive can handle petabytes of data across thousands of nodes, and can scale up or down as needed by adding or removing nodes. RDBMS is not very scalable, as it requires expensive hardware and software to handle large volumes of data, and it faces challenges in distributing and synchronizing the data across multiple servers. RDBMS can scale up by upgrading the hardware, or scale out by partitioning or sharding the data, but both options are costly and complex.
- **Write once, read many vs read and write many:** Hive is based on the Hadoop notion of write once, read many, which means that the data is loaded into the tables once, and then queried multiple times for analysis. Hive does not support record-level updates, insertions, or deletions, as these operations are expensive and inefficient in a distributed system. RDBMS supports read and write many, which means that the data can be

inserted, updated, or deleted at any time, and the changes are reflected immediately. RDBMS supports transactional and operational applications that require frequent and consistent data modifications.

- **Latency and performance:** Hive has a high latency and low performance compared to RDBMS, as it translates the HiveQL queries into MapReduce jobs, which are executed on the Hadoop cluster. MapReduce is a batch processing framework that involves multiple steps, such as splitting, mapping, shuffling, sorting, reducing, and merging, which add to the overhead and delay of the query execution. RDBMS has a low latency and high performance, as it executes the SQL queries directly on the database server, using optimized algorithms and data structures, such as indexes, hash tables, and B-trees, which speed up the query processing.

HiveQL

- HiveQL is a query language for Apache Hive, a data warehouse system for Apache Hadoop.
- HiveQL allows users to process and analyze structured data in a Metastore, which is a central repository of metadata that can be used for data-driven decisions.
- HiveQL provides the basic SQL-like operations, such as SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, JOIN, etc.
- HiveQL also supports some advanced features, such as partitioning, bucketing, windowing, user-defined functions, and subqueries.
- HiveQL separates users from the complexity of MapReduce programming, and reuses common concepts from relational databases, such as tables, rows, columns, and schema, to ease learning.
- HiveQL can also integrate with other query languages, such as Spark SQL, Pig Latin, and Tez, to leverage the benefits of different frameworks and engines.

Tables in Hive

- Tables in Hive are similar to tables in a relational database management system. They store data in columns and rows and belong to a database .
- There are two types of tables in Hive: internal and external .
 - Internal tables: Data is stored in the Hive data warehouse, which is located at /hive/warehouse/ on the default storage for the cluster. Internal tables are also called managed tables because Hive manages their lifecycle and data . When an internal table is dropped, the data and the metadata are deleted . Use internal tables when the data is temporary or exclusive to Hive .
 - External tables: Data is stored outside the Hive data warehouse, on any storage accessible by the cluster. External tables are also called unmanaged tables because Hive does not manage their lifecycle and

data . When an external table is dropped, only the metadata is deleted, but the data remains . Use external tables when the data is shared by other applications or needs to be preserved after dropping the table .

- The general syntax for creating a table in Hive is:

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
[(col_name data_type [COMMENT col_comment], ...)]
[COMMENT table_comment]
[PARTITIONED BY (col_name data_type [COMMENT col_comment], ...)]
[CLUSTERED BY (col_name, col_name, ...) [SORTED BY (col_name [ASC|DESC], ...)] INTO num]
[SKewed BY (col_name, col_name, ...) ON ((col_value, col_value, ...), (col_value, col_v
[
[ROW FORMAT row_format]
[STORED AS file_format]
| STORED BY 'storage.handler.class.name' [WITH SERDEPROPERTIES (...)]
]
[LOCATION hdfs_path]
[TBLPROPERTIES (property_name=property_value, ...)]
[AS select_statement];
```

- An example of creating an internal table in Hive is:

```
CREATE TABLE IF NOT EXISTS employees (
    id INT,
    name STRING,
    salary FLOAT,
    department STRING
)
COMMENT 'This is a table for storing employee data'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

- An example of creating an external table in Hive is:

```
CREATE EXTERNAL TABLE IF NOT EXISTS employees (
    id INT,
    name STRING,
    salary FLOAT,
    department STRING
)
COMMENT 'This is a table for storing employee data'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/hadoop/employees';
```

Querying data in Hive

- Hive is a data warehouse system that allows users to query and analyze large-scale data using a SQL-like language called HiveQL.
- HiveQL is a declarative language that abstracts the complexity of MapReduce jobs and converts queries into a series of MapReduce tasks that run on a Hadoop cluster.
- Hive supports various data formats, such as text, JSON, ORC, Parquet, Avro, etc., and can create external or managed tables to store and access the data.
- Hive also provides a rich set of built-in functions and operators for data manipulation, aggregation, filtering, joining, sorting, etc.
- To query data in Hive, users need to follow these steps:
 - Connect to the Hive server using a command-line interface (CLI), a graphical user interface (GUI), or an application programming interface (API).
 - Create a database and tables to store the data, or use existing ones. Specify the schema, data format, location, and other properties of the tables.
 - Load the data into the tables using the `LOAD DATA` or `INSERT` statements, or use external tables that point to existing data files.
 - Write and execute HiveQL queries to retrieve and analyze the data. Use the `SELECT` statement to specify the columns, conditions, and expressions to query. Use the `FROM` clause to specify the tables or subqueries to query from. Use the `WHERE` clause to filter the rows based on certain criteria. Use the `GROUP BY` clause to group the rows by certain columns and apply aggregate functions. Use the `HAVING` clause to filter the groups based on certain conditions. Use the `ORDER BY` or `SORT BY` clause to sort the rows by certain columns. Use the `LIMIT` clause to limit the number of rows returned. Use the `JOIN` clause to combine data from multiple tables based on certain conditions. Use the `UNION` clause to combine the results of multiple queries. Use the `WITH` clause to create temporary tables or views for reuse. Use the `CREATE TABLE AS SELECT` or `CREATE VIEW AS SELECT` statements to create new tables or views based on the query results.
 - Use the `SHOW`, `DESCRIBE`, or `EXPLAIN` statements to display the meta-data, schema, or execution plan of the tables or queries.
 - Use the `SET` statement to configure the Hive parameters, such as the number of reducers, the compression codec, the output format, etc.
 - Use the `DROP` or `ALTER` statements to delete or modify the tables, databases, or views.

User defined functions in Hive

- User defined functions (UDFs) are custom functions that can be created

and used in Hive queries to perform specific tasks that are not supported by the built-in functions.

- UDFs can be written in Java, Python, or any other scripting language that supports the Hive UDF interface.
- UDFs can be categorized into three types: scalar, aggregate, and table.
- Scalar UDFs take one or more input values and return a single output value. For example, a UDF that converts a string to uppercase.
- Aggregate UDFs take a set of input values and return a single output value. For example, a UDF that calculates the average of a column.
- Table UDFs take one or more input values and return a table of output values. For example, a UDF that splits a string into multiple rows.
- To create a UDF in Hive, the following steps are required:
 - Write the UDF code in the chosen language and compile it into a JAR file (for Java) or a ZIP file (for Python).
 - Add the JAR or ZIP file to the Hive classpath using the `ADD JAR` or `ADD FILE` command.
 - Register the UDF with Hive using the `CREATE FUNCTION` command, specifying the name, class, and type of the UDF.
 - Use the UDF in Hive queries by invoking the name and passing the arguments.
- To drop a UDF in Hive, the `DROP FUNCTION` command can be used.
- To list all the UDFs in Hive, the `SHOW FUNCTIONS` command can be used.

Sorting and aggregating in Hive

- Sorting data in Hive can be achieved by using a standard `ORDER BY` clause, but it has a drawback. `ORDER BY` produces a result that is totally sorted, as expected, but to do so it sets the number of reducers to one, making it very inefficient for large datasets.
- To sort data more efficiently, Hive supports two other clauses: `SORT BY` and `DISTRIBUTE BY`. `SORT BY` sorts the data within each reducer partition, but does not guarantee any global order. `DISTRIBUTE BY` partitions the data by a specified column or expression, but does not sort within each partition. These clauses can be combined to achieve a global order with multiple reducers.
- For example, the following query sorts the data by the `name` column using multiple reducers:

```
SELECT * FROM students
DISTRIBUTE BY name
SORT BY name;
```

- Aggregating data in Hive can be done by using built-in aggregate functions, such as `MAX`, `MIN`, `AVG`, `SUM`, `COUNT`, etc. These functions are usually used with the `GROUP BY` clause, which groups the data by one or more columns or expressions. If there is no `GROUP BY` clause specified, the aggregate functions apply to the whole table by default.

- For example, the following query calculates the average score of each student:

```
SELECT name, AVG(score) AS average_score
FROM students
GROUP BY name;
```

- Hive also supports advanced aggregation by using **GROUPING SETS**, **ROLLUP**, **CUBE**, analytic functions, and windowing. These features allow more complex and flexible grouping and aggregation operations.
- For example, the following query calculates the total score of each student, each subject, and the grand total using **GROUPING SETS**:

```
SELECT name, subject, SUM(score) AS total_score
FROM students
GROUP BY GROUPING SETS ((name, subject), (name), (subject), ());
```

- To aggregate strings in Hive, one can use the **collect_list** or **collect_set** functions, which return an array of strings for each group. The difference between them is that **collect_set** removes duplicates, while **collect_list** preserves them. To order the strings within each array, one can use the **sort_array** function.
- For example, the following query collects and sorts the subjects of each student:

```
SELECT name, sort_array(collect_list(subject)) AS subjects
FROM students
GROUP BY name;
```

Map Reduce scripts in Hive

- Map Reduce scripts in Hive are used to plug in custom mappers and reducers in the data stream by using the **TRANSFORM** clause in the Hive language .
- The **TRANSFORM** clause allows the user to specify an executable script that can process the input records and generate the output records .
- The script can be written in any language that can read from standard input and write to standard output, such as Python, Ruby, Perl, etc .
- The syntax of the **TRANSFORM** clause is as follows:

```
SELECT TRANSFORM (input_columns)
USING 'script'
AS output_columns
FROM table
```

- The **input_columns** are the columns from the table that are passed to the script as tab-separated values .
- The script is the path to the executable script file that can be either local or on HDFS .

- The output_columns are the columns that are returned by the script as tab-separated values .
- The TRANSFORM clause can be used in both the map and the reduce phases of the Map Reduce job .
- For example, to run a custom mapper script - map_script - and a custom reducer script - reduce_script - the user can issue the following command:

```
FROM (
  FROM table
  MAP input_columns
  USING 'map_script'
  AS map_output_columns
  CLUSTER BY map_output_columns
) map_output
REDUCE map_output_columns
USING 'reduce_script'
AS reduce_output_columns;
```

- The MAP keyword indicates that the script is used as a mapper, and the REDUCE keyword indicates that the script is used as a reducer .
- The CLUSTER BY clause is used to partition the mapper output by the key columns .
- The map_output_columns and the reduce_output_columns must have the same number and type of columns .
- Map Reduce scripts in Hive can be useful for performing complex data transformations that are not supported by the built-in Hive functions or UDFs.
- Map Reduce scripts in Hive can also be used to integrate with external libraries or tools that can provide additional functionality or performance.

Joins and Subqueries in Hive

- Joins are used to combine data from two or more tables based on a common column or condition.
- Subqueries are queries that are nested inside another query, usually in the FROM or WHERE clause.
- Hive supports different types of joins, such as inner join, left outer join, right outer join, full outer join, and cross join.
- Hive also supports different types of subqueries, such as scalar subquery, IN subquery, EXISTS subquery, and correlated subquery.
- Joins and subqueries can be used together to perform complex queries on Hive tables.

Some examples of joins and subqueries in Hive are:

- Inner join: This join returns the records that are common to both tables based on the join condition.

```
SELECT a.col1, b.col2 FROM table1 a JOIN table2 b ON a.id = b.id;
```

- Left outer join: This join returns all the records from the left table and the matching records from the right table. If there is no match, the right table columns are filled with NULL values.

```
SELECT a.col1, b.col2 FROM table1 a LEFT JOIN table2 b ON a.id = b.id;
```

- Right outer join: This join returns all the records from the right table and the matching records from the left table. If there is no match, the left table columns are filled with NULL values.

```
SELECT a.col1, b.col2 FROM table1 a RIGHT JOIN table2 b ON a.id = b.id;
```

- Full outer join: This join returns all the records from both tables, regardless of the match. If there is no match, the corresponding table columns are filled with NULL values.

```
SELECT a.col1, b.col2 FROM table1 a FULL JOIN table2 b ON a.id = b.id;
```

- Cross join: This join returns the Cartesian product of the two tables, i.e., every row of the left table is joined with every row of the right table.

```
SELECT a.col1, b.col2 FROM table1 a CROSS JOIN table2 b;
```

- Scalar subquery: This subquery returns a single value that can be used in the SELECT, WHERE, or HAVING clause of the outer query.

```
SELECT col1, (SELECT MAX(col2) FROM table2) AS max_col2 FROM table1;
```

- IN subquery: This subquery returns a set of values that can be used with the IN operator in the WHERE or HAVING clause of the outer query.

```
SELECT col1, col2 FROM table1 WHERE col1 IN (SELECT col3 FROM table2);
```

- EXISTS subquery: This subquery returns a boolean value that can be used with the EXISTS operator in the WHERE or HAVING clause of the outer query. The subquery is usually correlated with the outer query, i.e., it references some columns from the outer query.

```
SELECT col1, col2 FROM table1 a WHERE EXISTS (SELECT 1 FROM table2 b WHERE a.id = b.id);
```

- Correlated subquery: This subquery is a type of subquery that depends on the outer query for its values. It cannot be executed independently and is usually used with comparison operators in the WHERE or HAVING clause of the outer query.

```
SELECT col1, col2 FROM table1 a WHERE col2 > (SELECT AVG(col2) FROM table2 b WHERE a.id = b.id);
```

HBase

- HBase is a **column-oriented non-relational database management system** that runs on top of **Hadoop Distributed File System (HDFS)**.

- HBase provides a **fault-tolerant** way of storing **sparse data sets**, which are common in many big data use cases .
- HBase is well suited for **real-time data processing** or **random read/write access** to large volumes of data .
- HBase is modeled after **Google’s Bigtable**, a distributed storage system for structured data .
- HBase does not have a **fixed database schema** in a non-relational database, which means developers can add new data without conforming to a schema model.
- HBase is ideal for **high-scale real-time applications**, such as a social media app or a streaming application.
- HBase is an **open-source** project developed as part of **Apache Software Foundation’s Apache Hadoop project** .
- HBase can also run on top of **Alluxio**, a distributed in-memory file system.

HBase concepts

- HBase is a type of NoSQL database and is classified as a key-value store.
- HBase is a distributed column-oriented database built on top of the Hadoop file system (HDFS) .
- HBase is an open-source project and is horizontally scalable.
- HBase is a data model that is similar to Google’s big table designed to provide quick random access to huge amounts of structured data .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases.
- HBase is well suited for real-time data processing or random read/write access to large volumes of data.
- HBase has a master node that manages the cluster and region servers that store portions of the tables and perform the work on the data.
- HBase has a schema design that consists of tables, rows, columns, column families, and cells.
- HBase supports CRUD operations, filters, scanners, and coprocessors for data manipulation and processing.
- HBase also supports secondary indexes, replication, snapshots, and integration with other Hadoop components such as MapReduce, Hive, and Spark.

HBase clients

- HBase clients are applications that can interact with HBase using different programming languages and frameworks.
- HBase provides a native Java client library that exposes the HBase API and allows users to perform CRUD operations, scan tables, create and delete tables, and administer the cluster.
- HBase also supports other clients that use different protocols and inter-

faces, such as REST, Thrift, and Avro. These clients can be used by non-Java applications to access HBase data and functionality.

- Some examples of HBase clients are:
 - HBase Shell: A command-line interface that uses JRuby to execute HBase commands and scripts.
 - HBase Web UI: A web-based interface that provides information and statistics about the HBase cluster, tables, regions, and compactions.
 - Apache Phoenix: A SQL layer that enables users to query and manipulate HBase data using standard SQL syntax and JDBC drivers.
 - Apache Spark: A distributed computing framework that can read and write HBase data using Spark SQL, DataFrames, and RDDs.
 - Apache Hive: A data warehouse system that can access HBase data using HiveQL, a SQL-like language.
 - Apache Pig: A data analysis platform that can load and store HBase data using Pig Latin, a scripting language.
 - Apache Flume: A distributed service that can collect and transport data from various sources to HBase using Flume agents and sinks.
 - Apache Kafka: A distributed messaging system that can produce and consume data from HBase using Kafka producers and consumers.

HBase example

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- An HBase table consists of rows and columns, where each column belongs to a column family .
- An HBase column represents an attribute of an object; for example, if the table is storing diagnostic logs from servers, each row might be a log record, and a typical column could be the timestamp of when the log record was written, or the server name where the record originated .
- HBase supports CRUD (create, read, update, delete) operations, as well as scan and filter operations, using a shell command or a Java API .
- HBase also supports secondary indexes, coprocessors, replication, and snapshots .

Here is an example of how to create a table in HBase with the specified name and column family using the shell command :

```
hbase (main):001:0> create 'education','guru99'
0 rows (s) in 0.312 seconds
=>Hbase::Table - education
```

The above example explains how to create a table in HBase with the name ‘education’ and the column family ‘guru99’.

Some of the use cases of HBase are :

- In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area.
- In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights.
- In sports, HBase is used to store match details and the history of each match.

HBase vs RDBMS

HBase and RDBMS are both types of database management systems, but they differ in several ways:

- **Data Model:** RDBMS uses a relational data model, where data is stored in tables with predefined columns and rows. HBase, on the other hand, uses a column-family data model, where data is stored in column families, which contain columns and rows. HBase is often referred to as a NoSQL database because of its non-relational data model .
- **Scaling:** RDBMS scales vertically, which means adding more resources to a single server to handle more load. HBase scales horizontally, which means adding more servers to a cluster to distribute the load. HBase is designed to handle large amounts of data across multiple nodes in a distributed system .
- **Consistency:** RDBMS provides strong consistency, which means that all the clients see the same view of the data at any given time. HBase provides eventual consistency, which means that the clients may see different views of the data for a short period of time until the data is replicated across the nodes .
- **Speed:** RDBMS is optimized for fast and complex queries that involve joins, aggregations, and transactions. HBase is optimized for fast and simple queries that involve key-value lookups, scans, and updates. HBase can handle millions of operations per second on large data sets .
- **ACID Compliance:** RDBMS is ACID compliant, which means that it guarantees the properties of atomicity, consistency, isolation, and durability for each transaction. HBase is not ACID compliant, but it supports atomic and consistent operations at the row level. HBase also supports snapshots and backups for data recovery .

HBase and RDBMS have different use cases and trade-offs. RDBMS is more suitable for traditional, transactional applications that require strong consistency, whereas HBase is better suited for big data applications that require horizontal scaling and high-speed processing .

Advanced usage of HBase

HBase is a distributed, scalable, and column-oriented database that runs on top of the Hadoop Distributed File System (HDFS). It is designed to store and process large amounts of data in a fault-tolerant and consistent way. HBase can be used for various applications that require fast and random access to data, such as:

- Storing and querying genome sequences and disease history in the health-care sector .
- Storing and analyzing customer search history and target advertisement in the e-commerce sector.
- Storing and predicting match details and outcomes in the sports sector.
- Storing and processing logs, metrics, and events in the web and social media sector.
- Storing and retrieving documents, images, and videos in the media and entertainment sector.

HBase has two fundamental key structures: the row key and the column key. Both can be used to convey meaning, by either the data they store, or by exploiting their sorting order. In the following sections, we will discuss some advanced usage of HBase keys to solve commonly found problems when designing storage solutions.

- **Composite row keys:** A composite row key is a row key that consists of multiple parts, separated by a delimiter, such as a colon or a dash. For example, a composite row key for a customer table could be `customer_id:order_id`. Composite row keys can be used to create a hierarchical structure of data, such as grouping related rows together, or to enable range scans on a subset of the row key, such as scanning all the orders of a given customer.
- **Reverse row keys:** A reverse row key is a row key that is reversed in order, such as `cba` instead of `abc`. Reverse row keys can be used to avoid hotspots, which are regions of the table that receive a disproportionate amount of read or write requests, causing performance degradation. For example, if the row key is a timestamp, then the most recent rows will be clustered at the end of the table, creating a hotspot. By reversing the timestamp, the most recent rows will be distributed across the table, avoiding the hotspot.
- **Salted row keys:** A salted row key is a row key that is prefixed with a random value, such as `1abc` or `2abc`. Salted row keys can also be used to avoid hotspots, especially when the row key has a low cardinality, meaning that there are few distinct values. For example, if the row key is a country code, then there will be only a few regions of the table that will receive most of the requests, creating hotspots. By adding a random prefix, the row keys will be hashed to different regions, avoiding the hotspots.
- **Column families and qualifiers:** A column family is a logical grouping of columns that share the same characteristics, such as compression,

encoding, and versioning. A column qualifier is a name that identifies a column within a column family. For example, a column family **info** could have column qualifiers **name**, **age**, and **gender**. Column families and qualifiers can be used to store different types of data, such as structured, semi-structured, or unstructured data, or to store different versions or aspects of the same data, such as historical, current, or future data.

Schema design in HBase

- HBase is a NoSQL database that does not support joins or secondary indexes, but provides fast and scalable access to data by using a single primary key.
- HBase schema design is based on the following concepts:
 - Row key: This is the unique identifier for each row in a table. It is used to sort and locate the data. It should be carefully chosen to avoid hotspots and ensure even distribution of data across regions.
 - Column family: This is a logical grouping of columns that share some common characteristics, such as compression, encoding, or versioning. Each column family is stored separately on disk, so it is advisable to limit the number of column families per table to avoid performance degradation.
 - Column qualifier: This is the name of a column within a column family. It can be dynamic and vary from row to row. It can also be used to store nested entities or complex data types by concatenating multiple values or attributes.
 - Cell: This is the smallest unit of data in HBase. It consists of a row key, a column family, a column qualifier, a timestamp, and a value. The value can be any binary data, such as a string, a number, or an image.
 - Timestamp: This is the time when a cell was inserted or updated. It can be either assigned by the client or generated by the server. It is used to keep track of multiple versions of a cell and to perform time-based queries or scans.
- HBase schema design supports denormalization and wide tables, which means that data can be duplicated or stored in multiple columns or rows to optimize read performance and avoid joins. However, this also increases the write and storage costs and may introduce inconsistency issues.
- HBase schema design also supports filters, which are predicates that can be applied to a scan or a get operation to limit the results or to perform some transformations on the data. Filters can improve the read performance by reducing the amount of data transferred from the server to the client. Some common filters are:
 - PrefixFilter: This filter matches rows that have a row key that starts with a given prefix.
 - ColumnPrefixFilter: This filter matches columns that have a column qualifier that starts with a given prefix.

- `SingleColumnValueFilter`: This filter matches rows that have a specific value in a given column.
- `PageFilter`: This filter limits the number of rows returned by a scan operation.

Advanced Indexing in HBase

- HBase is a distributed, column-oriented database that stores data in a sorted order by the row key.
- HBase does not support secondary indexes, which are indexes on columns other than the row key.
- Secondary indexes can improve the performance of queries that filter or join on non-row key columns, but they also introduce challenges such as consistency, scalability, and maintenance.
- There are different approaches to implement secondary indexing in HBase, such as:
 - **Client-side indexing**: The application maintains a separate index table for each secondary index column, and updates it along with the main table. This approach requires extra logic and coordination in the application, and may cause inconsistency or stale data if the updates fail or are delayed.
 - **Coprocessor-based indexing**: The HBase server uses a coprocessor, which is a custom code that runs on the server side, to create and update the index table automatically. This approach reduces the network overhead and the application complexity, but it may affect the performance and availability of the HBase cluster.
 - **External indexing**: The HBase server uses an external system, such as Solr or Elasticsearch, to index the data and provide search capabilities. This approach leverages the features and scalability of the external system, but it may introduce latency and inconsistency between the HBase data and the index data.
- Each approach has its own advantages and disadvantages, and the choice depends on the use case and the trade-offs.
- Some of the factors to consider when choosing an indexing approach are:
 - **Query pattern**: The type and frequency of queries that need to use the secondary index, such as point queries, range queries, or full-text search.
 - **Data size and growth**: The amount and rate of data that needs to be indexed, and the impact on the storage and network resources.
 - **Data freshness and consistency**: The acceptable delay and divergence between the main table and the index table, and the impact on the query results and the user experience.

- **Index maintenance and management:** The complexity and overhead of creating, updating, and deleting the index table, and the impact on the HBase cluster and the application.
- HBase provides some tools and frameworks to help with the implementation and verification of secondary indexing, such as:
 - **HBase Indexer:** A service that uses Apache Lily to index HBase data into Solr in near real-time, using Morphline parsers and ZooKeeper coordination.
 - **Phoenix:** A SQL layer on top of HBase that supports secondary indexing using coprocessors and transactions.
 - **Index Scrutiny Tool:** A tool that compares the data between the main table and the index table, and reports any discrepancies or errors.

Zookeeper

A zookeeper is a person who manages zoo animals that are kept in captivity for conservation or to be displayed to the public. Some of the tasks and responsibilities of a zookeeper are:

- Feeding and watering the animals according to their dietary needs and schedules.
- Cleaning the exhibits and enclosures and maintaining hygiene and sanitation standards.
- Observing and monitoring the health and behavior of the animals and reporting any issues or concerns to veterinarians or supervisors.
- Enriching the animals' environment and stimulating their natural behaviors.
- Educating and interacting with the visitors and answering their questions about the animals and the zoo.
- Participating in research, conservation, and breeding programs.

A zookeeper may work with a variety of animals, such as mammals, birds, reptiles, amphibians, fish, and invertebrates. They may also specialize in a specific group of animals, such as primates, carnivores, or marine animals.

To become a zookeeper, one usually needs a bachelor's degree in zoology, biology, wildlife management, animal science, or a related field. Some zoos may also require previous experience in working with animals, such as volunteering, interning, or working at a pet store, farm, or wildlife rehabilitation center. Additionally, a zookeeper should have the following skills and qualities:

- A passion and respect for animals and their welfare.
- A good knowledge of animal behavior, anatomy, physiology, nutrition, and diseases.
- A physical stamina and ability to work in various weather conditions and handle heavy equipment and animals.

- A communication and interpersonal skills and ability to work in a team and with the public.
- A problem-solving and critical thinking skills and ability to handle emergencies and stressful situations.

A zookeeper may face some challenges and risks in their job, such as:

- Exposure to zoonotic diseases, parasites, bites, scratches, and allergens from the animals.
- Emotional stress from dealing with injured, sick, or dying animals.
- Ethical dilemmas from keeping animals in captivity and balancing their needs and interests with the zoo's goals and policies.
- Low pay and long hours, especially during weekends and holidays.

A zookeeper may also enjoy some benefits and rewards in their job, such as:

- Developing a close and meaningful bond with the animals and witnessing their growth and development.
- Contributing to the conservation and education of endangered and threatened species.
- Learning new and interesting facts and skills about the animals and their habitats.
- Working in a dynamic and diverse environment and meeting people from different backgrounds and cultures.

Zookeeper concepts

- Zookeeper is a distributed application that provides coordination services for distributed systems, such as naming, configuration management, synchronization, and group services .
- Zookeeper has a simple client-server model, where clients are nodes (machines) that use the services, and servers are nodes that provide the services.
- Zookeeper uses a hierarchical namespace, similar to a file system, to store data and metadata in znodes (zookeeper nodes) . Znodes can be persistent, ephemeral, or sequential.
- Zookeeper guarantees that the data and operations are consistent, ordered, and atomic across the cluster . Zookeeper also ensures that the clients always see the most recent data.
- Zookeeper can operate in standalone mode, with a single server, or in replicated mode, with a quorum of servers . In replicated mode, one server acts as the leader and the others as followers .
- Zookeeper uses a consensus protocol, such as Zab or Raft, to elect a leader and synchronize the state of the servers . The leader is responsible for coordinating the requests from the clients and broadcasting the updates to the followers .
- Zookeeper is designed to be reliable, scalable, and fast . Zookeeper can handle thousands of concurrent clients and millions of znodes. Zookeeper

can also tolerate failures and recover from them .

How ZooKeeper helps in monitoring a cluster

- ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services for distributed applications.
- ZooKeeper hosts are deployed in a cluster and, as long as a majority of hosts are up, the service will be available.
- ZooKeeper provides a simple and consistent interface to access the data stored in the cluster, which can be used by various monitoring tools and frameworks.
- ZooKeeper exposes several metrics and statistics that can be used to monitor the health and performance of the cluster, such as:
 - The number and status of nodes (znodes) in the ZooKeeper tree.
 - The number and state of connections, sessions, and watches.
 - The latency, throughput, and load of requests and responses.
 - The memory and CPU usage of the ZooKeeper processes.
 - The leader election and quorum formation events.
- ZooKeeper also provides several commands and APIs that can be used to query and manipulate the cluster, such as:
 - The **stat** command, which returns the mode, version, and statistics of a ZooKeeper server.
 - The **ruok** command, which checks if a ZooKeeper server is running.
 - The **conf** command, which returns the configuration of a ZooKeeper server.
 - The **mntr** command, which returns a list of key-value pairs of metrics.
 - The **wchp** and **wchc** commands, which return the watches set by sessions on paths or by paths on sessions.
 - The **cons** command, which returns connection details of all the clients connected to the server.
 - The **envi** command, which returns details of the server environment.
 - The **isro** command, which checks if the server is in read-only mode.
 - The **crst** command, which resets connection/session statistics for all connections.
 - The **srvr** command, which returns server information and statistics similar to the **stat** command.
 - The **srst** command, which resets server statistics.
 - The **dump** command, which lists the outstanding sessions and ephemeral nodes.
 - The **reqs** command, which lists the last 10 processed requests.
 - The **setTraceMask** command, which sets the trace mask for logging.
 - The **getTraceMask** command, which gets the trace mask for logging.
 - The **addauth** command, which adds an authentication scheme and associated data to the connection.
 - The **reconfig** command, which changes the cluster membership.

- The `getConfig` command, which returns the current configuration of the cluster.
- The `closeSession` command, which closes a session and removes its ephemeral nodes.
- The `history` command, which lists the last 10 commands executed on the server.
- The `redo` command, which executes a command from the history again.
- The `printwatches` command, which prints the watches on the server.
- The `deleteall` command, which deletes a node and all its children.
- The `delquota` command, which removes the quota for a node or a subtree.
- The `setquota` command, which sets the quota for a node or a subtree.
- The `listquota` command, which lists the quota for a node or a subtree.
- The `sync` command, which syncs with the leader.
- The `get` command, which gets the data and stat of a node.
- The `create` command, which creates a node with the given data.
- The `set` command, which sets the data of a node.
- The `delete` command, which deletes a node.
- The `ls` command, which lists the children of a node.
- The `exists` command, which checks the existence of a node and sets a watch^[4]

How to build applications with Zookeeper

ZooKeeper is a distributed system coordinator that provides services such as configuration management, synchronization, naming, and leader election. It can be used to build reliable and scalable distributed applications. Here are some steps to build applications with ZooKeeper:

- Install ZooKeeper on one or more servers. You can download a stable ZooKeeper release from the official website and unpack it. You need to create a configuration file that specifies the server ID, data directory, client port, and other parameters. You can use the sample configuration file `zoo_sample.cfg` as a template. If you have multiple servers, you need to assign a unique ID to each one and list all the servers in the configuration file. For example:

```
server.1=zoo1.example.com:2888:3888
server.2=zoo2.example.com:2888:3888
server.3=zoo3.example.com:2888:3888
```

- Start ZooKeeper on each server. You can use the script `zkServer.sh` (Linux) or `zkServer.cmd` (Windows) to start, stop, or restart the server. For example, to start the server on Linux, run:

```
bin/zkServer.sh start
```

- Connect to ZooKeeper from your application. You need to use a ZooKeeper client library that supports your programming language. For example, if you are using Java, you can use the `org.apache.zookeeper.ZooKeeper` class to create a ZooKeeper object and connect to a server or a cluster of servers. You need to provide a connection string that lists the host and port of the servers, separated by commas. For example:

```
ZooKeeper zk = new ZooKeeper("zoo1.example.com:2181,zoo2.example.com:2181,zoo3.example.com:2181,zoo4.example.com:2181", 5000, null);
```

- Use ZooKeeper to implement your application logic. You can use the ZooKeeper API to create, read, update, and delete nodes (called znodes) in the ZooKeeper data tree. You can also set watches on znodes to get notified of changes. You can use znodes to store configuration data, coordinate tasks, elect leaders, and implement other distributed features. For example, to create a znode named `/app/config` with some data, run:

```
zk.create("/app/config", "some data".getBytes(), Ids.OPEN_ACL_UNSAFE, CreateMode.PERSISTENT);
```

- Test and debug your application. You can use the ZooKeeper command-line interface (CLI) to interact with ZooKeeper and inspect the data tree. You can also use the ZooKeeper JMX interface to monitor the performance and health of the ZooKeeper servers. You can use the `zkCli.sh` (Linux) or `zkCli.cmd` (Windows) script to launch the CLI. For example, to list the children of the root znode, run:

```
ls /
```

- Deploy and scale your application. You can use tools such as Kubernetes, Docker, or Ansible to deploy and manage your ZooKeeper servers and your application components. You can also use the ZooKeeper reconfiguration feature to dynamically add or remove servers from the cluster without downtime. You can use the `reconfig` command in the CLI to perform reconfiguration. For example, to add a new server with ID 4, run:

```
reconfig -add server.4=zoo4.example.com:2888:3888
```

IBM Big Data strategy

- IBM is a US-based computer hardware and software manufacturer that has implemented a Big Data strategy to offer solutions to store, manage, and analyze the huge amounts of data generated daily and equip large and small companies to make informed business decisions .
- IBM's Big Data strategy is aligned with its Smarter Planet initiative, which seeks to highlight how government and business leaders can capture the potential of smarter systems to achieve economic and sustainable growth and societal progress.
- IBM's Big Data strategy consists of six steps:

- Understand the business objectives and connect the data strategy with the business strategy.
- Assess the current state and identify the pain points, blockers, and gaps in the data landscape.
- Map out the data strategy framework and define the target state, vision, and roadmap for the data transformation.
- Design the data architecture and select the appropriate data platforms, tools, and technologies to support the data strategy.
- Implement the data strategy and execute the data projects, initiatives, and use cases that deliver value and insights.
- Govern and manage the data strategy and ensure the data quality, security, privacy, and compliance are maintained throughout the data lifecycle.
- IBM's Big Data strategy leverages its expertise and offerings in cloud, AI, and open source technologies to help its clients create enterprise-grade, secure, governed, and scalable data lakes and data warehouses that can handle structured, semi-structured, and unstructured data from various sources and formats.

IBM Big Data strategy

- IBM Big Data strategy is a corporate initiative that aims to provide solutions for storing, managing, and analyzing the large volumes of data generated by various sources, such as social media, sensors, mobile devices, etc.
- IBM Big Data strategy is aligned with its Smarter Planet vision, which seeks to leverage smarter systems and technologies to achieve economic and social progress.
- IBM Big Data strategy consists of four key components: Big Data platform, Big Data services, Big Data solutions, and Big Data ecosystem.
- Big Data platform: IBM offers a comprehensive and integrated platform that supports various types of data, such as structured, unstructured, and streaming data, and enables data ingestion, processing, storage, and analysis. The platform includes products such as IBM InfoSphere BigInsights, IBM InfoSphere Streams, IBM PureData System for Analytics, IBM PureData System for Hadoop, IBM PureData System for Operational Analytics, IBM DB2 with BLU Acceleration, IBM Cloud Pak for Data, etc.
- Big Data services: IBM provides a range of services that help clients design, implement, and optimize their Big Data strategy, such as IBM Big Data and Analytics Services, IBM Big Data and Analytics Cloud Services, IBM Big Data and Analytics Managed Services, IBM Big Data and Analytics Consulting Services, etc.
- Big Data solutions: IBM delivers industry-specific and cross-industry solutions that address various business challenges and opportunities, such as customer analytics, fraud detection, risk management, operational efficiency, etc. The solutions include products such as IBM Cognos Analyt-

ics, IBM SPSS, IBM Watson, IBM Watson Studio, IBM Watson Machine Learning, IBM Watson OpenScale, IBM Watson Discovery, IBM Watson Assistant, etc.

- Big Data ecosystem: IBM collaborates with a network of partners, such as technology providers, solution providers, academic institutions, and research organizations, to enhance its Big Data capabilities and offerings, and to create value for its clients and stakeholders. Some of the partners include Cloudera, Hortonworks, MapR, MongoDB, Apache, SAS, SAP, Oracle, Microsoft, etc.

Introduction to Infosphere

- Infosphere is a term that combines information and sphere, and refers to an environment that is populated by informational entities called inforgs .
- Infosphere can be understood as the whole system of services and documents, encoded in any semiotic and physical media, whose contents include any sort of data, information and knowledge.
- Infosphere is not limited to purely online environments, but can also include offline and hybrid ones, such as libraries, archives, museums, etc.
- Infosphere is a metaphysical realm of information, data, knowledge and communication, where inforgs interact with each other and with other agents, such as humans, animals, plants, etc.
- Infosphere is also a term used by IBM to refer to a set of products and solutions that provide data integration, data quality, data governance, master data management and data analytics capabilities .
- Infosphere is a concept that can help us understand the nature, value and impact of information in the contemporary world, and the ethical and social implications of living in an information-rich environment.

Introduction to BigInsights

- BigInsights is a product and service offering from IBM that helps organizations to analyze and manage large volumes, varieties and velocities of data using open source technologies such as Hadoop and Spark.
- BigInsights consists of two main components: IBM Open Platform (IOP) with Apache Hadoop and IBM BigInsights Value Add (BIVA) modules.
- IOP is a distribution of Apache Hadoop and related projects that provides the core functionality of data storage, processing and governance. IOP includes Apache Spark, Apache HBase, Apache Hive, Apache Pig, Apache Sqoop, Apache Flume, Apache Oozie, Apache ZooKeeper and more.
- BIVA is a set of IBM-developed modules that extend the capabilities of IOP and provide additional features such as advanced analytics, data exploration, data integration, data security and data management. BIVA includes Big SQL, BigSheets, Text Analytics, Big R, Spectrum Scale, Platform Symphony and BigInsights Home.

- BigInsights can be deployed on-premises, on cloud or in a hybrid mode. BigInsights on Cloud is a fully managed service that offers a scalable and flexible platform for data analysis and application development. BigInsights on Cloud also provides access to IBM Watson Analytics, IBM Data Science Experience and IBM Bluemix services.
- BigInsights can help organizations to gain insights from various types of data such as structured, unstructured, semi-structured, streaming, geospatial, social media, sensor and machine data. BigInsights can also support various use cases across industries such as financial services, telecommunications, government, manufacturing, retail, healthcare and more.

Introduction to Big Sheets

Big Sheets can refer to different types of large-sized sheets, such as:

- Oversized bed sheets: These are any sheets that come in bigger dimensions than standard ones. They can be made for an extra large king, queen, full, or twin bed. They are useful for huge mattresses that need more coverage and comfort.
- Large plexiglass sheets: These are sheets of acrylic plastic that are sold and fabricated in XL and XXL sizes, up to 100"x150" or more. They are used for various applications, such as windows, signs, displays, aquariums, etc.
- Extra large bath sheets: These are towels that are larger than regular bath towels, usually measuring 35"x70" or more. They are ideal for wrapping around the body after a shower or a bath.
- Large sheet format graph paper: These are sheets of paper that have a grid of lines printed on them, usually in 1/4" or 1/5" increments. They are used for drawing, plotting, sketching, etc.

Big Sheets can be purchased from various online or offline stores, such as Dunelm, Garnet Hill, Walmart, Etsy, Amazon, etc. They can vary in price, quality, material, color, and design depending on the type and brand of the sheet.

Introduction to Big SQL

- Big SQL is a high performance massively parallel processing (MPP) SQL engine for Hadoop that makes querying enterprise data from across the organization an easy and secure experience .
- Big SQL can quickly access a variety of data sources including HDFS, RDBMS, NoSQL databases, object stores, and WebHDFS by using a single database connection or query .
- Big SQL is built on the world class IBM common SQL database technology and has all the standard RDBMS features such as transactions, concurrency, security, and optimization.
- Big SQL supports the ANSI SQL standard and provides compatibility with many SQL dialects such as Oracle, DB2, and Netezza.

- Big SQL provides connectivity for some applications through either a 32-bit or a 64-bit ODBC driver, on either Linux or Windows, that conforms to the Microsoft Open Database Connectivity 3.0.0 specification.
- Big SQL also provides a web-based user interface, a command line tool, and a REST API for executing and managing SQL queries.